

Final Program

Makefile

```
TARGET = main
OBS = main.o graphics.o framebuffer.o

CFLAGS = -O2 -G0 -Wall
CXXFLAGS = $(CFLAGS) -fno-exceptions -fno-rtti
ASFLAGS = $(CFLAGS)

LIBDIR =
LIBS = -lpspgu -lpng -lz -lm
LDFLAGS =

EXTRA_TARGETS = EBOOT.PBP
PSP_EBOOT_TITLE = DijkstraGPS

PSPSDK=$(shell psp-config --pspsdk-path)
include $(PSPSDK)/lib/build.mak
```

=====

15

16

17

=====

=====

=====

=====

=====

18

=====

=====

=====

=====

=====

=====

```

/*
    Dru Sepulveda
    September, 9 2006
    Picture Display Program
    Adapted from Lesson 4
*/
#include <pspkernel.h>
#include <pspdebug.h>
#include <pspdisplay.h>
#include <pspctrl.h>
#include <pspgu.h>
#include <psppower.h>
#include <png.h>
#include <stdio.h>
#include "graphics.h"    // prewritten graphics file
#include "Index.h"       // my organizing file

PSP_MODULE_INFO("Dijkstra GPS Map ", 0, 1, 1);

#define printf pspDebugScreenPrintf    // printf defined to make debug printing easier
#define MAX(X, Y) ((X) > (Y) ? (X) : (Y)) // function defined for non-existing operations

/* Exit callback */
int exit_callback(int arg1, int arg2, void *common) {
    sceKernelExitGame();
    return 0;
}

/* Callback thread */
int CallbackThread(SceSize args, void *argp) {
    int cbid;

    cbid = sceKernelCreateCallback("Exit Callback", exit_callback, NULL);
    sceKernelRegisterExitCallback(cbid);

    sceKernelSleepThreadCB();

    return 0;
}

/* Sets up the callback thread and returns its thread id */
int SetupCallbacks(void) {
    int thid = 0;

    thid = sceKernelCreateThread("update_thread", CallbackThread, 0x11, 0xFA0, 0, 0);
    if(thid >= 0) {
        sceKernelStartThread(thid, 0, 0);
    }

    return thid;
}

int main() {
    char buffer[200];    //character buffer to store the image "ourimage" which is the map
    Image* ourImage;     //image type evoked by the graphics.h file

    char buffer2[200];  //character buffer to store the image "splash" which is the splash screen
    Image* splash;      //image type evoked by the graphics.h file

```



```

    int i = 0;                                //counter for the splash screen timer

    int U_Start = 0;                          //main.c Starting position for the user which is set to 0 ✓
    = "YOU" in mapPoints
    int U_Finish = 21;                        //main.c Ending position for the user which is set to 21 ✓
    = "Rusle West" in mapPoints

    int joyX = 128;
    int joyY = 128;

    pspDebugScreenInit();                    //initialize PSP screen to blank setting default ✓
    paramater
    SetupCallbacks();                        //sets up the thread to run the program on
    initGraphics();                          //initilizes the graphic.h file

    SceCtrlData pad;                          //sets the variable pad to type ✓
    SceStrlData to capture buttons into
    sceCtrlSetSamplingCycle(0);              //sets the sampling cycle for the ✓
    joy stick
    sceCtrlSetSamplingMode(PSP_CTRL_MODE_ANALOG); //is necessary to capture the joy ✓
    stick

    sprintf(buffer, "ourImage.png");          //loads the image file into the ✓
    char buffer
    ourImage = loadImage(buffer);             //sets the image file stored in the ✓
    buffer to the image variable

    sprintf(buffer2, "splash.png");          //loads the image file into the ✓
    char buffer
    splash = loadImage(buffer2);              //sets the image file stored in the ✓
    buffer to the image variable

    changePositionManual(U_Start, U_Finish); //Feed the interface.h file with the ✓
    positions 0 and 21
    HeadInit();                             //HeadInit from the index.h file

    if (!ourImage || !splash) {              // checks to see if the pictures have ✓
        loaded.
        //Image load failed
        printf("Image load failed!\n");      // error message if not
    }
    else {                                    // if pictures have loaded
        for(i=0;i<1000;i++){                //timer to show ✓
            splash screen
            blitAlphaImageToScreen(0,0,480,272,splash,0,0); //puts the ✓
            picture on the PSP's screen at position x=0,y=0
            flipScreen();                    //flips to ✓
            the other memory, is necessary to update screen
        }
        ///////////////////////////////////10-31-06@2:22PM-START///////////////////////////////// ✓
        ///////////////////////////////////
        while(1) {

            if(pad.Lx < 100){
                if(joyX>0){
                    joyX=joyX-1;
                    for(i=0; i<5; i++) {
                        // ✓
                    }
                    buisy wait to prevent over pressing
                    sceDisplayWaitVblankStart(); // ✓
                    // blanks screen behind the map
                }
            }
            else if(pad.Lx > 155){
                if(joyX<480){
                    joyX=joyX+1;

```




```

        for(i=0; i<5; i++) { // ✓
    buisy wait to prevent over pressing
        sceDisplayWaitVblankStart(); ✓
        // blanks screen behind the map
        }
    }
    else if(pad.Ly < 100){
        if(joyY>0){
            joyY=joyY-1;
            for(i=0; i<5; i++) { // ✓
    buisy wait to prevent over pressing
                sceDisplayWaitVblankStart(); ✓
                // blanks screen behind the map
            }
        }
    }
    else if(pad.Ly > 155){
        if(joyY<272){
            joyY=joyY+1;
            for(i=0; i<5; i++) { // ✓
    buisy wait to prevent over pressing
                sceDisplayWaitVblankStart(); ✓
                // blanks screen behind the map
            }
        }
    }

    passXY(joyX, joyY); // captures the joy sticks position and ✓
    passes it to pass.h

    HeadInit(); //invoked ✓
    from Index.h
        blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0); //draws map ✓
    to the screen
        drawPathGPSMAP(result_plotted_points,result_counter); //draws lines ✓
    on map
        updateMapPicture(); //writes text ✓
    at bottom of screen
        flipScreen(); //refreshes ✓
    the screen to other pice of memory

        sceCtrlReadBufferPositive(&pad, 1); // sets pad ✓
    as the buffer for button presses
        if(pad.Buttons & PSP_CTRL_UP) { // if user ✓
    press up
            U_Finish=U_Finish+1; // Update the ✓
        destination index by adding one
            if(U_Finish > 21){ // make sure ✓
    it is still in the acceptable range
                U_Finish=1;} // start over ✓
        if not in acceptable range
            changePositionManual(U_Start, U_Finish); // update end ✓
        index in the interface.h file
            HeadInit(); // ✓
    reinitialize all values for a new draw
        blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0) ✓
;        //draw the base map
        drawPathGPSMAP(result_plotted_points,result_counter); ✓
        //draw the lines
        updateMapPicture(); ✓
        //update and write text
        flipScreen(); ✓
        //refresh the screen
        for(i=0; i<10; i++) { ✓
        // buisy wait to prevent over pressing

```

1

2

3

4

5

6

7

8

9

10

```

        sceDisplayWaitVblankStart();
        // blanks screen behind the map
    }

    }
    else if(pad.Buttons & PSP_CTRL_DOWN) { // if user presses down
        U_Finish=U_Finish-1; // Update the destination index by subtracting one
        if(U_Finish<1){ // make sure it is still in the acceptable range
            U_Finish=21;} // start over
        if not in acceptable range
            changePositionManual(U_Start, U_Finish); // update end index in the interface.h file
        HeadInit(); //
        reinitialize all values for a new draw
        blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0);
        //draw the base map
        drawPathGPSMAP(result_plotted_points,result_counter);
        //draw the lines
        updateMapPicture();
        //update and write text
        flipScreen();
        //refresh the screen
        for(i=0; i<10; i++) {
            // buisy wait to prevent over pressing
            sceDisplayWaitVblankStart();
            // blanks screen behind the map
        }
    } // while
} //else
////////////////////////////////////10-31-06@2:22PM-END////////////////////////////////////
////////////////////////////////////
sceKernelSleepThread(); //makes the thread go to sleep (kill) so the program can shut down
return 0;
}

```



```

// Dru Sepulvedca
// 10-29-06
// Index.h
// This .h contains a list of all include files, functions and variables and creates a
    easy init for all of them.

#include "Dijkstra.h" // The program that does the graph theory and finds the shortest
    path
//includes
// #include <stdio.h>
// #include <stdlib.h>

/* Functions
void initVertexArray();           // initiates vertex array to -1
void initMap();                   // initiates the map to 0
int smallestDist();               // finds smallest distance (private to Dijkstra)
void printMap();                  // DEBUG ONLY Prints out the map in text format
int dijkstra(int first);          // function that finds the shortest overall path
    takes int first node as param
void printPath( struct vertex v); // saves path to result_plotted_points[] takes a
    struct array as param
void printResult_Plotted_Points(); // DEBUG ONLY Prints out the array in text format
*/

/* Variables
int map[SIZE][SIZE]; // structure of the map (a two dentional array (Row x and Colum y) =
    From X to Y) vector format

struct vertex{ // structure of each vertex
    int current; // current index of vertex
    int dist; // distance to adjacent + distance from start
    int prev; // previous vertex with shortest path
    int known; // is this vertex known?
};

struct vertex vertexArray[SIZE]; // an array of type vertex and of size SIZE

int result_plotted_points[SIZE]; // used to hold the printed order of found vertexes in
    found shortest path
int result_counter = 0;          // used to to count in the the array for result ^^^^^^^
    ^^^^^^^^^^^
*/

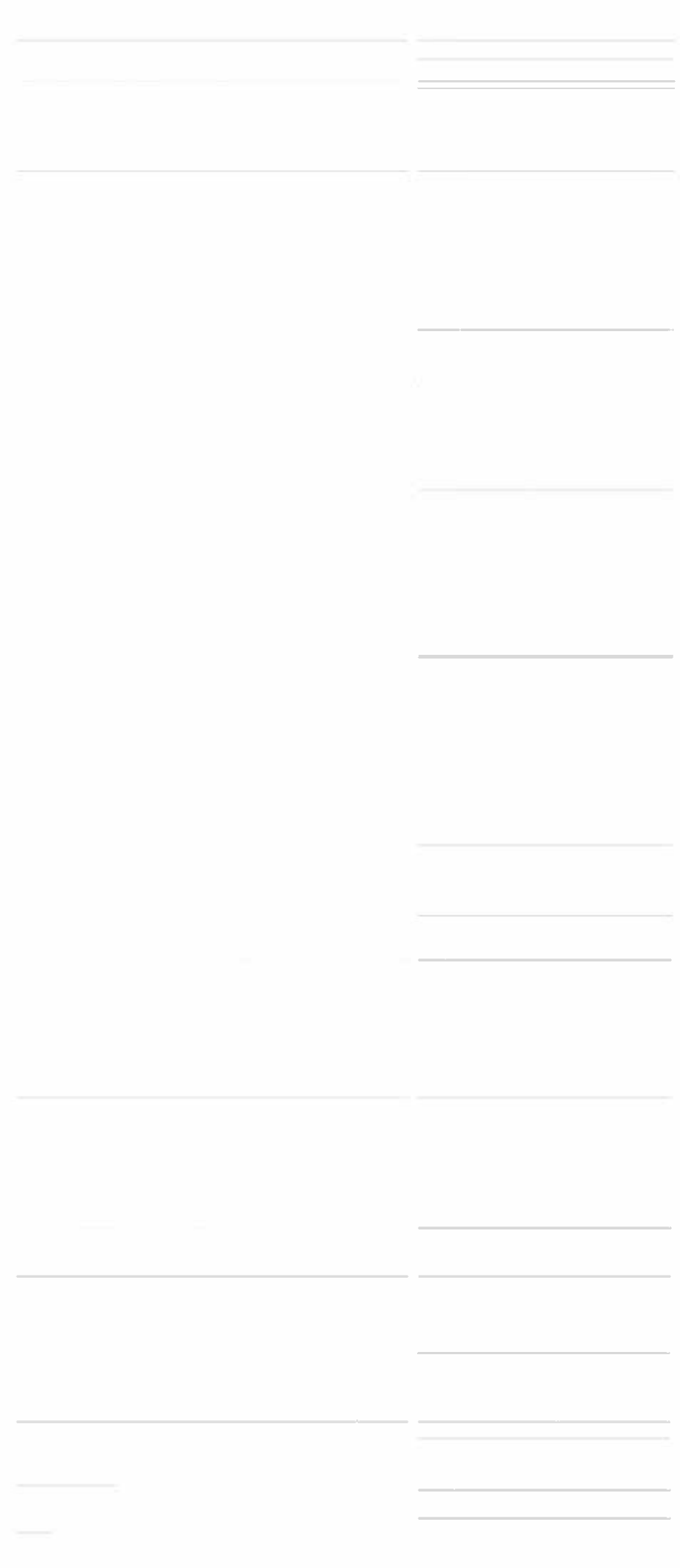
#include "mapPoints.h" // The program that creates a map that is true to the picture and
    prints lines on it.
//includes
// #include "graphics.h" //prewritten header from a lines program

/* Functions
void initGPSMap();               //initailizes the map to default
void defaultGPSMAP();            //loads the default map or map from the users input file
void printAllGPSMAP();           //DEBUG ONLY prints the array out in plane text for debug
    purposes
void drawAllGPSMAP();            //draws the lines based on the points given
void drawPathGPSMAP(int *result_plotted_points, int result_counter); // prints out path
    specified by dijkstra
*/

/* Variables
#define GPS_MAP_SIZE 22 // actual number of nodes on the map (real nodes + ghost nodes)
#define OFFSET_X 0 // how far off along the x axis should we offset
#define OFFSET_Y 0 // how far off along the y axis should we offset
#define SCALE 1 // chage the whole scale, a scale of 1 = 1:1 scale

struct gpsMapVertex{
    int x; // actual x value of the pixel on the screen

```



```

    int y; // actual y value of the pixel on the screen
    int ghost; // This gives me the ability to have nodes at turns with out having them
    show up in the selectiable array
    char turn; // This will let me give the direction of the turn at notes as is
    realitive to the direction vector of the user.
    char *location; // This will give the name of the location.
};

struct gpsMapVertex gpsMap[GPS_MAP_SIZE];
*/

#include "GPS_CURRENT_LOCATION.h" // sets the distance of you from every other node
//Includes
// #include <stdio.h>
// #include <stdlib.h>
// #include <math.h>
// #include "mapPoints.h"

/* Functions
void initMapZero(); // initiates the array that holds the distances
void GPS_LOCATION_YOU(); // get your current location from the GPS device
void DISTANCE_TO_ALL_NODES(); // fills the array with your distance from each node
    using gpsMap.h
*/

/* variables
struct YOUR_LOCATION{ // structure to catch and structure the lat and long to readable
    format for the psp
    float x;
    float y;
};

struct YOUR_LOCATION YOU; // a variable of the the type struct

int Map_Zero[GPS_MAP_SIZE]; // used to hold all the distances possible with

*/

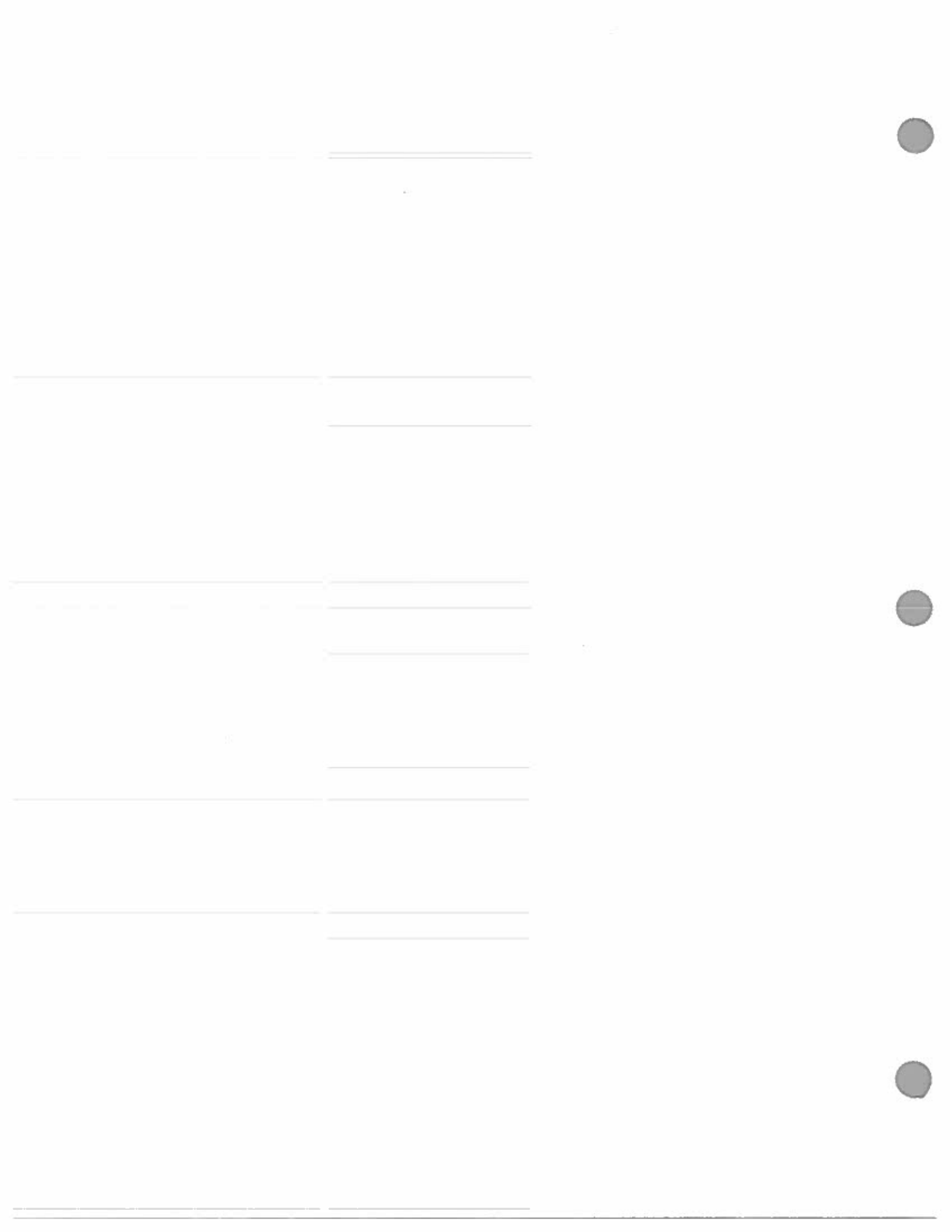
#include "interface.h" // gathers information to update the screen with.
// includes
// #include <stdio.h>
// #include <stdlib.h>
// #include "graphics.h"
// #include "mapPoints.h"
// #include "Dijkstra.h"

/* Functions
void initInterface(); // initiates all of the default settings for the interface
void firstRun(); // sets the values for the first iliteration of the algorithm
void updateMapStatus(); // updates all of the data that will be used to draw on the
    map
void updateMapPicture(); // updates the actual picture on the map with the users input
void debugPrint(); // print function used for debug only
*/

/* Variables
int destinationIndex; // the index of the user defined destination
float distanceLeft; // the distance remaining untill the end of the trip
float distanceTotal; // the total distance (constant for a chosen destination)
float lat; // the latitude value at your current location
float lng; // the long value at your current location
char *currentLocationName; // the name of your current location
char *destinationName; // the name of the user defined destination
*/

#include "pass.h"

```




```
/* passes x and y to .hs that need them. */
```

```
//////////Variables//////////
//////////
```

```
//////////protypes//////////
```

```
void InitAll();          // init all for driver test
```

```
void HeadInit();         // init some for header
```

```
//////////
```

```
//void initAll(){
```

```
//initVertexArray();
```

```
//initMapZero();
```

```
//initGPSMap();
```

```
//defaultGPSMAP();
```

```
//initInterface();
```

```
//GPS_LOCATION_YOU();
```

```
//DISTANCE_TO_ALL_NODES();
```

```
//initMap();
```

```
//dijkstra(User_Defined_Start_Current);
```

```
//printPath(vertexArray[User_Defined_Destination]);
```

```
//printf("\n");
```

```
//firstRun();
```

```
//updateMapStatus();
```

```
//debugPrint();
```

```
//printf("\n");
```

```
//drawPathGPSMAP(result_plotted_points,result_counter);
```

```
//}
```

```
//Dijkstra
```

```
//GPS_MAP_LOCATION
```

```
//mapPoints
```

```
//mapPoints
```

```
//interface
```

```
//GPS_MAP_LOCATION
```

```
//GPS_MAP_LOCATION
```

```
//Dijkstra
```

```
//Dijkstra
```

```
//Dijkstra
```

```
//interface
```

```
//interface
```

```
//interface
```

```
//mapPoints
```

```
void HeadInit(){
```

```
initVertexArray();
```

```
initMapZero();
```

```
initGPSMap();
```

```
defaultGPSMAP();
```

```
initInterface();
```

```
GPS_LOCATION_YOU();
```

```
DISTANCE_TO_ALL_NODES();
```

```
initMap();
```

```
dijkstra(User_Defined_Start_Current);
```

```
printPath(vertexArray[User_Defined_Destination]);
```

```
firstRun();
```

```
updateMapStatus();
```

```
}
```

```
//Dijkstra
```

```
//GPS_MAP_LOCATION
```

```
//mapPoints
```

```
//mapPoints
```

```
//interface
```

```
//GPS_MAP_LOCATION
```

```
//GPS_MAP_LOCATION
```

```
//Dijkstra
```

```
//Dijkstra
```

```
//Dijkstra
```

```
//interface
```

```
//interface
```




```
// Dru Sepulveda
// Dijkstra Implimentation
// Sep, 23 2006
// source: Dr. Lin's Book of Algorithms
```

```
#ifndef __DIJKSTRA__
#define __DIJKSTRA__
```

```
#include <stdio.h>
#include <stdlib.h>
#include "GPS_CURRENT_LOCATION.h"
```

```
#define SIZE 22 // the number of Vertex on the map
#define INFINITY 150000 // my infinity as integer to avoid casting
```

```
////////////////////variables////////////////////////////////////
////////////////////
int map[SIZE][SIZE]; // structure of the map (a two dentional array (Row x and Colum y) =
    From X to Y) vector format
```

```
struct vertex{ // structure of each vertex
    int current; // current index of vertex
    int dist; // distance to adjacent + distance from start
    int prev; // previous vertex with shortest path
    int known; // is this vertex known?
};
```

```
struct vertex vertexArray[SIZE]; // an array of type vertex and of size SIZE
```

```
int result_plotted_points[SIZE]; // used to hold the printed order of found vertexes in
    found shortest path
```

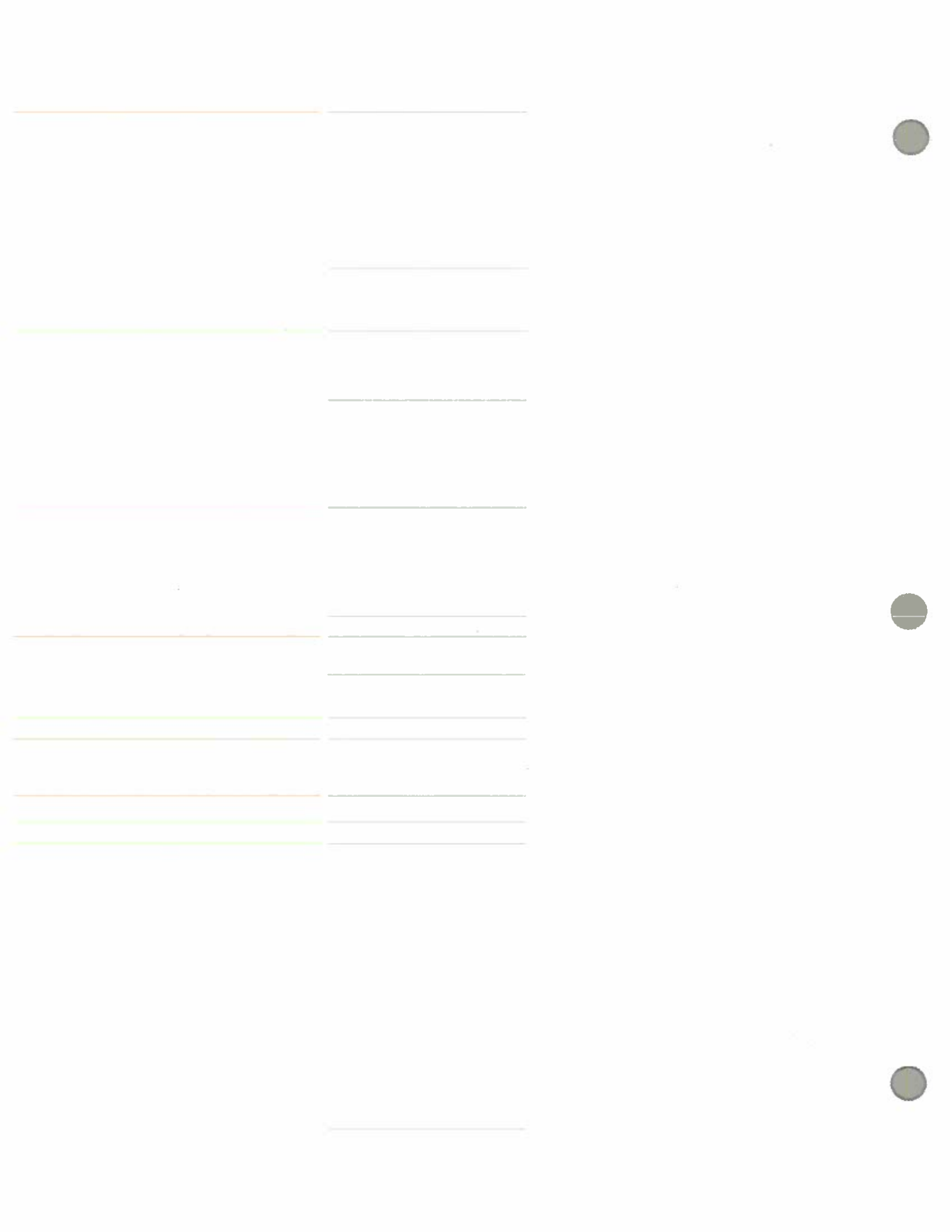
```
int result_counter = 0; // used to to count in the the array for result ^^^^^^
    ^^^^^^^^^^^
////////////////////////////////////
////////////////////////////////////
```

```
////////////////////////////////protptypes////////////////////////////////
void initVertexArray();
void initMap();
int smallestDist();
void printMap();
int dijkstra(int first);
void printPath( struct vertex v);
void printResult_Plotting_Points();
////////////////////////////////
```

```
void initVertexArray(){ // initialize the array of vertex's to default
    result_counter=0;
    int i;
    for(i=0;i<SIZE;i++)
    {
        result_plotted_points[i] = -1;
        vertexArray[i].current = i;
        vertexArray[i].dist = INFINITY;
        vertexArray[i].prev = 0;
        vertexArray[i].known = 0;
    }
}
```

```
void initMap(){
```

```
    map[0][0]=0;
    map[0][1]=Map_Zero[1];
    map[0][2]=Map_Zero[2];
    map[0][3]=Map_Zero[3];
    map[0][4]=Map_Zero[4];
```



```
map[0][5]=Map_Zero[5];
map[0][6]=Map_Zero[6];
map[0][7]=Map_Zero[7];
map[0][8]=Map_Zero[8];
map[0][9]=Map_Zero[9];
map[0][10]=Map_Zero[10];
map[0][11]=Map_Zero[11];
map[0][12]=Map_Zero[12];
map[0][13]=Map_Zero[13];
map[0][14]=Map_Zero[14];
map[0][15]=Map_Zero[15];
map[0][16]=Map_Zero[16];
map[0][17]=Map_Zero[17];
map[0][18]=Map_Zero[18];
map[0][19]=Map_Zero[19];
map[0][20]=Map_Zero[20];
map[0][21]=Map_Zero[21];
```

```
map[1][0]=Map_Zero[1];
map[1][1]=0;
map[1][2]=31;
map[1][3]=INFINITY;
map[1][4]=INFINITY;
map[1][5]=INFINITY;
map[1][6]=INFINITY;
map[1][7]=INFINITY;
map[1][8]=INFINITY;
map[1][9]=INFINITY;
map[1][10]=INFINITY;
map[1][11]=INFINITY;
map[1][12]=INFINITY;
map[1][13]=INFINITY;
map[1][14]=INFINITY;
map[1][15]=INFINITY;
map[1][16]=INFINITY;
map[1][17]=INFINITY;
map[1][18]=INFINITY;
map[1][19]=INFINITY;
map[1][20]=INFINITY;
map[1][21]=INFINITY;
```

```
map[2][0]=Map_Zero[2];
map[2][1]=31;
map[2][2]=0;
map[2][3]=40;
map[2][4]=INFINITY;
map[2][5]=INFINITY;
map[2][6]=INFINITY;
map[2][7]=INFINITY;
map[2][8]=INFINITY;
map[2][9]=INFINITY;
map[2][10]=INFINITY;
map[2][11]=INFINITY;
map[2][12]=INFINITY;
map[2][13]=INFINITY;
map[2][14]=INFINITY;
map[2][15]=INFINITY;
map[2][16]=INFINITY;
map[2][17]=INFINITY;
map[2][18]=INFINITY;
map[2][19]=INFINITY;
map[2][20]=INFINITY;
map[2][21]=INFINITY;
```

```
map[3][0]=Map_Zero[3];
map[3][1]=INFINITY;
```



```
map[3][2]=40;
map[3][3]=0;
map[3][4]=44;
map[3][5]=INFINITY;
map[3][6]=INFINITY;
map[3][7]=INFINITY;
map[3][8]=35;
map[3][9]=INFINITY;
map[3][10]=INFINITY;
map[3][11]=INFINITY;
map[3][12]=INFINITY;
map[3][13]=INFINITY;
map[3][14]=INFINITY;
map[3][15]=INFINITY;
map[3][16]=INFINITY;
map[3][17]=INFINITY;
map[3][18]=INFINITY;
map[3][19]=INFINITY;
map[3][20]=INFINITY;
map[3][21]=INFINITY;

map[4][0]=Map_Zero[4];
map[4][1]=INFINITY;
map[4][2]=INFINITY;
map[4][3]=44;
map[4][4]=0;
map[4][5]=9;
map[4][6]=68;
map[4][7]=INFINITY;
map[4][8]=INFINITY;
map[4][9]=INFINITY;
map[4][10]=INFINITY;
map[4][11]=INFINITY;
map[4][12]=INFINITY;
map[4][13]=INFINITY;
map[4][14]=INFINITY;
map[4][15]=INFINITY;
map[4][16]=INFINITY;
map[4][17]=INFINITY;
map[4][18]=INFINITY;
map[4][19]=INFINITY;
map[4][20]=INFINITY;
map[4][21]=INFINITY;

map[5][0]=Map_Zero[5];
map[5][1]=INFINITY;
map[5][2]=INFINITY;
map[5][3]=INFINITY;
map[5][4]=9;
map[5][5]=0;
map[5][6]=INFINITY;
map[5][7]=INFINITY;
map[5][8]=INFINITY;
map[5][9]=INFINITY;
map[5][10]=INFINITY;
map[5][11]=INFINITY;
map[5][12]=INFINITY;
map[5][13]=INFINITY;
map[5][14]=INFINITY;
map[5][15]=INFINITY;
map[5][16]=INFINITY;
map[5][17]=INFINITY;
map[5][18]=INFINITY;
map[5][19]=INFINITY;
map[5][20]=INFINITY;
map[5][21]=INFINITY;
```

```
map[6][0]=Map_Zero[6];
map[6][1]=INFINITY;
map[6][2]=INFINITY;
map[6][3]=INFINITY;
map[6][4]=68;
map[6][5]=INFINITY;
map[6][6]=0;
map[6][7]=9;
map[6][8]=INFINITY;
map[6][9]=INFINITY;
map[6][10]=INFINITY;
map[6][11]=INFINITY;
map[6][12]=INFINITY;
map[6][13]=INFINITY;
map[6][14]=INFINITY;
map[6][15]=INFINITY;
map[6][16]=INFINITY;
map[6][17]=INFINITY;
map[6][18]=INFINITY;
map[6][19]=INFINITY;
map[6][20]=INFINITY;
map[6][21]=INFINITY;
```

```
map[7][0]=Map_Zero[7];
map[7][1]=INFINITY;
map[7][2]=INFINITY;
map[7][3]=INFINITY;
map[7][4]=INFINITY;
map[7][5]=INFINITY;
map[7][6]=9;
map[7][7]=0;
map[7][8]=INFINITY;
map[7][9]=INFINITY;
map[7][10]=INFINITY;
map[7][11]=INFINITY;
map[7][12]=INFINITY;
map[7][13]=INFINITY;
map[7][14]=INFINITY;
map[7][15]=INFINITY;
map[7][16]=INFINITY;
map[7][17]=INFINITY;
map[7][18]=INFINITY;
map[7][19]=INFINITY;
map[7][20]=INFINITY;
map[7][21]=INFINITY;
```

```
map[8][0]=Map_Zero[8];
map[8][1]=INFINITY;
map[8][2]=INFINITY;
map[8][3]=35;
map[8][4]=INFINITY;
map[8][5]=INFINITY;
map[8][6]=INFINITY;
map[8][7]=INFINITY;
map[8][8]=0;
map[8][9]=12;
map[8][10]=INFINITY;
map[8][11]=7;
map[8][12]=INFINITY;
map[8][13]=INFINITY;
map[8][14]=INFINITY;
map[8][15]=INFINITY;
map[8][16]=INFINITY;
map[8][17]=INFINITY;
map[8][18]=INFINITY;
```

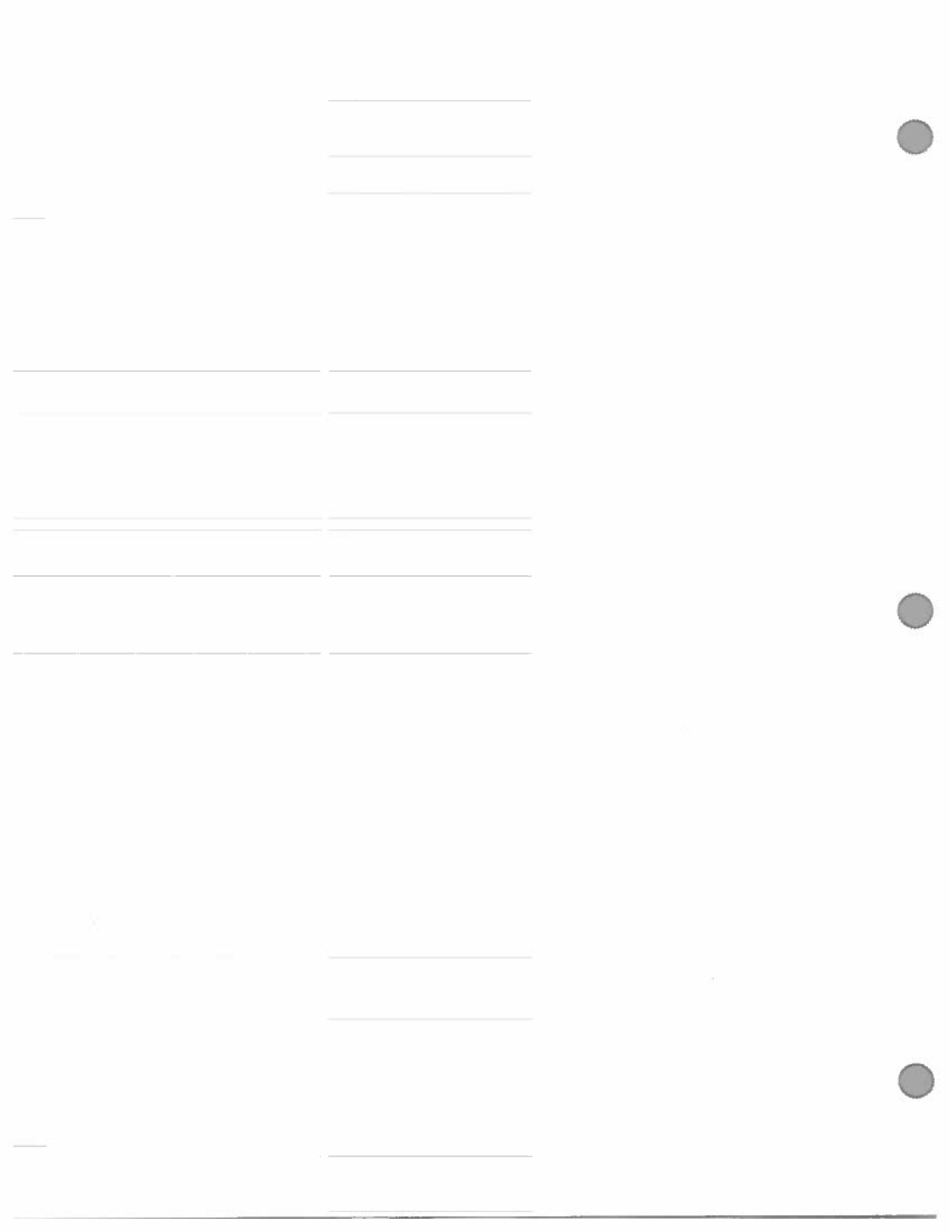


```
map[8][19]=INFINITY;
map[8][20]=INFINITY;
map[8][21]=INFINITY;

map[9][0]=Map_Zero[9];
map[9][1]=INFINITY;
map[9][2]=INFINITY;
map[9][3]=INFINITY;
map[9][4]=INFINITY;
map[9][5]=INFINITY;
map[9][6]=INFINITY;
map[9][7]=INFINITY;
map[9][8]=12;
map[9][9]=0;
map[9][10]=6;
map[9][11]=INFINITY;
map[9][12]=INFINITY;
map[9][13]=INFINITY;
map[9][14]=INFINITY;
map[9][15]=INFINITY;
map[9][16]=INFINITY;
map[9][17]=INFINITY;
map[9][18]=INFINITY;
map[9][19]=28;
map[9][20]=INFINITY;
map[9][21]=53;

map[10][0]=Map_Zero[10];
map[10][1]=INFINITY;
map[10][2]=INFINITY;
map[10][3]=INFINITY;
map[10][4]=INFINITY;
map[10][5]=INFINITY;
map[10][6]=INFINITY;
map[10][7]=INFINITY;
map[10][8]=INFINITY;
map[10][9]=6;
map[10][10]=0;
map[10][11]=INFINITY;
map[10][12]=INFINITY;
map[10][13]=INFINITY;
map[10][14]=INFINITY;
map[10][15]=INFINITY;
map[10][16]=INFINITY;
map[10][17]=INFINITY;
map[10][18]=INFINITY;
map[10][19]=INFINITY;
map[10][20]=INFINITY;
map[10][21]=INFINITY;

map[11][0]=Map_Zero[11];
map[11][1]=INFINITY;
map[11][2]=INFINITY;
map[11][3]=INFINITY;
map[11][4]=INFINITY;
map[11][5]=INFINITY;
map[11][6]=INFINITY;
map[11][7]=INFINITY;
map[11][8]=7;
map[11][9]=INFINITY;
map[11][10]=INFINITY;
map[11][11]=0;
map[11][12]=10;
map[11][13]=INFINITY;
map[11][14]=22;
map[11][15]=INFINITY;
```



```
map[11][16]=INFINITY;
map[11][17]=INFINITY;
map[11][18]=INFINITY;
map[11][19]=INFINITY;
map[11][20]=INFINITY;
map[11][21]=INFINITY;
```

```
map[12][0]=Map_Zero[12];
map[12][1]=INFINITY;
map[12][2]=INFINITY;
map[12][3]=INFINITY;
map[12][4]=INFINITY;
map[12][5]=INFINITY;
map[12][6]=INFINITY;
map[12][7]=INFINITY;
map[12][8]=INFINITY;
map[12][9]=INFINITY;
map[12][10]=INFINITY;
map[12][11]=10;
map[12][12]=0;
map[12][13]=INFINITY;
map[12][14]=INFINITY;
map[12][15]=INFINITY;
map[12][16]=INFINITY;
map[12][17]=INFINITY;
map[12][18]=INFINITY;
map[12][19]=INFINITY;
map[12][20]=INFINITY;
map[12][21]=INFINITY;
```

```
map[13][0]=Map_Zero[13];
map[13][1]=INFINITY;
map[13][2]=INFINITY;
map[13][3]=INFINITY;
map[13][4]=INFINITY;
map[13][5]=INFINITY;
map[13][6]=INFINITY;
map[13][7]=INFINITY;
map[13][8]=INFINITY;
map[13][9]=INFINITY;
map[13][10]=INFINITY;
map[13][11]=INFINITY;
map[13][12]=INFINITY;
map[13][13]=0;
map[13][14]=6;
map[13][15]=INFINITY;
map[13][16]=INFINITY;
map[13][17]=INFINITY;
map[13][18]=INFINITY;
map[13][19]=INFINITY;
map[13][20]=INFINITY;
map[13][21]=INFINITY;
```

```
map[14][0]=Map_Zero[14];
map[14][1]=INFINITY;
map[14][2]=INFINITY;
map[14][3]=INFINITY;
map[14][4]=INFINITY;
map[14][5]=INFINITY;
map[14][6]=INFINITY;
map[14][7]=INFINITY;
map[14][8]=INFINITY;
map[14][9]=INFINITY;
map[14][10]=INFINITY;
map[14][11]=22;
map[14][12]=INFINITY;
```

10

11

12

13

14

15

```
map[14][13]=6;
map[14][14]=0;
map[14][15]=15;
map[14][16]=INFINITY;
map[14][17]=INFINITY;
map[14][18]=INFINITY;
map[14][19]=INFINITY;
map[14][20]=INFINITY;
map[14][21]=INFINITY;

map[15][0]=Map_Zero[15];
map[15][1]=INFINITY;
map[15][2]=INFINITY;
map[15][3]=INFINITY;
map[15][4]=INFINITY;
map[15][5]=INFINITY;
map[15][6]=INFINITY;
map[15][7]=INFINITY;
map[15][8]=INFINITY;
map[15][9]=INFINITY;
map[15][10]=INFINITY;
map[15][11]=INFINITY;
map[15][12]=INFINITY;
map[15][13]=INFINITY;
map[15][14]=15;
map[15][15]=0;
map[15][16]=15;
map[15][17]=13;
map[15][18]=INFINITY;
map[15][19]=INFINITY;
map[15][20]=INFINITY;
map[15][21]=INFINITY;

map[16][0]=Map_Zero[16];
map[16][1]=INFINITY;
map[16][2]=INFINITY;
map[16][3]=INFINITY;
map[16][4]=INFINITY;
map[16][5]=INFINITY;
map[16][6]=INFINITY;
map[16][7]=INFINITY;
map[16][8]=INFINITY;
map[16][9]=INFINITY;
map[16][10]=INFINITY;
map[16][11]=INFINITY;
map[16][12]=INFINITY;
map[16][13]=INFINITY;
map[16][14]=INFINITY;
map[16][15]=15;
map[16][16]=0;
map[16][17]=INFINITY;
map[16][18]=INFINITY;
map[16][19]=INFINITY;
map[16][20]=INFINITY;
map[16][21]=INFINITY;

map[17][0]=Map_Zero[17];
map[17][1]=INFINITY;
map[17][2]=INFINITY;
map[17][3]=INFINITY;
map[17][4]=INFINITY;
map[17][5]=INFINITY;
map[17][6]=INFINITY;
map[17][7]=INFINITY;
map[17][8]=INFINITY;
map[17][9]=INFINITY;
```



```
map[17][10]=INFINITY;
map[17][11]=INFINITY;
map[17][12]=INFINITY;
map[17][13]=INFINITY;
map[17][14]=INFINITY;
map[17][15]=13;
map[17][16]=INFINITY;
map[17][17]=0;
map[17][18]=10;
map[17][19]=10;
map[17][20]=INFINITY;
map[17][21]=INFINITY;

map[18][0]=Map_Zero[18];
map[18][1]=INFINITY;
map[18][2]=INFINITY;
map[18][3]=INFINITY;
map[18][4]=INFINITY;
map[18][5]=INFINITY;
map[18][6]=INFINITY;
map[18][7]=INFINITY;
map[18][8]=INFINITY;
map[18][9]=INFINITY;
map[18][10]=INFINITY;
map[18][11]=INFINITY;
map[18][12]=INFINITY;
map[18][13]=INFINITY;
map[18][14]=INFINITY;
map[18][15]=INFINITY;
map[18][16]=INFINITY;
map[18][17]=10;
map[18][18]=0;
map[18][19]=INFINITY;
map[18][20]=INFINITY;
map[18][21]=INFINITY;

map[19][0]=Map_Zero[19];
map[19][1]=INFINITY;
map[19][2]=INFINITY;
map[19][3]=INFINITY;
map[19][4]=INFINITY;
map[19][5]=INFINITY;
map[19][6]=INFINITY;
map[19][7]=INFINITY;
map[19][8]=INFINITY;
map[19][9]=28;
map[19][10]=INFINITY;
map[19][11]=INFINITY;
map[19][12]=INFINITY;
map[19][13]=INFINITY;
map[19][14]=INFINITY;
map[19][15]=INFINITY;
map[19][16]=INFINITY;
map[19][17]=10;
map[19][18]=INFINITY;
map[19][19]=0;
map[19][20]=10;
map[19][21]=INFINITY;

map[20][0]=Map_Zero[20];
map[20][1]=INFINITY;
map[20][2]=INFINITY;
map[20][3]=INFINITY;
map[20][4]=INFINITY;
map[20][5]=INFINITY;
map[20][6]=INFINITY;
```



```

map[20][7]=INFINITY;
map[20][8]=INFINITY;
map[20][9]=INFINITY;
map[20][10]=INFINITY;
map[20][11]=INFINITY;
map[20][12]=INFINITY;
map[20][13]=INFINITY;
map[20][14]=INFINITY;
map[20][15]=INFINITY;
map[20][16]=INFINITY;
map[20][17]=INFINITY;
map[20][18]=INFINITY;
map[20][19]=10;
map[20][20]=0;
map[20][21]=INFINITY;

```

```

map[21][0]=Map_Zero[21];
map[21][1]=INFINITY;
map[21][2]=INFINITY;
map[21][3]=INFINITY;
map[21][4]=INFINITY;
map[21][5]=INFINITY;
map[21][6]=INFINITY;
map[21][7]=INFINITY;
map[21][8]=INFINITY;
map[21][9]=53;
map[21][10]=INFINITY;
map[21][11]=INFINITY;
map[21][12]=INFINITY;
map[21][13]=INFINITY;
map[21][14]=INFINITY;
map[21][15]=INFINITY;
map[21][16]=INFINITY;
map[21][17]=INFINITY;
map[21][18]=INFINITY;
map[21][19]=INFINITY;
map[21][20]=INFINITY;
map[21][21]=0;

```

```

}

```

```

int smallestDist(){ //function to return the smallest vertex's index that is not yet
    found (.known != 1)
    int i;
    int indexSmallest = -1;
    int smallest = INFINITY;
    for(i=0;i<SIZE;i++){
        if(vertexArray[i].known != 1){
            if(vertexArray[i].dist<smallest){
                smallest = vertexArray[i].dist;
                indexSmallest = i;
            }
        }
    }
    return indexSmallest; // returns -1 if no no more non-true indexes
}

```

```

void printMap(){ // a simple program to the map and the values to the screen, used to
    debug
    int n,p,top,side;
    top=side=0;

    printf("\n");

    printf("        "); // prints out the top line
    for(n=0;n<SIZE;n++)

```

11

12

13

14

```

printf(" [%d]", top++);

for(n=0;n<SIZE;n++){
    printf(" \n "); // changes line and prints the side numbers
    if(side<10)
        printf(" [%d] ", side++);
    else
        printf(" [%d] ", side++);
    for(p=0;p<SIZE;p++){
        if(map[n][p] == INFINITY)
            printf(" - ");
        else
            printf(" %d ", map[n][p]); // print out of array
    }
}
printf("\n");
}

int dijkstra(int first){
    // debug lines will not be in the final nor will the print out of the map
    //printf("Starting dijkstra\n"); // debug line for Dr Lin
    int i,cvw,index;
    int tag = 0;
    initVertexArray();
    vertexArray[first].dist = 0;
    vertexArray[first].prev = -1;
    while(tag!= 1){
        if(smallestDist()!=-1){
            index = smallestDist();
            //printf("The index is %d \n", index); // debug line for Dr Lin
        }
        else{
            //printf("Dijkstra has exited with index -1 \n"); // debug line for Dr Lin
            tag = 1;
            break;
        }

        vertexArray[index].known = 1;

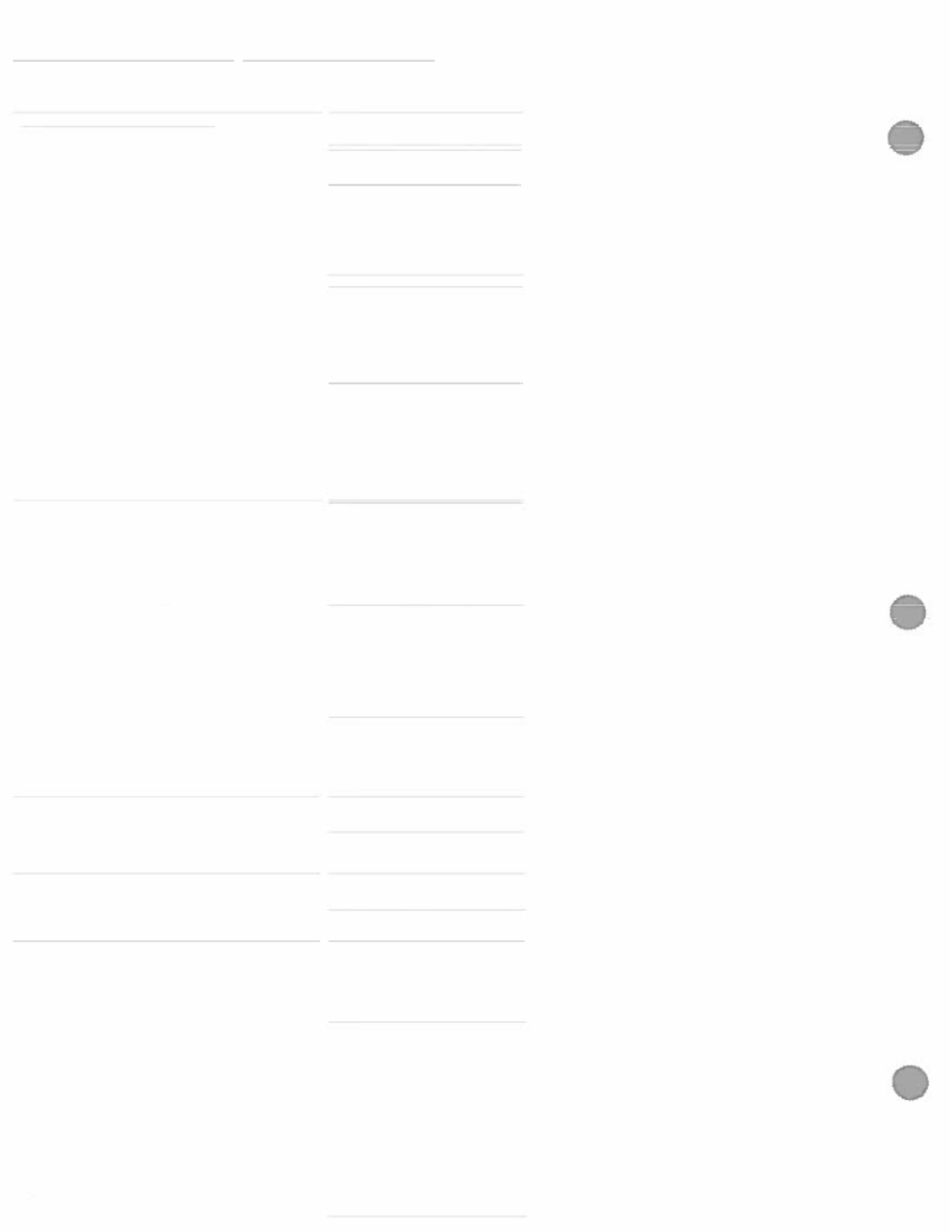
        //printf("The current smallest known index is %d \n", index); //debug line for
Dr Lin
        for(i=0;i<SIZE;i++){
            if(map[index][i]!=INFINITY){
                cvw = map[index][i];
                if(cvw + vertexArray[index].dist < vertexArray[i].dist)
                {
                    //printf("it was found that %d was smaller than %d \n", index, i); //
debug line for Dr Lin
                    vertexArray[i].dist = vertexArray[index].dist + cvw;
                    vertexArray[i].prev = index;
                }
            }
        }
    }
}

void printPath( struct vertex v){ // prints out the path by recursively following the ends
previous
if(v.prev != -1){
    printPath(vertexArray[v.prev]);
    //printf(" to ");
}
//printf("%d", v.current);
result_plotted_points[result_counter++]=v.current;
}

```

```
void printResult_Plotting_Points(){ // for debug only, prints out the saved array
    //int i;
    //printf("\n Print array of saved values \n"); // debug line
    //for(i=0;i<result_counter;i++)
        //printf("Shortest path %d = %d\n", i, result_plotted_points[i]);
}

#endif
```




```

////////////////////////////////////
// Dru Sepulveda
// mapPoints v 1.0
// 9-26-06
//
// This header file contains a set of functions to associate the
// generated shortest path with the actual points on the map.
//
////////////////////////////////////

#ifndef __MAPPOINTS__
#define __MAPPOINTS__

#include "graphics.h" //prewritten headder from a lines program
#include "pass.h" // used to pass x and y value to map

#define GPS_MAP_SIZE 22 // actual number of nodes on the map (real nodes + ghost nodes)
#define OFFSET_X 0 // how far off along the x axis should we offset
#define OFFSET_Y 0 // how far off along the y axis should we offset
#define SCALE 1 // chage the whole scale, a scale of 1 = 1:1 scale

////////////////////////////////////variables////////////////////////////////////
struct gpsMapVertex{
    int x; // actual x value of the pixel on the screen
    int y; // actual y value of the pixel on the screen
    int ghost; // This gives me the ability to have nodes at turns with out having them
               // show up in the selectiable array
    char turn; // This will let me give the direction of the turn at notes as is
               // realitive to the direction vector of the user.
    char *location; // This will give the name of the location.
};

struct gpsMapVertex gpsMap[GPS_MAP_SIZE];
//Color white = RGB(0, 0, 0); //Added 10-31-2006 @ 2:21PM
////////////////////////////////////

////////////////////////////////////PROTOTYPES////////////////////////////////////
void initGPSMap(); //initailizes the map to default
void defaultGPSMAP(); //loads the default map or map from the users input file
void printAllGPSMAP(); //prints the array out in plane text for debug purposes
void drawAllGPSMAP(); //draws the lines based on the points given
void drawPathGPSMAP(int *result_plotted_points, int result_counter); // prints out path
                             // specified by dijkstra
////////////////////////////////////

void initGPSMap(){ //simple program
    //printf("Initilizing GPSMAP\n"); //debug line
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        gpsMap[i].x = -1; // default value
        gpsMap[i].y = -1; // default value
        gpsMap[i].ghost = 0; // not a ghost
        gpsMap[i].turn = 'o'; // default
        gpsMap[i].location = NULL; // location unknown and defaulted
    }
}

void defaultGPSMAP(){ // sets map from a file or from default map <assumes size 6>
    // get information from a file?

    //---> open and read file dialogue comming soon

```



```
// default test informaiton
printf("Loading Default Map\n"); //debug line
gpsMap[0].x = (floor(passX) + OFFSET_X)*SCALE;
gpsMap[0].y = (floor(passY) + OFFSET_Y)*SCALE;
gpsMap[0].ghost = 0;
gpsMap[0].turn = 'o';
gpsMap[0].location = "You";

gpsMap[1].x = (125 + OFFSET_X)*SCALE;
gpsMap[1].y = (48 + OFFSET_Y)*SCALE;
gpsMap[1].ghost = 0;
gpsMap[1].turn = 'o';
gpsMap[1].location = "Quads";

gpsMap[2].x = (125 + OFFSET_X)*SCALE;
gpsMap[2].y = (72 + OFFSET_Y)*SCALE;
gpsMap[2].ghost = 1;
gpsMap[2].turn = 'o';
gpsMap[2].location = "Infront of Quads";

gpsMap[3].x = (165 + OFFSET_X)*SCALE;
gpsMap[3].y = (72 + OFFSET_Y)*SCALE;
gpsMap[3].ghost = 1;
gpsMap[3].turn = 'o';
gpsMap[3].location = "Lightsy Intersection";

gpsMap[4].x = (209 + OFFSET_X)*SCALE;
gpsMap[4].y = (72 + OFFSET_Y)*SCALE;
gpsMap[4].ghost = 1;
gpsMap[4].turn = 'o';
gpsMap[4].location = "Infront of Whittington Hall";

gpsMap[5].x = (209 + OFFSET_X)*SCALE;
gpsMap[5].y = (63 + OFFSET_Y)*SCALE;
gpsMap[5].ghost = 0;
gpsMap[5].turn = 'o';
gpsMap[5].location = "Whittington Hall";

gpsMap[6].x = (277 + OFFSET_X)*SCALE;
gpsMap[6].y = (72 + OFFSET_Y)*SCALE;
gpsMap[6].ghost = 1;
gpsMap[6].turn = 'o';
gpsMap[6].location = "Infront of Brewer Center";

gpsMap[7].x = (277 + OFFSET_X)*SCALE;
gpsMap[7].y = (63 + OFFSET_Y)*SCALE;
gpsMap[7].ghost = 0;
gpsMap[7].turn = 'o';
gpsMap[7].location = "Brewer Center";

gpsMap[8].x = (165 + OFFSET_X)*SCALE;
gpsMap[8].y = (107 + OFFSET_Y)*SCALE;
gpsMap[8].ghost = 1;
gpsMap[8].turn = 'o';
gpsMap[8].location = "Pond Circle";

gpsMap[9].x = (177 + OFFSET_X)*SCALE;
gpsMap[9].y = (107 + OFFSET_Y)*SCALE;
gpsMap[9].ghost = 1;
gpsMap[9].turn = 'o';
gpsMap[9].location = "Pond Circle";

gpsMap[10].x = (177 + OFFSET_X)*SCALE;
gpsMap[10].y = (107 + OFFSET_Y)*SCALE;
gpsMap[10].ghost = 0;
```



10

11

12

13

14

15

```
gpsMap[10].turn = 'o';
gpsMap[10].location = "Strom Thurmond Center";

gpsMap[11].x = (158 + OFFSET_X)*SCALE;
gpsMap[11].y = (107 + OFFSET_Y)*SCALE;
gpsMap[11].ghost = 1;
gpsMap[11].turn = 'o';
gpsMap[11].location = "Infront of Lightsy Chapel";

gpsMap[12].x = (158 + OFFSET_X)*SCALE;
gpsMap[12].y = (97 + OFFSET_Y)*SCALE;
gpsMap[12].ghost = 0;
gpsMap[12].turn = 'o';
gpsMap[12].location = "Lightsy Chappel";

gpsMap[13].x = (136 + OFFSET_X)*SCALE;
gpsMap[13].y = (100 + OFFSET_Y)*SCALE;
gpsMap[13].ghost = 0;
gpsMap[13].turn = 'o';
gpsMap[13].location = "Rivers Library";

gpsMap[14].x = (136 + OFFSET_X)*SCALE;
gpsMap[14].y = (107 + OFFSET_Y)*SCALE;
gpsMap[14].ghost = 1;
gpsMap[14].turn = 'o';
gpsMap[14].location = "Infront of Rivers Library";

gpsMap[15].x = (136 + OFFSET_X)*SCALE;
gpsMap[15].y = (122 + OFFSET_Y)*SCALE;
gpsMap[15].ghost = 1;
gpsMap[15].turn = 'o';
gpsMap[15].location = "Infront of Science Building";

gpsMap[16].x = (121 + OFFSET_X)*SCALE;
gpsMap[16].y = (122 + OFFSET_Y)*SCALE;
gpsMap[16].ghost = 0;
gpsMap[16].turn = 'o';
gpsMap[16].location = "Science Building";

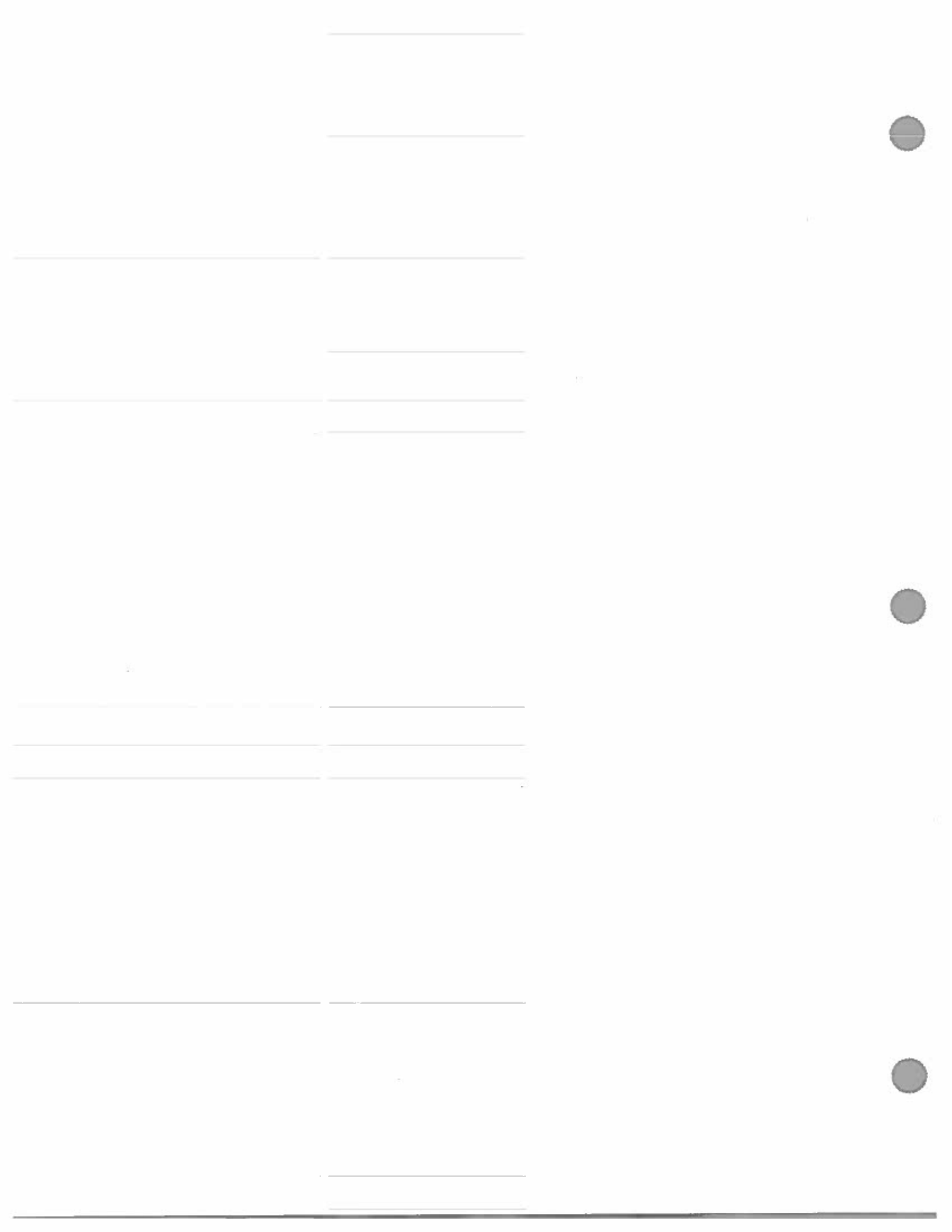
gpsMap[17].x = (136 + OFFSET_X)*SCALE;
gpsMap[17].y = (135 + OFFSET_Y)*SCALE;
gpsMap[17].ghost = 1;
gpsMap[17].turn = 'o';
gpsMap[17].location = "Infront of Wingo/Norris";

gpsMap[18].x = (136 + OFFSET_X)*SCALE;
gpsMap[18].y = (145 + OFFSET_Y)*SCALE;
gpsMap[18].ghost = 0;
gpsMap[18].turn = 'o';
gpsMap[18].location = "Wingo/Norris";

gpsMap[19].x = (177 + OFFSET_X)*SCALE;
gpsMap[19].y = (135 + OFFSET_Y)*SCALE;
gpsMap[19].ghost = 1;
gpsMap[19].turn = 'o';
gpsMap[19].location = "Infront of Jones/Ashby";

gpsMap[20].x = (177 + OFFSET_X)*SCALE;
gpsMap[20].y = (145 + OFFSET_Y)*SCALE;
gpsMap[20].ghost = 0;
gpsMap[20].turn = 'o';
gpsMap[20].location = "Jones/Ashby";

gpsMap[21].x = (230 + OFFSET_X)*SCALE;
gpsMap[21].y = (107 + OFFSET_Y)*SCALE;
gpsMap[21].ghost = 0;
```



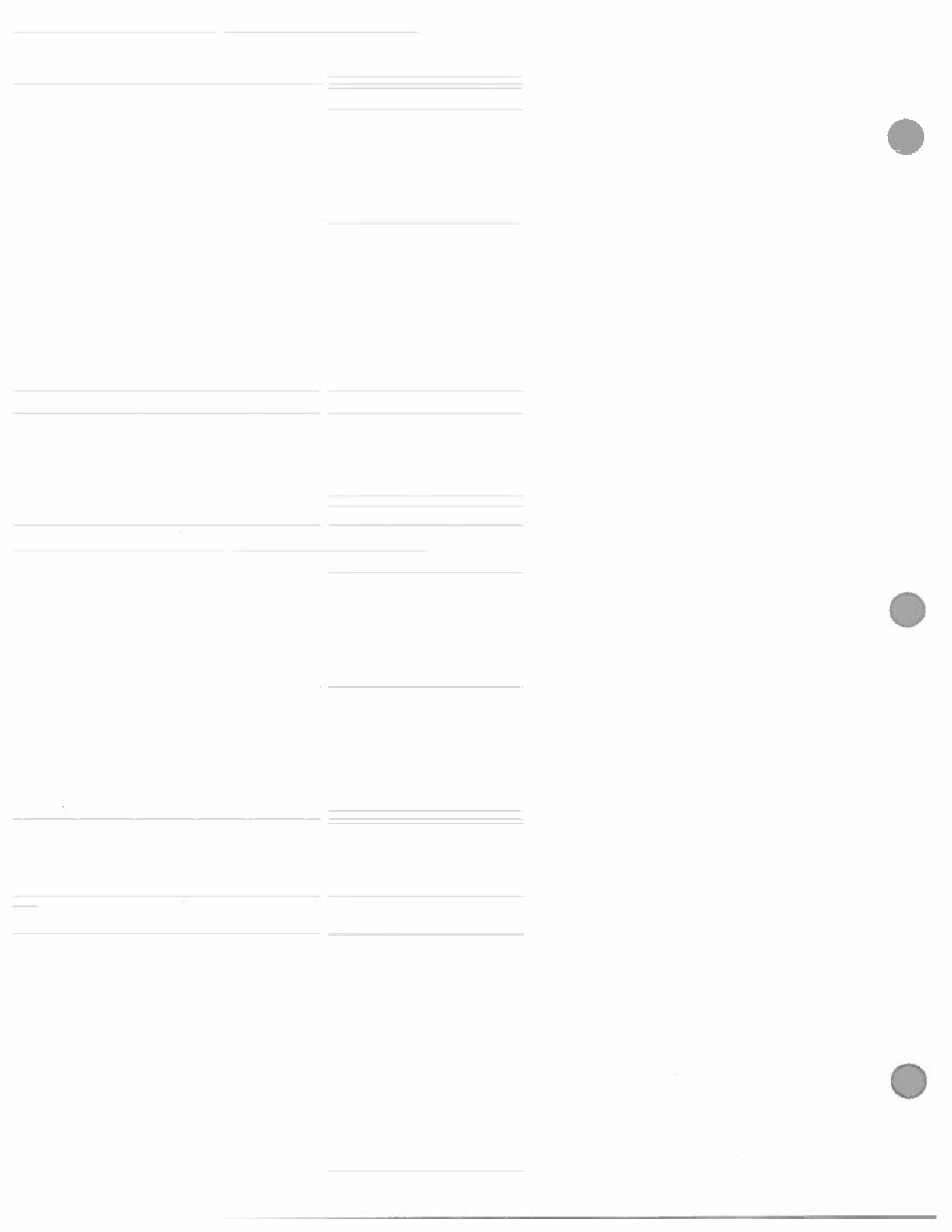
```
    gpsMap[21].turn = 'o';
    gpsMap[21].location = "Russel West";
}

void printAllGPSMAP(){ // prints out the entire map for debug
    // normally it would impliment the graphics.h from the lines and circles program
    // but while testing it will cimply print out the information.
    printf("Printing out GPSMAP\n"); //debug line
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        printf("Index %d has x = %d and y = %d, ghost = %d, turn = %c, name = %s \n", i ,
            gpsMap[i].x, gpsMap[i].y, gpsMap[i].ghost, gpsMap[i].turn, gpsMap[i].location);
    }
}

void drawAllGPSMAP(){ // draws the entire map
    //printf("Starting drawALLgpsMAP Draw\n");
    int i;
    for(i=0;i<GPS_MAP_SIZE-1;i++){
        drawLineScreen(gpsMap[i].x,gpsMap[i].y,gpsMap[i+1].x,gpsMap[i+1].y,R(100)); //color
        is defined 10-31-06 @ 2:21PM
        //printf("(%d,%d) to (%d,%d) with color %d %d %d\n", gpsMap[i].x,gpsMap[i].y,gpsMap[i+1].x,gpsMap[i+1].y,255,0,0); //debug line
    }
}

void drawPathGPSMAP(int *result_plotted_points, int result_counter){ // draws the path
    selected by Dijkstra
    //printf("Starting drawPathGPSMAP\n");
    int i;
    for(i=0;i<result_counter-1;i++){
        drawLineScreen(gpsMap[result_plotted_points[i]].x,gpsMap[result_plotted_points[i]].y,
            gpsMap[result_plotted_points[i+1]].x,gpsMap[result_plotted_points[i+1]].y,R(100)); //
        color can be 0 - 255
        //printf("(%d,%d) to (%d,%d) with color %d %d %d\n", gpsMap[result_plotted_points[i]].x,gpsMap[result_plotted_points[i]].y,gpsMap[result_plotted_points[i+1]].x,gpsMap[result_plotted_points[i+1]].y,255,0,0); //debug line
    }
}

#endif
```




```
// Dru Sepulveda
// Dynamic GPS Location
// Sep, 28 2006
```

```
#ifndef GPS_CURRENT_LOCATION
#define GPS_CURRENT_LOCATION
```

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "mapPoints.h"
#include "pass.h" // used to pass x and y values
```

```
struct YOUR_LOCATION{ // structure to catch and structure the lat and long to readable
    format for the psp
    float x;
    float y;
};
```

```
struct YOUR_LOCATION YOU; // a variable of the the type struct
```

```
int Map_Zero[GPS_MAP_SIZE]; // used to hold all the distances possible with
```

```
/////////////////////////prototypes/////////////////////////
void initMapZero(); // initiate Map_Zero array
void GPS_LOCATION_YOU(); // gather your current location
void DISTANCE_TO_ALL_NODES(); // find the distance to all nodes and return smallest by
    Map_Zero array
/////////////////////////
```

```
void initMapZero(){ // initializes all values in the Map_Zero array to infinity
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        Map_Zero[i]=150000;
    }
}
```

```
void GPS_LOCATION_YOU(){
//This information will be taken from the GPS device
    YOU.x = passX;
    YOU.y = passY;
}
```

```
void DISTANCE_TO_ALL_NODES(){
    // find the distance between each point on the map and the user, then save it to the
    array.
    int i;
    for(i=1;i<GPS_MAP_SIZE;i++){ ///altered this! :D needs to go from 1 to 0 to find
    least.
    Map_Zero[i] = floor(sqrt(pow((gpsMap[i].x-YOU.x),2)+pow((gpsMap[i].y-YOU.y),2)));
    // set distances for Dijkstra
    //printf("Distance %d --> %d\n", i, Map_Zero[i]); // for debug
}
```

```
int indexGPSSmallest = 0; // variable for the smallest index in
Map_Zero
int GPSSmallest = 150000; // initiates the smallest variable to
"infinity"
for(i=0;i<GPS_MAP_SIZE;i++){ // for all indexes (includeing itself but
is fixed above)
    if(Map_Zero[i]<GPSSmallest){ // find the smallest indexed distance
        GPSSmallest = Map_Zero[i];
        indexGPSSmallest = i;
    }
}
```



```
    for(i=0;i<GPS_MAP_SIZE;i++){
        infinity but the smallest node
        if(i != indexGPSSmallest){
            pick the closest node and not get
            Map_Zero[i] = 150000;
        }
    }
}

#endif
```

// make all entries to Map_Zero = ✓

// this makes it easy for the program to ✓

// confused by the other ones.



```
// Dru Sepulveda
// Interface for pspGPS_Program
// Oct, 19 2006
// 1:00PM <right after the fire alarm>
//
```

```
#ifndef __INTERFACE__
#define __INTERFACE__
```

```
#include <stdio.h>
#include <stdlib.h>
// #include "graphics.h"
#include "mapPoints.h"
#include "Dijkstra.h"
```

```
////////////////////variables//////////////////////////////////////✓
//
```

```
int destinationIndex;           // the index of the user defined destination
float distanceLeft;             // the distance remaining untill the end of the trip
float distanceTotal;           // the total distance (constant for a chosen destination)
float lat;                     // the latitude value at your current location
float lng;                     // the long value at your current location
char *currentLocationName;     // the name of your current location
char *destinationName;         // the name of the user defined destination
```

```
int User_Defined_Start_Current; //holds the value passes from main
```

```
int User_Defined_Destination;   //holds the value passes from main
```

```
//////////////////////////////////////✓
//
```

```
//////////////////////////////////////prototypes//////////////////////////////////////✓
//
```

```
void changePositionManual(int start, int end); // 10-31-06 @ 3:00PM
void initInterface();           // initiates all of the default settings for the interface
void firstRun();               // sets the values for the first iteration of the algorithm
void updateMapStatus();        // updates all of the data that will be used to draw on ✓
    the map
void updateMapPicture();       // updates the actual picture on the map with the users input
//void debugPrint();          // print function used for debug only
//////////////////////////////////////✓
//
```

```
void changePositionManual(int start, int end){
    User_Defined_Start_Current = start;           //starting index passed by main
    User_Defined_Destination = end;               //ending index passes by main
}
```

```
void initInterface(){           // initializing all values to default
```

```
    destinationIndex = 0;
    distanceLeft = 0;
    distanceTotal = 0;
    lat = 0;
    lng = 0;
    currentLocationName = NULL;
    destinationName = NULL;
}
```

```
void firstRun(){               // sets the needed values each time a new location is chosen
```

```
    distanceTotal = vertexArray[SIZE-1].dist;     //The Last value from the first time ✓
    Dijkstra runs
    destinationIndex = User_Defined_Destination;   // The index of the location chosen ✓
    by the user from the chooseable list in gpsMaps.h
    destinationName = gpsMap[User_Defined_Destination].location; //this value is pulled ✓
    from the inbeded information in the gpsMaps.h file
```



```
}

void updateMapStatus(){ // updates all data that will be used to draw on the map
    currentLocationName = gpsMap[User_Defined_Start_Current].location; // this name ✓
    is gotten by the Second jump in dijkstra after you
}

void updateMapPicture(){ // This function actually writes to the PSP screen the TEXT that ✓
    has been gathered.
    // void printTextScreen(int x, int y, const char* text, u32 color);
    printTextScreen(15,245,"C_Location: ",B(100));
    printTextScreen(105,245,currentLocationName,B(100));
    printTextScreen(15,260,"D_Location: ",B(100));
    printTextScreen(105,260,destinationName,B(100));
    //printTextScreen(400,254,"DIST: ",B(100));
    //printTextScreen(420,254,convertDistTotal,B(100));
}

//void debugPrint(){
//printf("Lat: %f\nLong: %f\ntotalDistance: %f\ndestinationIndex: %d\ndistanceLeft: %f\
    ncurrentLocN: %s\ndestLocN: %s", lat, lng, distanceTotal, destinationIndex,
    distanceLeft, currentLocationName, destinationName);
//}

#endif
```




```
// Dru Sepulveda
// 11-5-06
// file to pass the current location to all .h files that need it

#ifndef __pass__
#define __pass__

//////////variables//////////
float passX = 0;    //x position of user
float passY = 0;    //y position of user
//////////

//////////prototype//////////
void passXY(float pX, float pY);    //makes sure that this .h file has access to the
    information
//////////

void passXY(float pX, float pY){    //gathers x and y for user to pass to other files
    passX = pX;
    passY = pY;
}

#endif
```

W

Q

Q

A

Q

```
// Final Driver for Dijkstra Program
// Dru Sepulveda
// 10-29-06
// Runs from Index
```

```
#include "Index.h"
```

```
int main(){
    initAll();
    return 0;
}
```



```
#include <stdlib.h>
#include <malloc.h>
#include <pspdisplay.h>
#include <psputils.h>
#include <png.h>
#include <pspgu.h>

#include "graphics.h"
#include "framebuffer.h"

#define IS_ALPHA(color) (((color)&0xff000000)==0xff000000?0:1)
#define FRAMEBUFFER_SIZE (PSP_LINE_SIZE*SCREEN_HEIGHT*4)
#define MAX(X, Y) ((X) > (Y) ? (X) : (Y))

typedef struct
{
    unsigned short u, v;
    short x, y, z;
} Vertex;

extern u8 msx[];

unsigned int __attribute__((aligned(16))) list[262144];
static int dispBufferNumber;
static int initialized = 0;

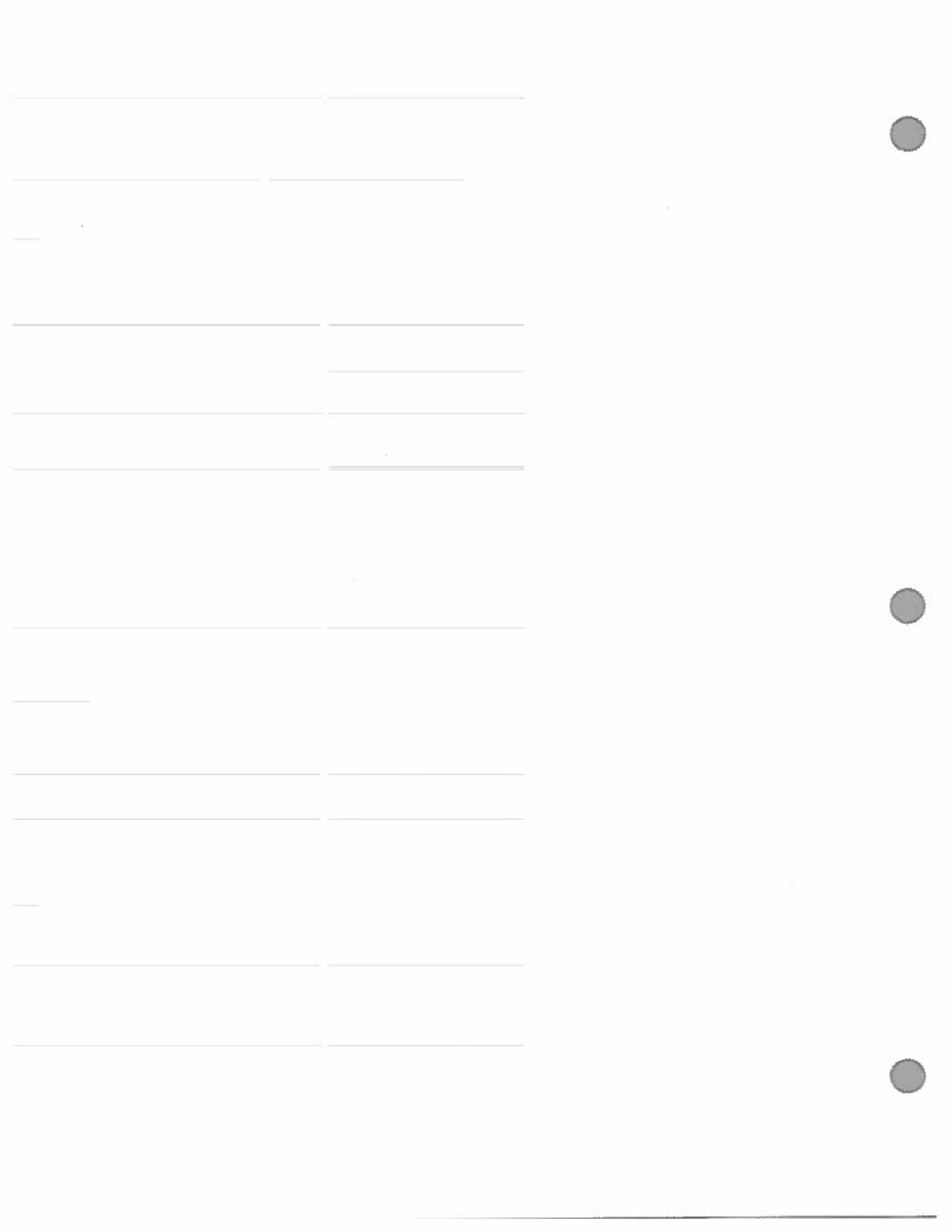
static int getNextPower2(int width)
{
    int b = width;
    int n;
    for (n = 0; b != 0; n++) b >>= 1;
    b = 1 << n;
    if (b == 2 * width) b >>= 1;
    return b;
}

Color* getVramDrawBuffer()
{
    Color* vram = (Color*) g_vram_base;
    if (dispBufferNumber == 0) vram += FRAMEBUFFER_SIZE / sizeof(Color);
    return vram;
}

Color* getVramDisplayBuffer()
{
    Color* vram = (Color*) g_vram_base;
    if (dispBufferNumber == 1) vram += FRAMEBUFFER_SIZE / sizeof(Color);
    return vram;
}

void user_warning_fn(png_structp png_ptr, png_const_charp warning_msg)
{
}

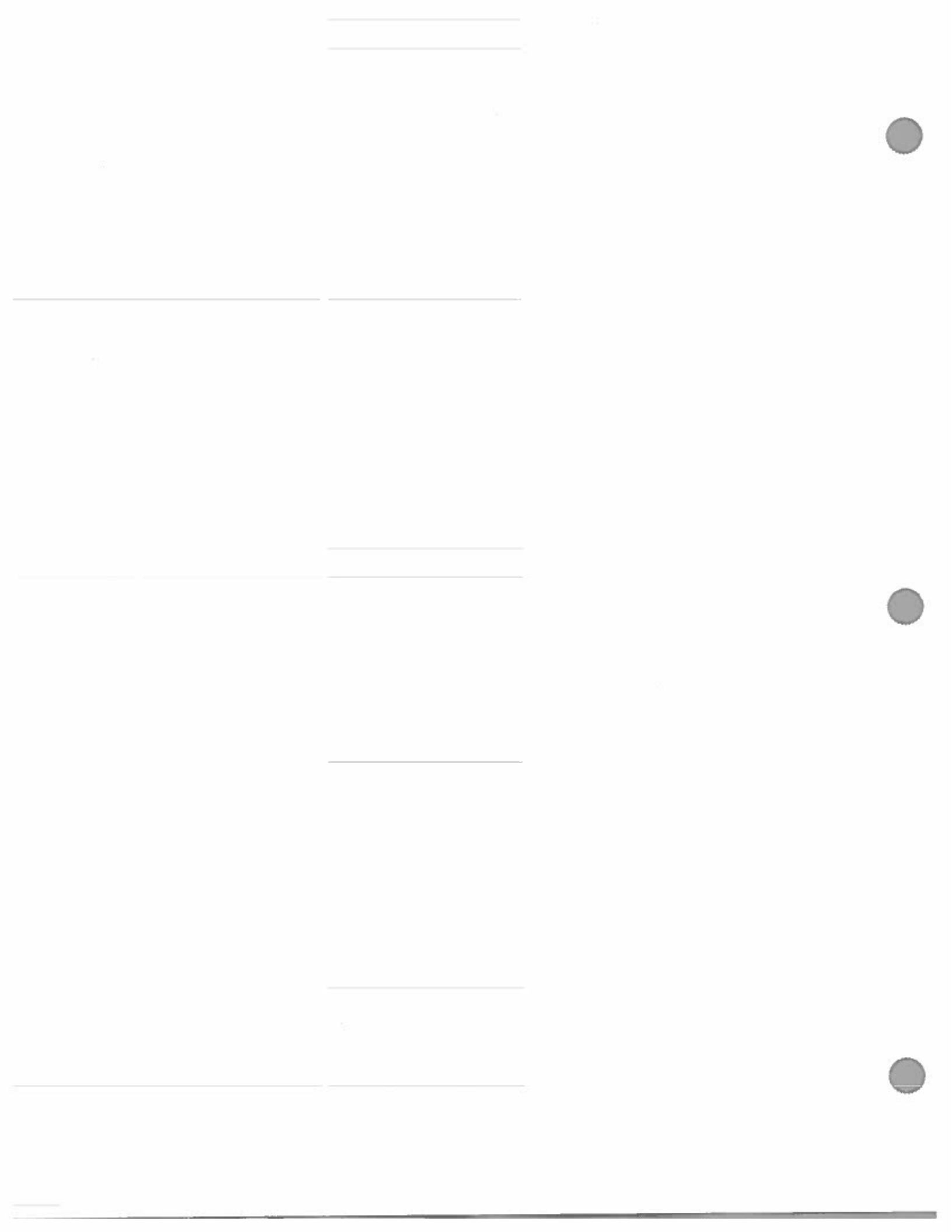
Image* loadImage(const char* filename)
{
    png_structp png_ptr;
    png_infop info_ptr;
    unsigned int sig_read = 0;
    png_uint_32 width, height;
    int bit_depth, color_type, interlace_type, x, y;
    u32* line;
    FILE *fp;
    Image* image = (Image*) malloc(sizeof(Image));
    if (!image) return NULL;
```



```

if ((fp = fopen(filename, "rb")) == NULL) return NULL;
png_ptr = png_create_read_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
if (png_ptr == NULL) {
    free(image);
    fclose(fp);
    return NULL;
}
png_set_error_fn(png_ptr, (png_voidp) NULL, (png_error_ptr) NULL, user_warning_fn);
info_ptr = png_create_info_struct(png_ptr);
if (info_ptr == NULL) {
    free(image);
    fclose(fp);
    png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
    return NULL;
}
png_init_io(png_ptr, fp);
png_set_sig_bytes(png_ptr, sig_read);
png_read_info(png_ptr, info_ptr);
png_get_IHDR(png_ptr, info_ptr, &width, &height, &bit_depth, &color_type, &
interlace_type, int_p_NULL, int_p_NULL);
if (width > 512 || height > 512) {
    free(image);
    fclose(fp);
    png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
    return NULL;
}
image->imageWidth = width;
image->imageHeight = height;
image->textureWidth = getNextPower2(width);
image->textureHeight = getNextPower2(height);
png_set_strip_16(png_ptr);
png_set_packing(png_ptr);
if (color_type == PNG_COLOR_TYPE_PALETTE) png_set_palette_to_rgb(png_ptr);
if (color_type == PNG_COLOR_TYPE_GRAY && bit_depth < 8) png_set_gray_1_2_4_to_8
(png_ptr);
if (png_get_valid(png_ptr, info_ptr, PNG_INFO_tRNS)) png_set_tRNS_to_alpha(png_ptr);
png_set_filler(png_ptr, 0xff, PNG_FILLER_AFTER);
image->data = (Color*) memalign(16, image->textureWidth * image->textureHeight *
sizeof(Color));
if (!image->data) {
    free(image);
    fclose(fp);
    png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
    return NULL;
}
line = (u32*) malloc(width * 4);
if (!line) {
    free(image->data);
    free(image);
    fclose(fp);
    png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
    return NULL;
}
for (y = 0; y < height; y++) {
    png_read_row(png_ptr, (u8*) line, png_bytep_NULL);
    for (x = 0; x < width; x++) {
        u32 color = line[x];
        image->data[x + y * image->textureWidth] = color;
    }
}
free(line);
png_read_end(png_ptr, info_ptr);
png_destroy_read_struct(&png_ptr, &info_ptr, png_infopp_NULL);
fclose(fp);
return image;

```




```

void blitImageToImage(int sx, int sy, int width, int height, Image* source, int dx, int
    dy, Image* destination)
{
    Color* destinationData = &destination->data[destination->textureWidth * dy + dx];
    int destinationSkipX = destination->textureWidth - width;
    Color* sourceData = &source->data[source->textureWidth * sy + sx];
    int sourceSkipX = source->textureWidth - width;
    int x, y;
    for (y = 0; y < height; y++, destinationData += destinationSkipX, sourceData +=
        sourceSkipX) {
        for (x = 0; x < width; x++, destinationData++, sourceData++) {
            *destinationData = *sourceData;
        }
    }
}

void blitImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int
    dy)
{
    if (!initialized) return;
    Color* vram = getVramDrawBuffer();
    sceKernelDcacheWritebackInvalidateAll();
    guStart();
    sceGuCopyImage(GU_PSM_8888, sx, sy, width, height, source->textureWidth, source->data,
        dx, dy, PSP_LINE_SIZE, vram);
    sceGuFinish();
    sceGuSync(0, 0);
}

void blitAlphaImageToImage(int sx, int sy, int width, int height, Image* source, int dx,
    int dy, Image* destination)
{
    // TODO Blend!
    Color* destinationData = &destination->data[destination->textureWidth * dy + dx];
    int destinationSkipX = destination->textureWidth - width;
    Color* sourceData = &source->data[source->textureWidth * sy + sx];
    int sourceSkipX = source->textureWidth - width;
    int x, y;
    for (y = 0; y < height; y++, destinationData += destinationSkipX, sourceData +=
        sourceSkipX) {
        for (x = 0; x < width; x++, destinationData++, sourceData++) {
            Color color = *sourceData;
            if (!IS_ALPHA(color)) *destinationData = color;
        }
    }
}

void blitAlphaImageToScreen(int sx, int sy, int width, int height, Image* source, int dx,
    int dy)
{
    if (!initialized) return;

    sceKernelDcacheWritebackInvalidateAll();
    guStart();
    sceGuTexImage(0, source->textureWidth, source->textureHeight, source->textureWidth,
        (void*) source->data);
    float u = 1.0f / ((float)source->textureWidth);
    float v = 1.0f / ((float)source->textureHeight);
    sceGuTexScale(u, v);

    int j = 0;
    while (j < width) {
        Vertex* vertices = (Vertex*) sceGuGetMemory(2 * sizeof(Vertex));
        int sliceWidth = 64;
        if (j + sliceWidth > width) sliceWidth = width - j;
    }
}

```



```

        vertices[0].u = sx + j;
        vertices[0].v = sy;
        vertices[0].x = dx + j;
        vertices[0].y = dy;
        vertices[0].z = 0;
        vertices[1].u = sx + j + sliceWidth;
        vertices[1].v = sy + height;
        vertices[1].x = dx + j + sliceWidth;
        vertices[1].y = dy + height;
        vertices[1].z = 0;
        sceGuDrawArray(GU_SPRITES, GU_TEXTURE_16BIT | GU_VERTEX_16BIT | GU_TRANSFORM_2D,
2, 0, vertices);
        j += sliceWidth;
    }

    sceGuFinish();
    sceGuSync(0, 0);
}

Image* createImage(int width, int height)
{
    Image* image = (Image*) malloc(sizeof(Image));
    if (!image) return NULL;
    image->imageWidth = width;
    image->imageHeight = height;
    image->textureWidth = getNextPower2(width);
    image->textureHeight = getNextPower2(height);
    image->data = (Color*) memalign(16, image->textureWidth * image->textureHeight *
sizeof(Color));
    if (!image->data) return NULL;
    memset(image->data, 0, image->textureWidth * image->textureHeight * sizeof(Color));
    return image;
}

void freeImage(Image* image)
{
    free(image->data);
    free(image);
}

void clearImage(Color color, Image* image)
{
    int i;
    int size = image->textureWidth * image->textureHeight;
    Color* data = image->data;
    for (i = 0; i < size; i++, data++) *data = color;
}

void clearScreen(Color color)
{
    if (!initialized) return;
    guStart();
    sceGuClearDepth(0);
    sceGuClear(GU_COLOR_BUFFER_BIT|GU_DEPTH_BUFFER_BIT);
    sceGuFinish();
    sceGuSync(0, 0);
}

void fillImageRect(Color color, int x0, int y0, int width, int height, Image* image)
{
    int skipX = image->textureWidth - width;
    int x, y;
    Color* data = image->data + x0 + y0 * image->textureWidth;
    for (y = 0; y < height; y++, data += skipX) {
        for (x = 0; x < width; x++, data++) *data = color;
    }
}

```



```

}

void fillScreenRect(Color color, int x0, int y0, int width, int height)
{
    if (!initialized) return;
    int skipX = PSP_LINE_SIZE - width;
    int x, y;
    Color* data = getVramDrawBuffer() + x0 + y0 * PSP_LINE_SIZE;
    for (y = 0; y < height; y++, data += skipX) {
        for (x = 0; x < width; x++, data++) *data = color;
    }
}

void putPixelScreen(Color color, int x, int y)
{
    Color* vram = getVramDrawBuffer();
    vram[PSP_LINE_SIZE * y + x] = color;
}

void putPixelImage(Color color, int x, int y, Image* image)
{
    image->data[x + y * image->textureWidth] = color;
}

Color getPixelScreen(int x, int y)
{
    Color* vram = getVramDrawBuffer();
    return vram[PSP_LINE_SIZE * y + x];
}

Color getPixelImage(int x, int y, Image* image)
{
    return image->data[x + y * image->textureWidth];
}

void printTextScreen(int x, int y, const char* text, u32 color)
{
    int c, i, j, l;
    u8 *font;
    Color *vram_ptr;
    Color *vram;

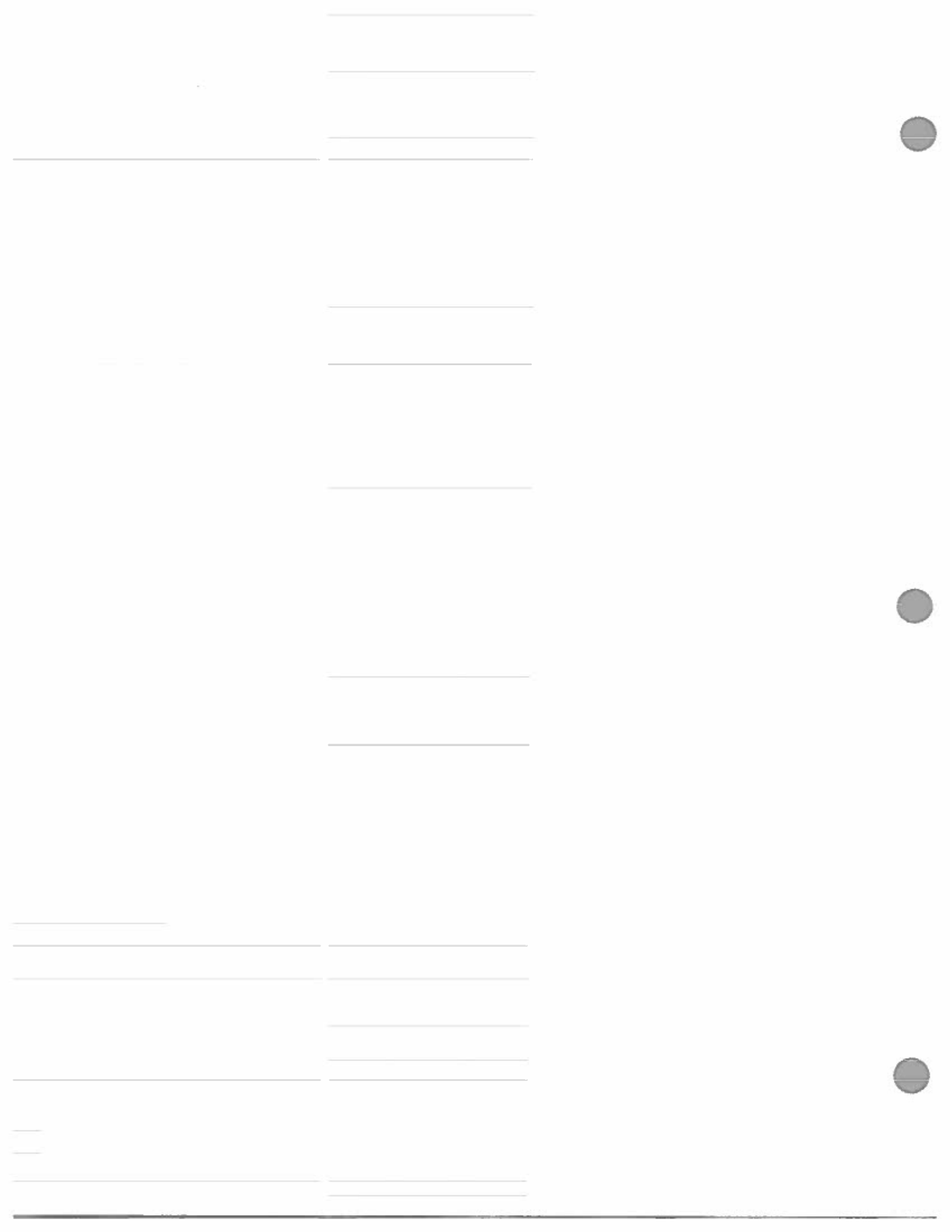
    if (!initialized) return;

    for (c = 0; c < strlen(text); c++) {
        if (x < 0 || x + 8 > SCREEN_WIDTH || y < 0 || y + 8 > SCREEN_HEIGHT) break;
        char ch = text[c];
        vram = getVramDrawBuffer() + x + y * PSP_LINE_SIZE;

        font = &msx[ (int)ch * 8];
        for (i = l = 0; i < 8; i++, l += 8, font++) {
            vram_ptr = vram;
            for (j = 0; j < 8; j++) {
                if ((*font & (128 >> j))) *vram_ptr = color;
                vram_ptr++;
            }
            vram += PSP_LINE_SIZE;
        }
        x += 8;
    }
}

void printTextImage(int x, int y, const char* text, u32 color, Image* image)
{
    int c, i, j, l;
    u8 *font;

```



```

    Color *data_ptr;
    Color *data;

    if (!initialized) return;

    for (c = 0; c < strlen(text); c++) {
        if (x < 0 || x + 8 > image->imageWidth || y < 0 || y + 8 > image->imageHeight)
            break;
        char ch = text[c];
        data = image->data + x + y * image->textureWidth;

        font = &msx[ (int)ch * 8];
        for (i = l = 0; i < 8; i++, l += 8, font++) {
            data_ptr = data;
            for (j = 0; j < 8; j++) {
                if ((*font & (128 >> j))) *data_ptr = color;
                data_ptr++;
            }
            data += image->textureWidth;
        }
        x += 8;
    }
}

void saveImage(const char* filename, Color* data, int width, int height, int lineSize,
               int saveAlpha)
{
    png_structp png_ptr;
    png_infop info_ptr;
    FILE* fp;
    int i, x, y;
    u8* line;

    if ((fp = fopen(filename, "wb")) == NULL) return;
    png_ptr = png_create_write_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
    if (!png_ptr) return;
    info_ptr = png_create_info_struct(png_ptr);
    if (!info_ptr) {
        png_destroy_write_struct(&png_ptr, (png_infopp)NULL);
        return;
    }
    png_init_io(png_ptr, fp);
    png_set_IHDR(png_ptr, info_ptr, width, height, 8,
                 saveAlpha ? PNG_COLOR_TYPE_RGBA : PNG_COLOR_TYPE_RGB,
                 PNG_INTERLACE_NONE, PNG_COMPRESSION_TYPE_DEFAULT, PNG_FILTER_TYPE_DEFAULT);
    png_write_info(png_ptr, info_ptr);
    line = (u8*) malloc(width * (saveAlpha ? 4 : 3));
    for (y = 0; y < height; y++) {
        for (i = 0, x = 0; x < width; x++) {
            Color color = data[x + y * lineSize];
            u8 r = color & 0xff;
            u8 g = (color >> 8) & 0xff;
            u8 b = (color >> 16) & 0xff;
            u8 a = saveAlpha ? (color >> 24) & 0xff : 0xff;
            line[i++] = r;
            line[i++] = g;
            line[i++] = b;
            if (saveAlpha) line[i++] = a;
        }
        png_write_row(png_ptr, line);
    }
    free(line);
    png_write_end(png_ptr, info_ptr);
    png_destroy_write_struct(&png_ptr, (png_infopp)NULL);
    fclose(fp);
}

```




```

void flipScreen()
{
    if (!initialized) return;
    sceGuSwapBuffers();
    dispBufferNumber ^= 1;
}

static void drawLine(int x0, int y0, int x1, int y1, int color, Color* destination, int width)
{
    int dy = y1 - y0;
    int dx = x1 - x0;
    int stepx, stepy;

    if (dy < 0) { dy = -dy; stepy = -width; } else { stepy = width; }
    if (dx < 0) { dx = -dx; stepx = -1; } else { stepx = 1; }
    dy <<= 1;
    dx <<= 1;

    y0 *= width;
    y1 *= width;
    destination[x0+y0] = color;
    if (dx > dy) {
        int fraction = dy - (dx >> 1);
        while (x0 != x1) {
            if (fraction >= 0) {
                y0 += stepy;
                fraction -= dx;
            }
            x0 += stepx;
            fraction += dy;
            destination[x0+y0] = color;
        }
    } else {
        int fraction = dx - (dy >> 1);
        while (y0 != y1) {
            if (fraction >= 0) {
                x0 += stepx;
                fraction -= dy;
            }
            y0 += stepy;
            fraction += dx;
            destination[x0+y0] = color;
        }
    }
}

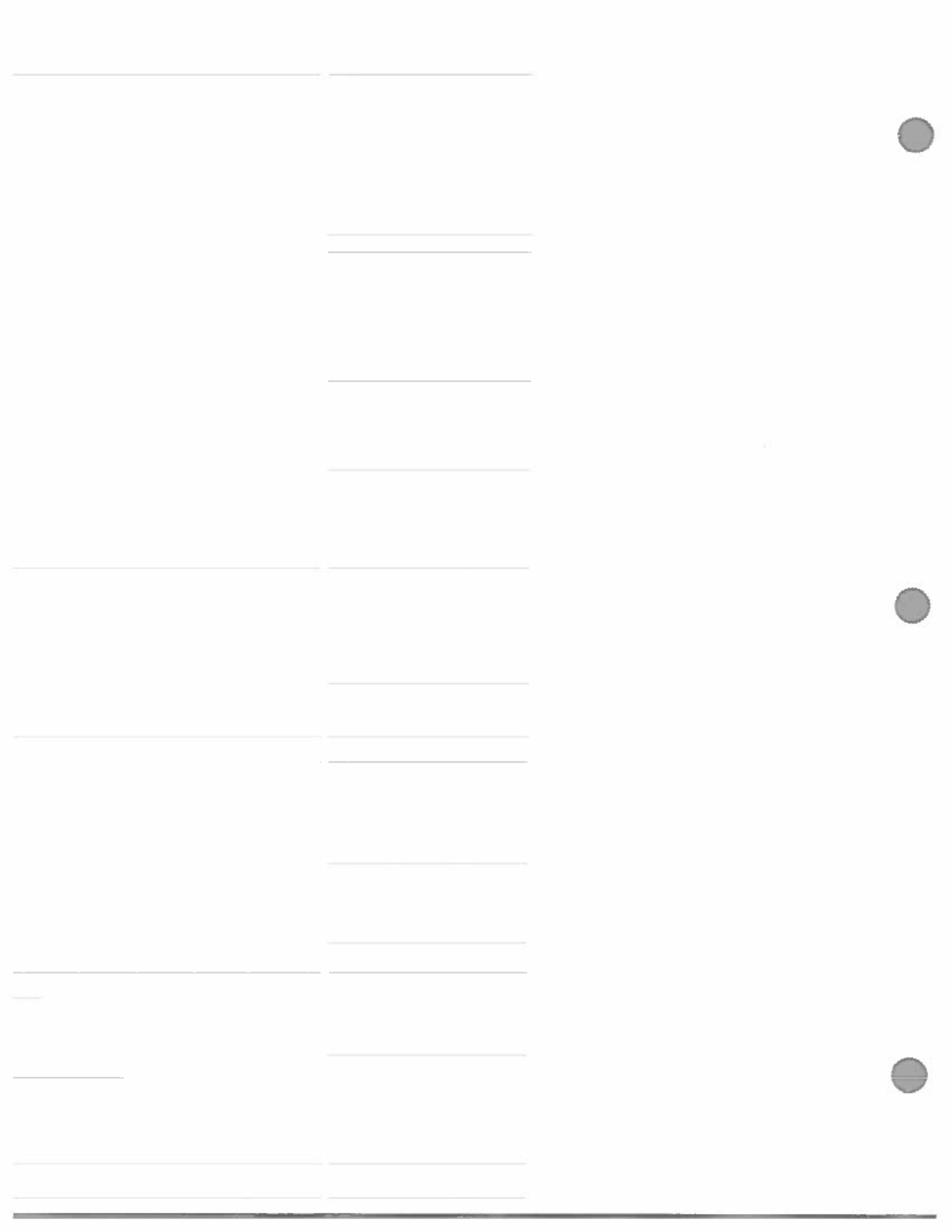
void drawLineScreen(int x0, int y0, int x1, int y1, Color color)
{
    drawLine(x0, y0, x1, y1, color, getVramDrawBuffer(), PSP_LINE_SIZE);
}

void drawLineImage(int x0, int y0, int x1, int y1, Color color, Image* image)
{
    drawLine(x0, y0, x1, y1, color, image->data, image->textureWidth);
}

#define BUF_WIDTH (512)
#define SCR_WIDTH (480)
#define SCR_HEIGHT (272)
#define PIXEL_SIZE (4) /* change this if you change to another screenmode */
#define FRAME_SIZE (BUF_WIDTH * SCR_HEIGHT * PIXEL_SIZE)
#define ZBUF_SIZE (BUF_WIDTH SCR_HEIGHT * 2) /* zbuffer seems to be 16-bit? */

void initGraphics()

```



```
{
    dispBufferNumber = 0;

    sceGuInit();

    guStart();
    sceGuDrawBuffer(GU_PSM_8888, (void*)FRAMEBUFFER_SIZE, PSP_LINE_SIZE);
    sceGuDispBuffer(SCREEN_WIDTH, SCREEN_HEIGHT, (void*)0, PSP_LINE_SIZE);
    sceGuClear(GU_COLOR_BUFFER_BIT | GU_DEPTH_BUFFER_BIT);
    sceGuDepthBuffer((void*) (FRAMEBUFFER_SIZE*2), PSP_LINE_SIZE);
    sceGuOffset(2048 - (SCREEN_WIDTH / 2), 2048 - (SCREEN_HEIGHT / 2));
    sceGuViewport(2048, 2048, SCREEN_WIDTH, SCREEN_HEIGHT);
    sceGuDepthRange(0xc350, 0x2710);
    sceGuScissor(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);
    sceGuEnable(GU_SCISSOR_TEST);
    sceGuAlphaFunc(GU_GREATER, 0, 0xff);
    sceGuEnable(GU_ALPHA_TEST);
    sceGuDepthFunc(GU_EQUAL);
    sceGuEnable(GU_DEPTH_TEST);
    sceGuFrontFace(GU_CW);
    sceGuShadeModel(GU_SMOOTH);
    sceGuEnable(GU_CULL_FACE);
    sceGuEnable(GU_TEXTURE_2D);
    sceGuEnable(GU_CLIP_PLANES);
    sceGuTexMode(GU_PSM_8888, 0, 0, 0);
    sceGuTexFunc(GU_TFX_REPLACE, GU_TCC_RGBA);
    sceGuTexFilter(GU_NEAREST, GU_NEAREST);
    sceGuAmbientColor(0xffffffff);
    sceGuEnable(GU_BLEND);
    sceGuBlendFunc(GU_ADD, GU_SRC_ALPHA, GU_ONE_MINUS_SRC_ALPHA, 0, 0);
    sceGuFinish();
    sceGuSync(0, 0);

    sceDisplayWaitVblankStart();
    sceGuDisplay(GU_TRUE);
    initialized = 1;
}

void disableGraphics()
{
    initialized = 0;
}

void guStart()
{
    sceGuStart(GU_DIRECT, list);
}
```

20
21

22

23

24

```

#ifndef GRAPHICS_H
#define GRAPHICS_H

#include <psptypes.h>

#define PSP_LINE_SIZE 512
#define SCREEN_WIDTH 480
#define SCREEN_HEIGHT 272

typedef u32 Color;
#define A(color) ((u8)(color >> 24 & 0xFF))
#define B(color) ((u8)(color >> 16 & 0xFF))
#define G(color) ((u8)(color >> 8 & 0xFF))
#define R(color) ((u8)(color & 0xFF))

typedef struct
{
    int textureWidth; // the real width of data, 2^n with n>=0
    int textureHeight; // the real height of data, 2^n with n>=0
    int imageWidth; // the image width
    int imageHeight;
    Color* data;
} Image;

/**
 * Load a PNG image.
 *
 * @pre filename != NULL
 * @param filename - filename of the PNG image to load
 * @return pointer to a new allocated Image struct, or NULL on failure
 */
extern Image* loadImage(const char* filename);

/**
 * Blit a rectangle part of an image to another image.
 *
 * @pre source != NULL && destination != NULL &&
 *      sx >= 0 && sy >= 0 &&
 *      width > 0 && height > 0 &&
 *      sx + width <= source->width && sy + height <= source->height &&
 *      dx + width <= destination->width && dy + height <= destination->height
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 * @param destination - pointer to Image struct of the destination image
 */
extern void blitImageToImage(int sx, int sy, int width, int height, Image* source, int dx,
    int dy, Image* destination);

/**
 * Blit a rectangle part of an image to screen.
 *
 * @pre source != NULL && destination != NULL &&
 *      sx >= 0 && sy >= 0 &&
 *      width > 0 && height > 0 &&
 *      sx + width <= source->width && sy + height <= source->height &&
 *      dx + width <= SCREEN_WIDTH && dy + height <= SCREEN_HEIGHT
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image

```



```

* @param dx - left target position in destination image
* @param dy - top target position in destination image
*/
extern void blitImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int dy);

/**
 * Blit a rectangle part of an image to another image without alpha pixels in source image.
 *
 * @pre source != NULL && destination != NULL &&
 *       sx >= 0 && sy >= 0 &&
 *       width > 0 && height > 0 &&
 *       sx + width <= source->width && sy + height <= source->height &&
 *       dx + width <= destination->width && dy + height <= destination->height
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 * @param destination - pointer to Image struct of the destination image
 */
extern void blitAlphaImageToImage(int sx, int sy, int width, int height, Image* source, int dx, int dy, Image* destination);

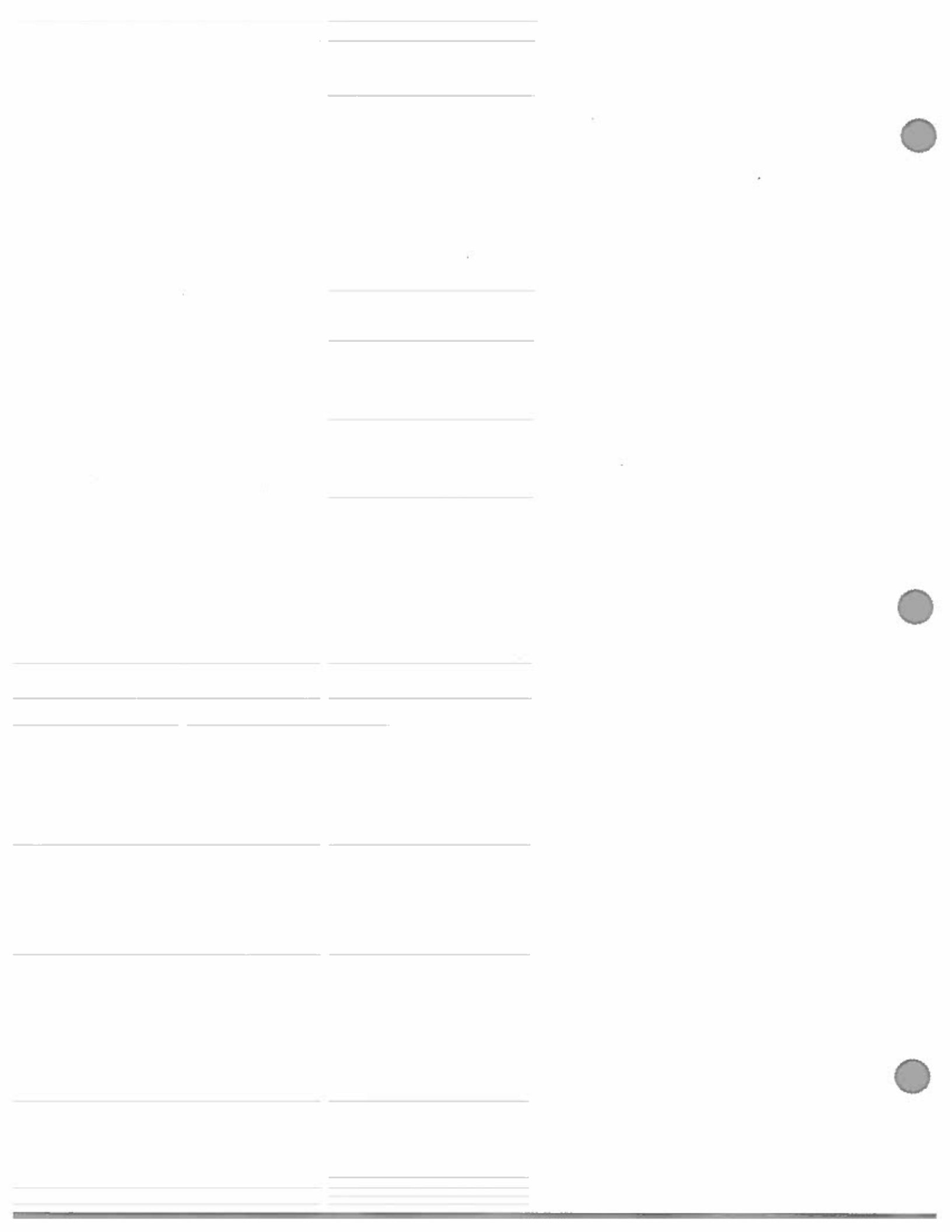
/**
 * Blit a rectangle part of an image to screen without alpha pixels in source image.
 *
 * @pre source != NULL && destination != NULL &&
 *       sx >= 0 && sy >= 0 &&
 *       width > 0 && height > 0 &&
 *       sx + width <= source->width && sy + height <= source->height &&
 *       dx + width <= SCREEN_WIDTH && dy + height <= SCREEN_HEIGHT
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 */
extern void blitAlphaImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int dy);

/**
 * Create an empty image.
 *
 * @pre width > 0 && height > 0 && width <= 512 && height <= 512
 * @param width - width of the new image
 * @param height - height of the new image
 * @return pointer to a new allocated Image struct, all pixels initialized to color 0, or NULL on failure
 */
extern Image* createImage(int width, int height);

/**
 * Frees an allocated image.
 *
 * @pre image != null
 * @param image a pointer to an image struct
 */
extern void freeImage(Image* image);

/**

```




```
* Initialize all pixels of an image with a color.
*
* @pre image != NULL
* @param color - new color for the pixels
* @param image - image to clear
*/
extern void clearImage(Color color, Image* image);

/**
* Initialize all pixels of the screen with a color.
*
* @param color - new color for the pixels
*/
extern void clearScreen(Color color);

/**
* Fill a rectangle of an image with a color.
*
* @pre image != NULL
* @param color - new color for the pixels
* @param x0 - left position of rectangle in image
* @param y0 - top position of rectangle in image
* @param width - width of rectangle in image
* @param height - height of rectangle in image
* @param image - image
*/
extern void fillImageRect(Color color, int x0, int y0, int width, int height, Image*
    image);

/**
* Fill a rectangle of an image with a color.
*
* @pre image != NULL
* @param color - new color for the pixels
* @param x0 - left position of rectangle in image
* @param y0 - top position of rectangle in image
* @param width - width of rectangle in image
* @param height - height of rectangle in image
*/
extern void fillScreenRect(Color color, int x0, int y0, int width, int height);

/**
* Set a pixel on screen to the specified color.
*
* @pre x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT
* @param color - new color for the pixels
* @param x - left position of the pixel
* @param y - top position of the pixel
*/
extern void putPixelScreen(Color color, int x, int y);

/**
* Set a pixel in an image to the specified color.
*
* @pre x >= 0 && x < image->imageWidth && y >= 0 && y < image->imageHeight && image !=
    NULL
* @param color - new color for the pixels
* @param x - left position of the pixel
* @param y - top position of the pixel
*/
extern void putPixelImage(Color color, int x, int y, Image* image);

/**
* Get the color of a pixel on screen.
*
* @pre x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT
```



```

* @param x - left position of the pixel
* @param y - top position of the pixel
* @return the color of the pixel
*/
extern Color getPixelScreen(int x, int y);

/**
 * Get the color of a pixel of an image.
 *
 * @pre x >= 0 && x < image->imageWidth && y >= 0 && y < image->imageHeight && image != NULL
 * @param x - left position of the pixel
 * @param y - top position of the pixel
 * @return the color of the pixel
 */
extern Color getPixelImage(int x, int y, Image* image);

/**
 * Print a text (pixels out of the screen or image are clipped).
 *
 * @param x - left position of text
 * @param y - top position of text
 * @param text - the text to print
 * @param color - new color for the pixels
 */
extern void printTextScreen(int x, int y, const char* text, u32 color);

/**
 * Print a text (pixels out of the screen or image are clipped).
 *
 * @param x - left position of text
 * @param y - top position of text
 * @param text - the text to print
 * @param color - new color for the pixels
 * @param image - image
 */
extern void printTextImage(int x, int y, const char* text, u32 color, Image* image);

/**
 * Save an image or the screen in PNG format.
 *
 * @pre filename != NULL
 * @param filename - filename of the PNG image
 * @param data - start of Color type pixel data (can be getVramDisplayBuffer())
 * @param width - logical width of the image or SCREEN_WIDTH
 * @param height - height of the image or SCREEN_HEIGHT
 * @param lineSize - physical width of the image or PSP_LINE_SIZE
 * @param saveAlpha - if 0, image is saved without alpha channel
 */
extern void saveImage(const char* filename, Color* data, int width, int height, int lineSize, int saveAlpha);

/**
 * Exchange display buffer and drawing buffer.
 */
extern void flipScreen();

/**
 * Initialize the graphics.
 */
extern void initGraphics();

/**
 * Disable graphics, used for debug text output.
 */
extern void disableGraphics();

```

```
/**
 * Draw a line to screen.
 *
 * @pre x0 >= 0 && x0 < SCREEN_WIDTH && y0 >= 0 && y0 < SCREEN_HEIGHT &&
 *       x1 >= 0 && x1 < SCREEN_WIDTH && y1 >= 0 && y1 < SCREEN_HEIGHT
 * @param x0 - x line start position
 * @param y0 - y line start position
 * @param x1 - x line end position
 * @param y1 - y line end position
 */
void drawLineScreen(int x0, int y0, int x1, int y1, Color color);

/**
 * Draw a line to screen.
 *
 * @pre x0 >= 0 && x0 < image->imageWidth && y0 >= 0 && y0 < image->imageHeight &&
 *       x1 >= 0 && x1 < image->imageWidth && y1 >= 0 && y1 < image->imageHeight
 * @param x0 - x line start position
 * @param y0 - y line start position
 * @param x1 - x line end position
 * @param y1 - y line end position
 */
extern void drawLineImage(int x0, int y0, int x1, int y1, Color color, Image* image);

/**
 * Get the current draw buffer for fast unchecked access.
 *
 * @return the start address of the current draw buffer
 */
extern Color* getVramDrawBuffer();

/**
 * Get the current display buffer for fast unchecked access.
 *
 * @return the start address of the current display buffer
 */
extern Color* getVramDisplayBuffer();

extern void guStart();

#endif
```



```
#ifndef FRAMEBUFFER_H
#define FRAMEBUFFER_H

#include <psptypes.h>

extern u32* g_vram_base;

#endif
```



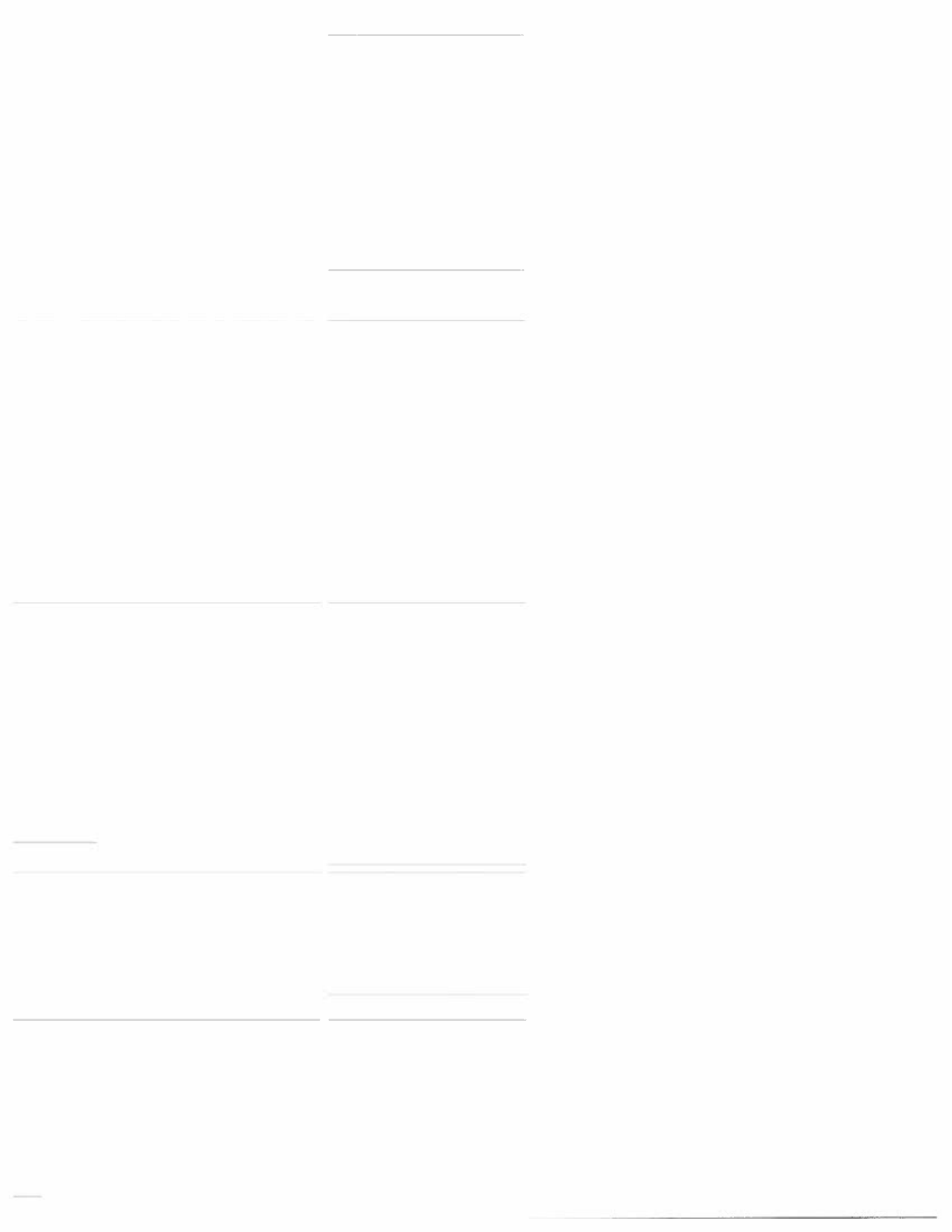
=====

```
#include <psptypes.h>
#include "graphics.h"
```

```
Color* g_vram_base = (Color*) (0x40000000 | 0x04000000);
```



Slide Show



PSP Dijkstra GPS

Programmer: Dru Sepulveda

Advisor: Dr. Yu-ju Lin

Project Purpose

- To make a easy to use, fully functional map program that can dynamically guide a user around the Charleston Southern Campus using the Sony PSP as a platform, the GPSSlim236 by Holux to find the users position and Dijkstra's Theorem to find the shortest path from point to point.

Project History

- Internet Browser
 - The 2.0 PSP firmware had it's own browser and I did not want to compete with it.
- Sony IRPS Project
 - The SIRC did not have high level documentation and the timing from controller to PSP proved to be inconsistent.
 - IR is not consistent

Project History

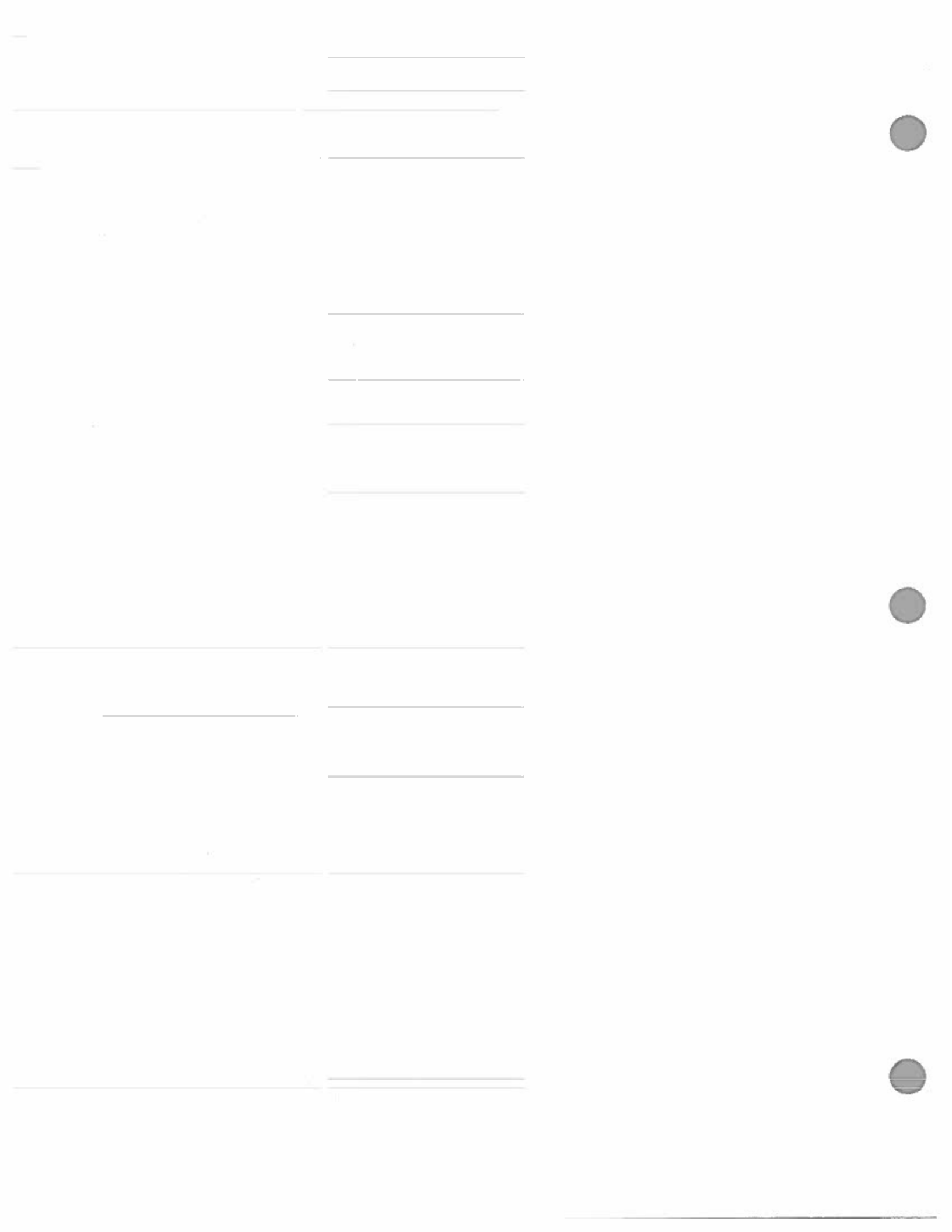
- Sony IRPS / GPS
 - A synthesis was proposed, but discontinued because of the time restriction. This synthesis is still a applicable future addition.
- Sony Dijkstra GPS
 - The current and final project that was chosen because of time and the increased functionality and potential of using GPS instead of IR.

Problems and Solutions

- P: Where to start?
- S: Dr. Lin, Forums
- P: Documentation?
- S: What documentation? Experiment and find out on your own.
- P: Debug on the PSP?
- S: Nearly all debug was done on an alternate Linux console version of the program and had to be constantly be updated with the PSP only version.
- P: Integrate the GPS?
- S: Solder points, resistors and tape, oh my.

Continuing Problems

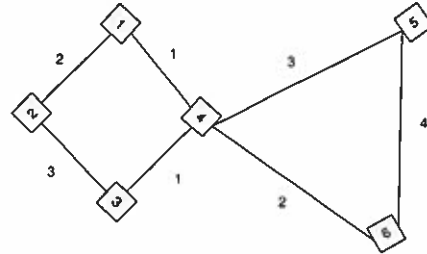
- Some things were never implemented or not supported by the compiler, so some of the information being produced could not be printed to the screen.
 - ltoa() was not implemented



Continuing Problems

- Non PNG picture graphics support (in C)
 - Bitmap can also be used but the buffer did not like the way I did it
- Lack of documentation
 - Always a problem with hacked software
- Lack of reliability with unoriginal code
 - Also, usually a problem with hacked software

Dijkstra Example



What Is What

- "V" is the designation of the current node
- "Known" is whether or not path is known
- "dV" is the current shortest distance to that node
- "pV" is the node that it was found to be adjacent and closer to than any other node

Table

V	Known	dV	pV
V1	F	∞	0
V2	F	∞	0
V3	F	∞	0
V4	F	∞	0
V5	F	∞	0
V6	F	∞	0

Table

V	Known	dV	pV
V1	F	0	0
V2	F	∞	0
V3	F	∞	0
V4	F	∞	0
V5	F	∞	0
V6	F	∞	0

Table

V	Known	dV	pV
V1	T	0	0
V2	F	2	V1
V3	F	∞	0
V4	F	1	V1
V5	F	∞	0
V6	F	∞	0



Table

V	Known	dV	pV
V1	T	0	0
V2	F	2	V1
V3	F	2	V4
V4	T	1	V1
V5	F	4	V4
V6	F	3	V4

Table

V	Known	dV	pV
V1	T	0	0
V2	T	2	V1
V3	F	2	V4
V4	T	1	V1
V5	F	4	V4
V6	F	3	V4

Table

V	Known	dV	pV
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	F	4	V4
V6	F	3	V4

Table

V	Known	dV	pV
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	F	4	V4
V6	T	3	V4

Table

V	Known	dV	pV
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	T	4	V4
V6	T	3	V4

Table

V	Known	dV	pV
V1	T	0	0
V2	T	2	V1
V3	T	2	V4
V4	T	1	V1
V5	T	4	V4
V6	T	3	V4



Recursive Solution For Path

- Ask for currents previous and change current to currents previous after each call until a termination value is found.
- $V6.pV \rightarrow V4.pV \rightarrow V1.pv \rightarrow 0$

Distance

- Current and total distance are easy to find.
- Current Distance = $V_{\text{---}}.dV$
- Total Distance = $V_{\text{---}}.dV$

My Contributions

- Implementation of Dijkstra's Algorithm to fit the needs of the greater GPS Program
- Implementation of effective data structures to handle the information used
- Creation of a drawing module to handle writing of specific information to the screen and interface with the graphics drivers

My Contribution

- Design and implementation of:
 - the GUI
 - a software interface to the GPS Drivers
 - a Indexing module to keep track of all the other modules

The GPS

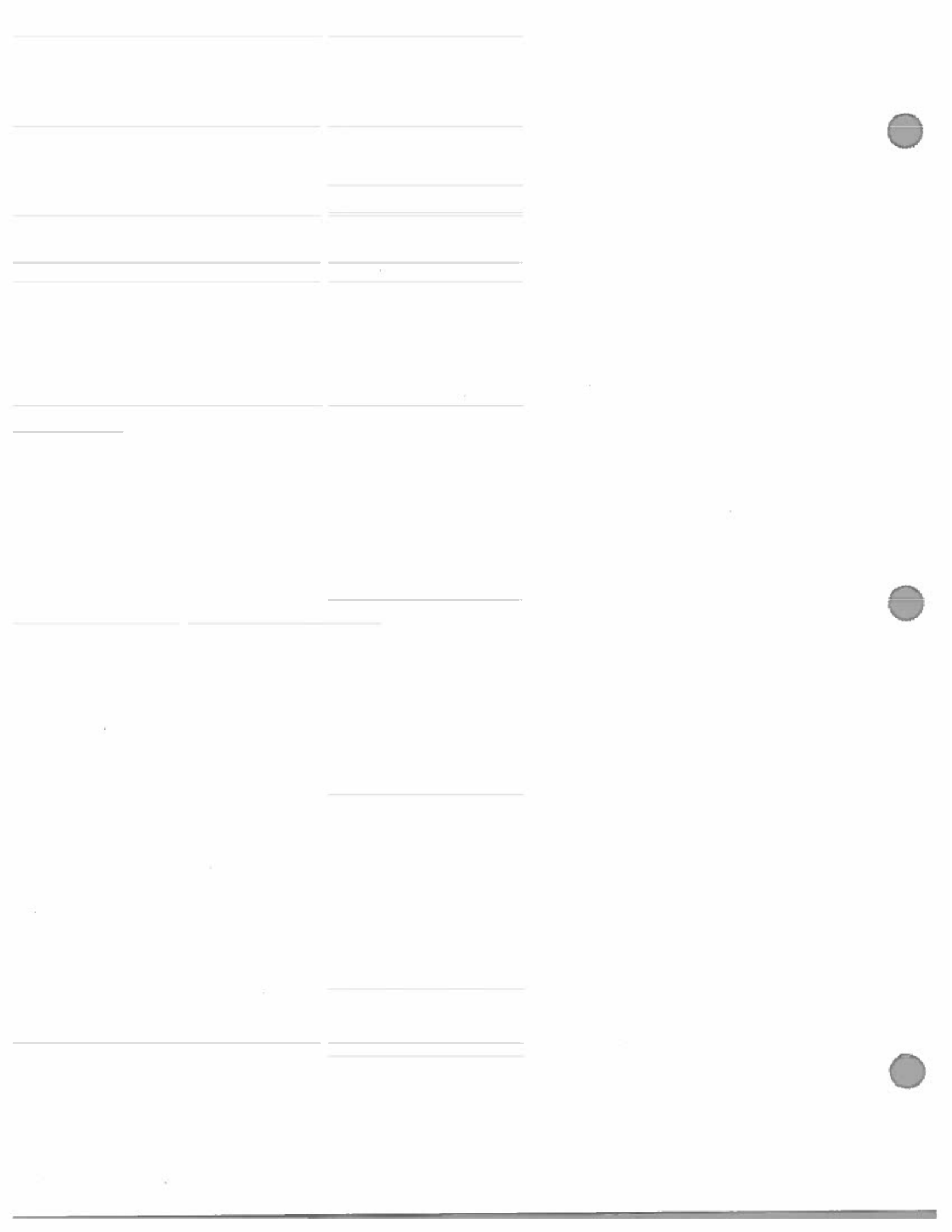
- GPSSlim236 by Holux
- Dinsky and his GPS drivers
- The serial interface and the USB miniB connection wire
- Soldering and tires



Homebrew



- The community
- The forums
- FOSS, the sharing of data and knowledge for everyone's benefit



The Next Steps

- Audio
 - For visually impaired users and ease of use
- Increase the functionality of the buttons
 - Add support for diagonal mouse capture
- Add more modes to the program for increased functionality
 - Emergency escape mode or handicap accessible mode
- More intuitive graphics

The Next Steps

- Interfacing the GPS with the PSP
 - The resistors I am using are too strong for the output, but my tools are too large and unwieldy to accurately solder the small points again
- Add IR functions back to the project
 - These functions make it more marketable to companies like Disney or museums like the Smithsonian
- Make maps more dynamic with zoom and scroll functions
 - This would also increase its marketability and make it so that other maps could easily be added

Why keep going?

- The next generation is on the way.
 - This project has been shown to be applicable to other platforms



(Dr Paragus at Clemson is doing very similar research with PDAs)

•Questions?

Credits

- This project was made possible by the invaluable guidance of Dr Lin and the efforts of the FOSS community. Through the unselfish sharing of information we can make a better world.





Makefile

```
TARGET = main
OBJS = main.o graphics.o framebuffer.o

CFLAGS = -O2 -G0 -Wall
CXXFLAGS = $(CFLAGS) -fno-exceptions -fno-rtti
ASFLAGS = $(CFLAGS)

LIBDIR =
LIBS = -lpspgu -lpng -lz -lm
LDFLAGS =

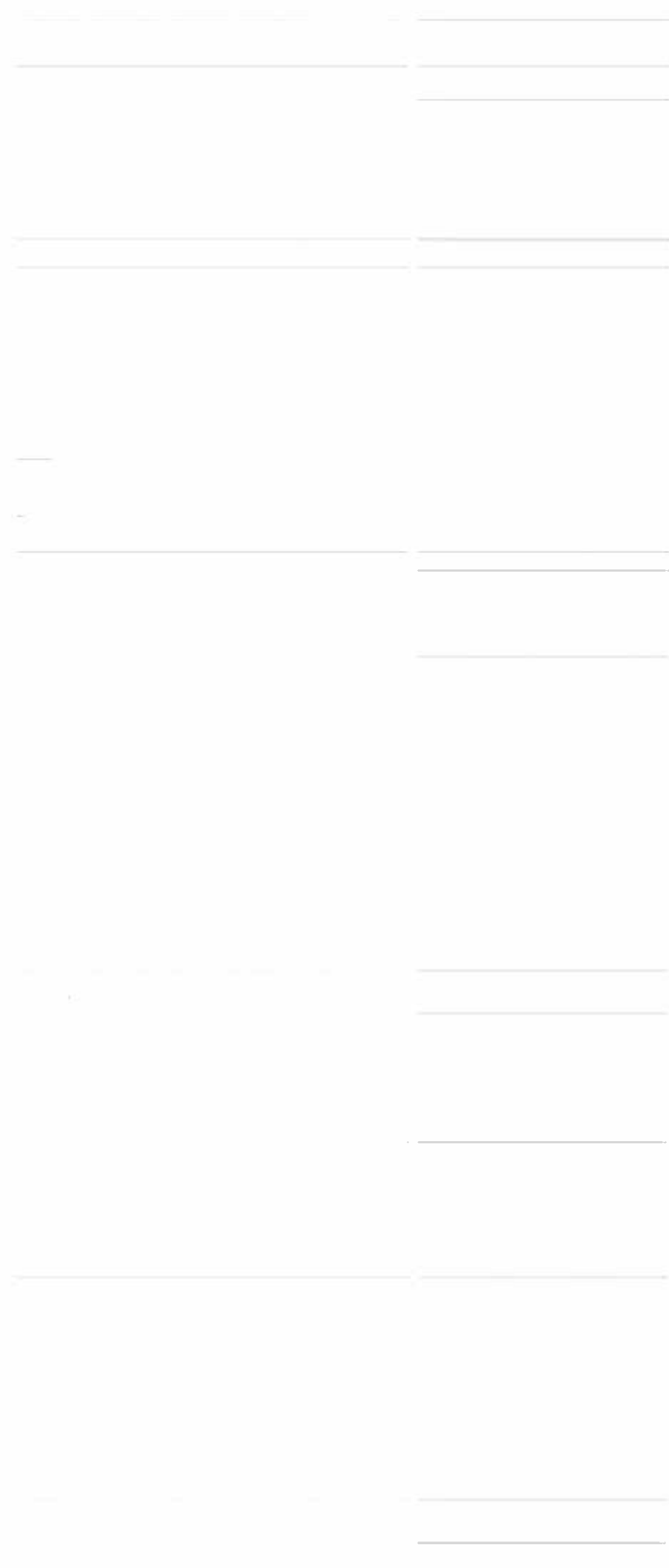
EXTRA_TARGETS = EBOOT.PBP
PSP_EBOOT_TITLE = DijkstraGPS

PSPSDK=$(shell psp-config --pspsdk-path)
include $(PSPSDK)/lib/build.mak
```

9m AS zutuh

11-9-06

old draft




```
// Dru Sepulveda
// 11-5-06
// file to pass the current location to all .h files that need it

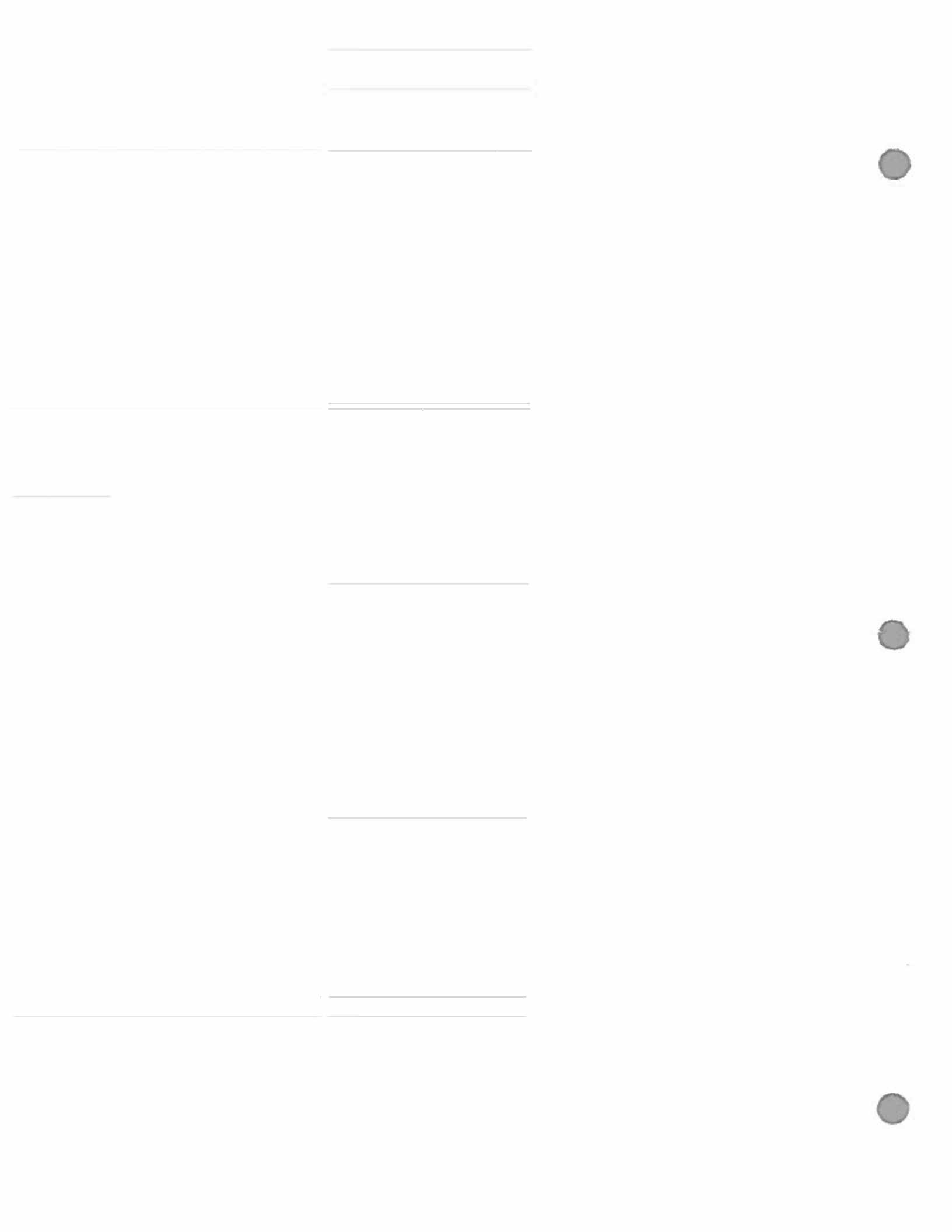
#ifndef __pass__
#define __pass__

//////////variables//////////
float passX = 0;    //x position of user
float passY = 0;    //y position of user
//////////

//////////prototype//////////
void passXY(float pX, float pY);    //makes sure that this .h file has access to the information
//////////

void passXY(float pX, float pY){    //gathers x and y for user to pass to other files
    passX = pX;
    passY = pY;
}

#endif
```



```
void initMap(){
    map[0][0]=0;
    map[0][1]=Map_Zero[1];
    map[0][2]=Map_Zero[2];
    map[0][3]=Map_Zero[3];
    map[0][4]=Map_Zero[4];
```

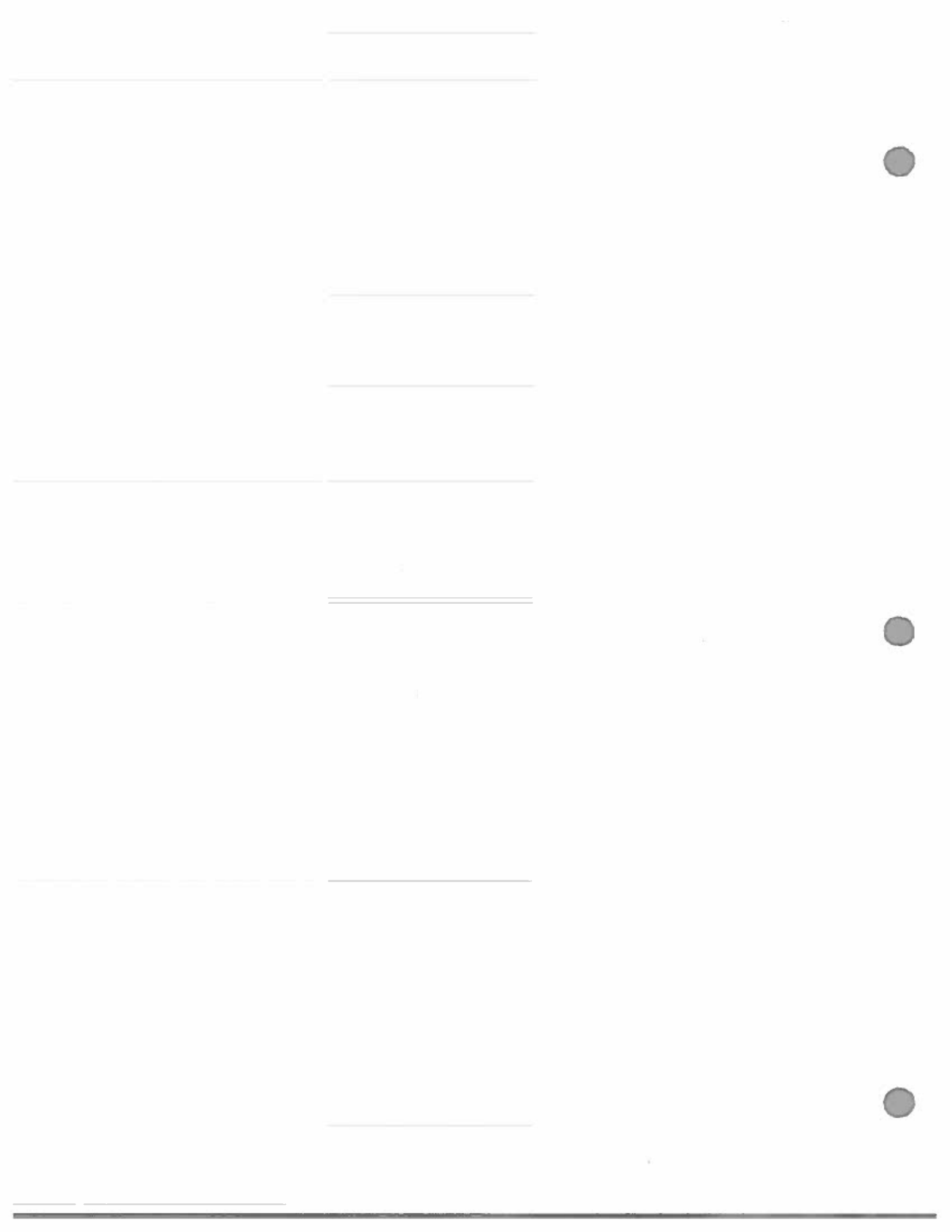


```
map[0][5]=Map_Zero[5];
map[0][6]=Map_Zero[6];
map[0][7]=Map_Zero[7];
map[0][8]=Map_Zero[8];
map[0][9]=Map_Zero[9];
map[0][10]=Map_Zero[10];
map[0][11]=Map_Zero[11];
map[0][12]=Map_Zero[12];
map[0][13]=Map_Zero[13];
map[0][14]=Map_Zero[14];
map[0][15]=Map_Zero[15];
map[0][16]=Map_Zero[16];
map[0][17]=Map_Zero[17];
map[0][18]=Map_Zero[18];
map[0][19]=Map_Zero[19];
map[0][20]=Map_Zero[20];
map[0][21]=Map_Zero[21];
```

```
map[1][0]=Map_Zero[1];
map[1][1]=0;
map[1][2]=31;
map[1][3]=INFINITY;
map[1][4]=INFINITY;
map[1][5]=INFINITY;
map[1][6]=INFINITY;
map[1][7]=INFINITY;
map[1][8]=INFINITY;
map[1][9]=INFINITY;
map[1][10]=INFINITY;
map[1][11]=INFINITY;
map[1][12]=INFINITY;
map[1][13]=INFINITY;
map[1][14]=INFINITY;
map[1][15]=INFINITY;
map[1][16]=INFINITY;
map[1][17]=INFINITY;
map[1][18]=INFINITY;
map[1][19]=INFINITY;
map[1][20]=INFINITY;
map[1][21]=INFINITY;
```

```
map[2][0]=Map_Zero[2];
map[2][1]=31;
map[2][2]=0;
map[2][3]=40;
map[2][4]=INFINITY;
map[2][5]=INFINITY;
map[2][6]=INFINITY;
map[2][7]=INFINITY;
map[2][8]=INFINITY;
map[2][9]=INFINITY;
map[2][10]=INFINITY;
map[2][11]=INFINITY;
map[2][12]=INFINITY;
map[2][13]=INFINITY;
map[2][14]=INFINITY;
map[2][15]=INFINITY;
map[2][16]=INFINITY;
map[2][17]=INFINITY;
map[2][18]=INFINITY;
map[2][19]=INFINITY;
map[2][20]=INFINITY;
map[2][21]=INFINITY;
```

```
map[3][0]=Map_Zero[3];
map[3][1]=INFINITY;
```



```
map[3][2]=40;
map[3][3]=0;
map[3][4]=44;
map[3][5]=INFINITY;
map[3][6]=INFINITY;
map[3][7]=INFINITY;
map[3][8]=35;
map[3][9]=INFINITY;
map[3][10]=INFINITY;
map[3][11]=INFINITY;
map[3][12]=INFINITY;
map[3][13]=INFINITY;
map[3][14]=INFINITY;
map[3][15]=INFINITY;
map[3][16]=INFINITY;
map[3][17]=INFINITY;
map[3][18]=INFINITY;
map[3][19]=INFINITY;
map[3][20]=INFINITY;
map[3][21]=INFINITY;

map[4][0]=Map_Zero[4];
map[4][1]=INFINITY;
map[4][2]=INFINITY;
map[4][3]=44;
map[4][4]=0;
map[4][5]=9;
map[4][6]=68;
map[4][7]=INFINITY;
map[4][8]=INFINITY;
map[4][9]=INFINITY;
map[4][10]=INFINITY;
map[4][11]=INFINITY;
map[4][12]=INFINITY;
map[4][13]=INFINITY;
map[4][14]=INFINITY;
map[4][15]=INFINITY;
map[4][16]=INFINITY;
map[4][17]=INFINITY;
map[4][18]=INFINITY;
map[4][19]=INFINITY;
map[4][20]=INFINITY;
map[4][21]=INFINITY;

map[5][0]=Map_Zero[5];
map[5][1]=INFINITY;
map[5][2]=INFINITY;
map[5][3]=INFINITY;
map[5][4]=9;
map[5][5]=0;
map[5][6]=INFINITY;
map[5][7]=INFINITY;
map[5][8]=INFINITY;
map[5][9]=INFINITY;
map[5][10]=INFINITY;
map[5][11]=INFINITY;
map[5][12]=INFINITY;
map[5][13]=INFINITY;
map[5][14]=INFINITY;
map[5][15]=INFINITY;
map[5][16]=INFINITY;
map[5][17]=INFINITY;
map[5][18]=INFINITY;
map[5][19]=INFINITY;
map[5][20]=INFINITY;
map[5][21]=INFINITY;
```




```
map[6][0]=Map_Zero[6];
map[6][1]=INFINITY;
map[6][2]=INFINITY;
map[6][3]=INFINITY;
map[6][4]=68;
map[6][5]=INFINITY;
map[6][6]=0;
map[6][7]=9;
map[6][8]=INFINITY;
map[6][9]=INFINITY;
map[6][10]=INFINITY;
map[6][11]=INFINITY;
map[6][12]=INFINITY;
map[6][13]=INFINITY;
map[6][14]=INFINITY;
map[6][15]=INFINITY;
map[6][16]=INFINITY;
map[6][17]=INFINITY;
map[6][18]=INFINITY;
map[6][19]=INFINITY;
map[6][20]=INFINITY;
map[6][21]=INFINITY;
```

```
map[7][0]=Map_Zero[7];
map[7][1]=INFINITY;
map[7][2]=INFINITY;
map[7][3]=INFINITY;
map[7][4]=INFINITY;
map[7][5]=INFINITY;
map[7][6]=9;
map[7][7]=0;
map[7][8]=INFINITY;
map[7][9]=INFINITY;
map[7][10]=INFINITY;
map[7][11]=INFINITY;
map[7][12]=INFINITY;
map[7][13]=INFINITY;
map[7][14]=INFINITY;
map[7][15]=INFINITY;
map[7][16]=INFINITY;
map[7][17]=INFINITY;
map[7][18]=INFINITY;
map[7][19]=INFINITY;
map[7][20]=INFINITY;
map[7][21]=INFINITY;
```

```
map[8][0]=Map_Zero[8];
map[8][1]=INFINITY;
map[8][2]=INFINITY;
map[8][3]=35;
map[8][4]=INFINITY;
map[8][5]=INFINITY;
map[8][6]=INFINITY;
map[8][7]=INFINITY;
map[8][8]=0;
map[8][9]=12;
map[8][10]=INFINITY;
map[8][11]=7;
map[8][12]=INFINITY;
map[8][13]=INFINITY;
map[8][14]=INFINITY;
map[8][15]=INFINITY;
map[8][16]=INFINITY;
map[8][17]=INFINITY;
map[8][18]=INFINITY;
```

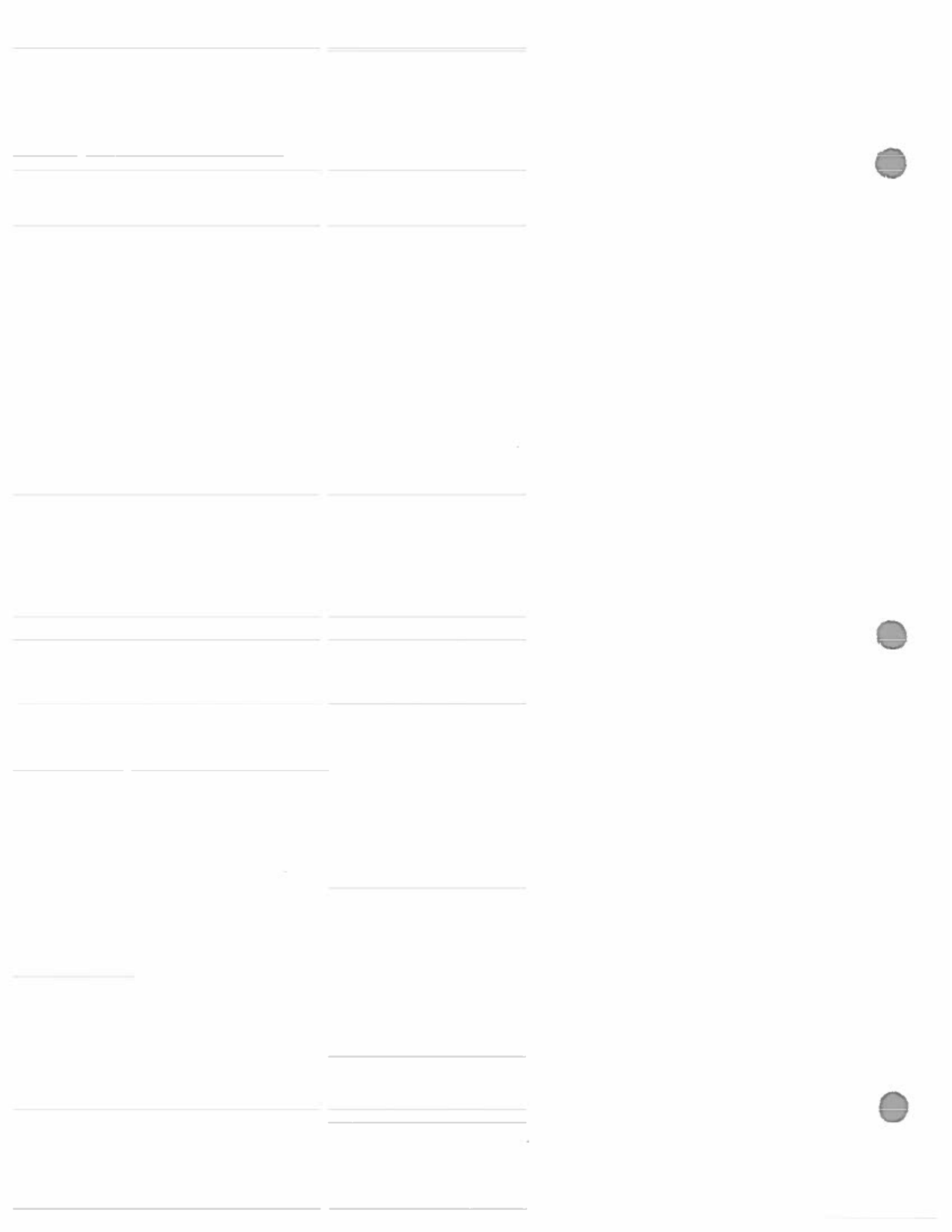


```
map[8][19]=INFINITY;
map[8][20]=INFINITY;
map[8][21]=INFINITY;
```

```
map[9][0]=Map_Zero[9];
map[9][1]=INFINITY;
map[9][2]=INFINITY;
map[9][3]=INFINITY;
map[9][4]=INFINITY;
map[9][5]=INFINITY;
map[9][6]=INFINITY;
map[9][7]=INFINITY;
map[9][8]=12;
map[9][9]=0;
map[9][10]=6;
map[9][11]=INFINITY;
map[9][12]=INFINITY;
map[9][13]=INFINITY;
map[9][14]=INFINITY;
map[9][15]=INFINITY;
map[9][16]=INFINITY;
map[9][17]=INFINITY;
map[9][18]=INFINITY;
map[9][19]=28;
map[9][20]=INFINITY;
map[9][21]=53;
```

```
map[10][0]=Map_Zero[10];
map[10][1]=INFINITY;
map[10][2]=INFINITY;
map[10][3]=INFINITY;
map[10][4]=INFINITY;
map[10][5]=INFINITY;
map[10][6]=INFINITY;
map[10][7]=INFINITY;
map[10][8]=INFINITY;
map[10][9]=6;
map[10][10]=0;
map[10][11]=INFINITY;
map[10][12]=INFINITY;
map[10][13]=INFINITY;
map[10][14]=INFINITY;
map[10][15]=INFINITY;
map[10][16]=INFINITY;
map[10][17]=INFINITY;
map[10][18]=INFINITY;
map[10][19]=INFINITY;
map[10][20]=INFINITY;
map[10][21]=INFINITY;
```

```
map[11][0]=Map_Zero[11];
map[11][1]=INFINITY;
map[11][2]=INFINITY;
map[11][3]=INFINITY;
map[11][4]=INFINITY;
map[11][5]=INFINITY;
map[11][6]=INFINITY;
map[11][7]=INFINITY;
map[11][8]=7;
map[11][9]=INFINITY;
map[11][10]=INFINITY;
map[11][11]=0;
map[11][12]=10;
map[11][13]=INFINITY;
map[11][14]=22;
map[11][15]=INFINITY;
```



```
map[11][16]=INFINITY;
map[11][17]=INFINITY;
map[11][18]=INFINITY;
map[11][19]=INFINITY;
map[11][20]=INFINITY;
map[11][21]=INFINITY;
```

```
map[12][0]=Map_Zero[12];
map[12][1]=INFINITY;
map[12][2]=INFINITY;
map[12][3]=INFINITY;
map[12][4]=INFINITY;
map[12][5]=INFINITY;
map[12][6]=INFINITY;
map[12][7]=INFINITY;
map[12][8]=INFINITY;
map[12][9]=INFINITY;
map[12][10]=INFINITY;
map[12][11]=10;
map[12][12]=0;
map[12][13]=INFINITY;
map[12][14]=INFINITY;
map[12][15]=INFINITY;
map[12][16]=INFINITY;
map[12][17]=INFINITY;
map[12][18]=INFINITY;
map[12][19]=INFINITY;
map[12][20]=INFINITY;
map[12][21]=INFINITY;
```

```
map[13][0]=Map_Zero[13];
map[13][1]=INFINITY;
map[13][2]=INFINITY;
map[13][3]=INFINITY;
map[13][4]=INFINITY;
map[13][5]=INFINITY;
map[13][6]=INFINITY;
map[13][7]=INFINITY;
map[13][8]=INFINITY;
map[13][9]=INFINITY;
map[13][10]=INFINITY;
map[13][11]=INFINITY;
map[13][12]=INFINITY;
map[13][13]=0;
map[13][14]=6;
map[13][15]=INFINITY;
map[13][16]=INFINITY;
map[13][17]=INFINITY;
map[13][18]=INFINITY;
map[13][19]=INFINITY;
map[13][20]=INFINITY;
map[13][21]=INFINITY;
```

```
map[14][0]=Map_Zero[14];
map[14][1]=INFINITY;
map[14][2]=INFINITY;
map[14][3]=INFINITY;
map[14][4]=INFINITY;
map[14][5]=INFINITY;
map[14][6]=INFINITY;
map[14][7]=INFINITY;
map[14][8]=INFINITY;
map[14][9]=INFINITY;
map[14][10]=INFINITY;
map[14][11]=22;
map[14][12]=INFINITY;
```



1

2





```
map[14][13]=6;
map[14][14]=0;
map[14][15]=15;
map[14][16]=INFINITY;
map[14][17]=INFINITY;
map[14][18]=INFINITY;
map[14][19]=INFINITY;
map[14][20]=INFINITY;
map[14][21]=INFINITY;

map[15][0]=Map_Zero[15];
map[15][1]=INFINITY;
map[15][2]=INFINITY;
map[15][3]=INFINITY;
map[15][4]=INFINITY;
map[15][5]=INFINITY;
map[15][6]=INFINITY;
map[15][7]=INFINITY;
map[15][8]=INFINITY;
map[15][9]=INFINITY;
map[15][10]=INFINITY;
map[15][11]=INFINITY;
map[15][12]=INFINITY;
map[15][13]=INFINITY;
map[15][14]=15;
map[15][15]=0;
map[15][16]=15;
map[15][17]=13;
map[15][18]=INFINITY;
map[15][19]=INFINITY;
map[15][20]=INFINITY;
map[15][21]=INFINITY;

map[16][0]=Map_Zero[16];
map[16][1]=INFINITY;
map[16][2]=INFINITY;
map[16][3]=INFINITY;
map[16][4]=INFINITY;
map[16][5]=INFINITY;
map[16][6]=INFINITY;
map[16][7]=INFINITY;
map[16][8]=INFINITY;
map[16][9]=INFINITY;
map[16][10]=INFINITY;
map[16][11]=INFINITY;
map[16][12]=INFINITY;
map[16][13]=INFINITY;
map[16][14]=INFINITY;
map[16][15]=15;
map[16][16]=0;
map[16][17]=INFINITY;
map[16][18]=INFINITY;
map[16][19]=INFINITY;
map[16][20]=INFINITY;
map[16][21]=INFINITY;

map[17][0]=Map_Zero[17];
map[17][1]=INFINITY;
map[17][2]=INFINITY;
map[17][3]=INFINITY;
map[17][4]=INFINITY;
map[17][5]=INFINITY;
map[17][6]=INFINITY;
map[17][7]=INFINITY;
map[17][8]=INFINITY;
map[17][9]=INFINITY;
```

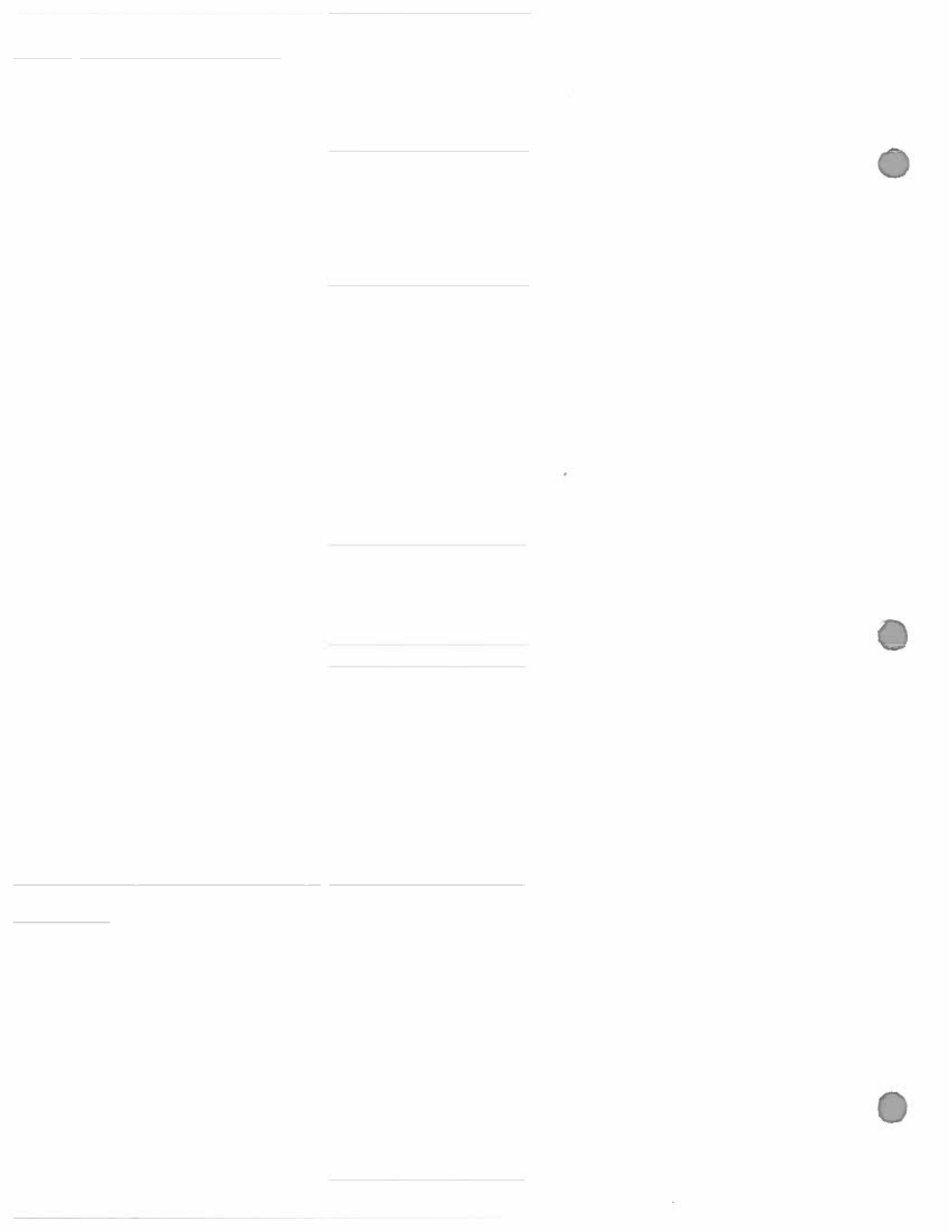



```
map[17][10]=INFINITY;
map[17][11]=INFINITY;
map[17][12]=INFINITY;
map[17][13]=INFINITY;
map[17][14]=INFINITY;
map[17][15]=13;
map[17][16]=INFINITY;
map[17][17]=0;
map[17][18]=10;
map[17][19]=10;
map[17][20]=INFINITY;
map[17][21]=INFINITY;
```

```
map[18][0]=Map_Zero[18];
map[18][1]=INFINITY;
map[18][2]=INFINITY;
map[18][3]=INFINITY;
map[18][4]=INFINITY;
map[18][5]=INFINITY;
map[18][6]=INFINITY;
map[18][7]=INFINITY;
map[18][8]=INFINITY;
map[18][9]=INFINITY;
map[18][10]=INFINITY;
map[18][11]=INFINITY;
map[18][12]=INFINITY;
map[18][13]=INFINITY;
map[18][14]=INFINITY;
map[18][15]=INFINITY;
map[18][16]=INFINITY;
map[18][17]=10;
map[18][18]=0;
map[18][19]=INFINITY;
map[18][20]=INFINITY;
map[18][21]=INFINITY;
```

```
map[19][0]=Map_Zero[19];
map[19][1]=INFINITY;
map[19][2]=INFINITY;
map[19][3]=INFINITY;
map[19][4]=INFINITY;
map[19][5]=INFINITY;
map[19][6]=INFINITY;
map[19][7]=INFINITY;
map[19][8]=INFINITY;
map[19][9]=28;
map[19][10]=INFINITY;
map[19][11]=INFINITY;
map[19][12]=INFINITY;
map[19][13]=INFINITY;
map[19][14]=INFINITY;
map[19][15]=INFINITY;
map[19][16]=INFINITY;
map[19][17]=10;
map[19][18]=INFINITY;
map[19][19]=0;
map[19][20]=10;
map[19][21]=INFINITY;
```

```
map[20][0]=Map_Zero[20];
map[20][1]=INFINITY;
map[20][2]=INFINITY;
map[20][3]=INFINITY;
map[20][4]=INFINITY;
map[20][5]=INFINITY;
map[20][6]=INFINITY;
```



```

map[20][7]=INFINITY;
map[20][8]=INFINITY;
map[20][9]=INFINITY;
map[20][10]=INFINITY;
map[20][11]=INFINITY;
map[20][12]=INFINITY;
map[20][13]=INFINITY;
map[20][14]=INFINITY;
map[20][15]=INFINITY;
map[20][16]=INFINITY;
map[20][17]=INFINITY;
map[20][18]=INFINITY;
map[20][19]=10;
map[20][20]=0;
map[20][21]=INFINITY;

```

```

map[21][0]=Map_Zero[21];
map[21][1]=INFINITY;
map[21][2]=INFINITY;
map[21][3]=INFINITY;
map[21][4]=INFINITY;
map[21][5]=INFINITY;
map[21][6]=INFINITY;
map[21][7]=INFINITY;
map[21][8]=INFINITY;
map[21][9]=53;
map[21][10]=INFINITY;
map[21][11]=INFINITY;
map[21][12]=INFINITY;
map[21][13]=INFINITY;
map[21][14]=INFINITY;
map[21][15]=INFINITY;
map[21][16]=INFINITY;
map[21][17]=INFINITY;
map[21][18]=INFINITY;
map[21][19]=INFINITY;
map[21][20]=INFINITY;
map[21][21]=0;

```

```

}

```

```

int smallestDist(){ //function to return the smallest vertex's index that is not yet
    found (.known != 1)
    int i;
    int indexSmallest = -1;
    int smallest = INFINITY;
    for(i=0;i<SIZE;i++){
        if(vertexArray[i].known != 1){
            if(vertexArray[i].dist<smallest){
                smallest = vertexArray[i].dist;
                indexSmallest = i;
            }
        }
    }
    return indexSmallest; // returns -1 if no more non-true indexes
}

```

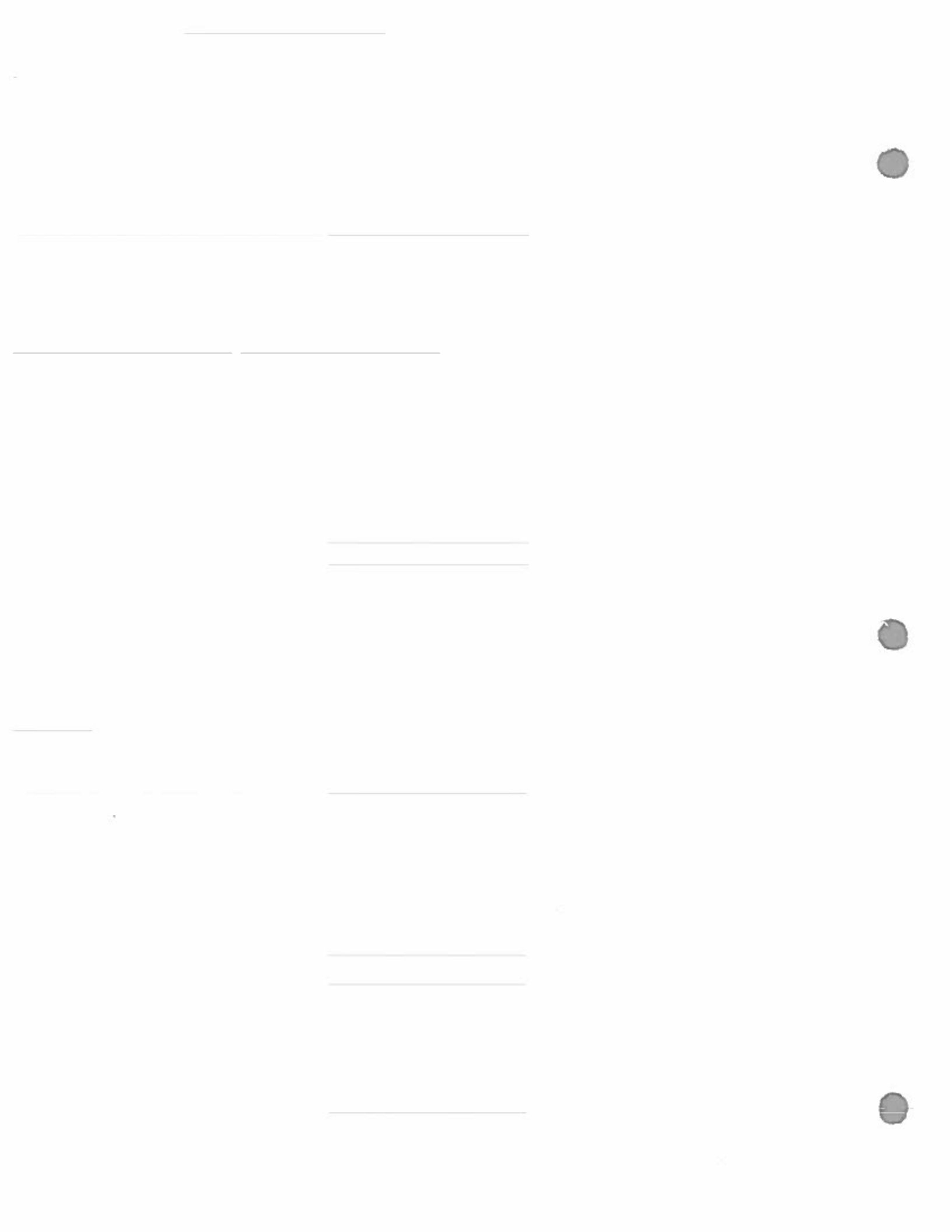
```

void printMap(){ // a simple program to the map and the values to the screen, used to
    debug
    int n,p,top,side;
    top=side=0;

    printf("\n");

    printf("          "); // prints out the top line
    for(n=0;n<SIZE;n++)

```



```

printf(" [%d]", top++);

for(n=0;n<SIZE;n++){
    printf(" \n "); // changes line and prints the side numbers
    if(side<10)
        printf(" [%d] ", side++);
    else
        printf(" [%d] ", side++);
    for(p=0;p<SIZE;p++){
        if(map[n][p] == INFINITY)
            printf(" - ");
        else
            printf(" %d ", map[n][p]); // print out of array
    }
}
printf("\n");
}

int dijkstra(int first){
    // debug lines will not be in the final nor will the print out of the map
    //printf("Starting dijkstra\n"); // debug line for Dr Lin
    int i,cvw,index;
    int tag = 0;
    initVertexArray();
    vertexArray[first].dist = 0;
    vertexArray[first].prev = -1;
    while(tag!= 1){
        if(smallestDist()!=-1){
            index = smallestDist();
            //printf("The index is %d \n", index); // debug line for Dr Lin
        }
        else{
            //printf("Dijkstra has exited with index -1 \n"); // debug line for Dr Lin
            tag = 1;
            break;
        }

        vertexArray[index].known = 1;

        //printf("The current smallest known index is %d \n", index); //debug line for
Dr Lin
        for(i=0;i<SIZE;i++){
            if(map[index][i]!=INFINITY){
                cvw = map[index][i];
                if(cvw + vertexArray[index].dist < vertexArray[i].dist)
                {
                    //printf("it was found that %d was smaller than %d \n", index, i); //
debug line for Dr Lin
                    vertexArray[i].dist = vertexArray[index].dist + cvw;
                    vertexArray[i].prev = index;
                }
            }
        }
    }
}

void printPath( struct vertex v){ // prints out the path by recursively following the ends
previous
if(v.prev != -1){
    printPath(vertexArray[v.prev]);
    //printf(" to ");
}
//printf("%d", v.current);
result_plotted_points[result_counter++]=v.current;
}

```



```
void printResult_Plotting_Points(){ // for debug only, prints out the saved array
    //int i;
    //printf("\n Print array of saved values \n"); // debug line
    //for(i=0;i<result_counter;i++)
        //printf("Shortest path %d = %d\n", i, result_plotted_points[i]);
}

#endif
```



1






```

/*
    Dru Sepulveda
    September, 9 2006
    Picture Display Program
    Adapted from Lesson 4
*/
#include <pspkernel.h>
#include <pspdebug.h>
#include <pspdisplay.h>
#include <pspctrl.h>
#include <pspgu.h>
#include <psppower.h>
#include <png.h>
#include <stdio.h>
#include "graphics.h"    // prewritten graphics file
#include "Index.h"       // my organizing file

PSP_MODULE_INFO("Dijkstra GPS Map ", 0, 1, 1);

#define printf pspDebugScreenPrintf    // printf defined to make debug printing ✓
#define MAX(X, Y) ((X) > (Y) ? (X) : (Y)) // function defined for non-existing ✓
operations

/* Exit callback */
int exit_callback(int arg1, int arg2, void *common) {
    sceKernelExitGame();
    return 0;
}

/* Callback thread */
int CallbackThread(SceSize args, void *argp) {
    int cbid;

    cbid = sceKernelCreateCallback("Exit Callback", exit_callback, NULL);
    sceKernelRegisterExitCallback(cbid);

    sceKernelSleepThreadCB();

    return 0;
}

/* Sets up the callback thread and returns its thread id */
int SetupCallbacks(void) {
    int thid = 0;

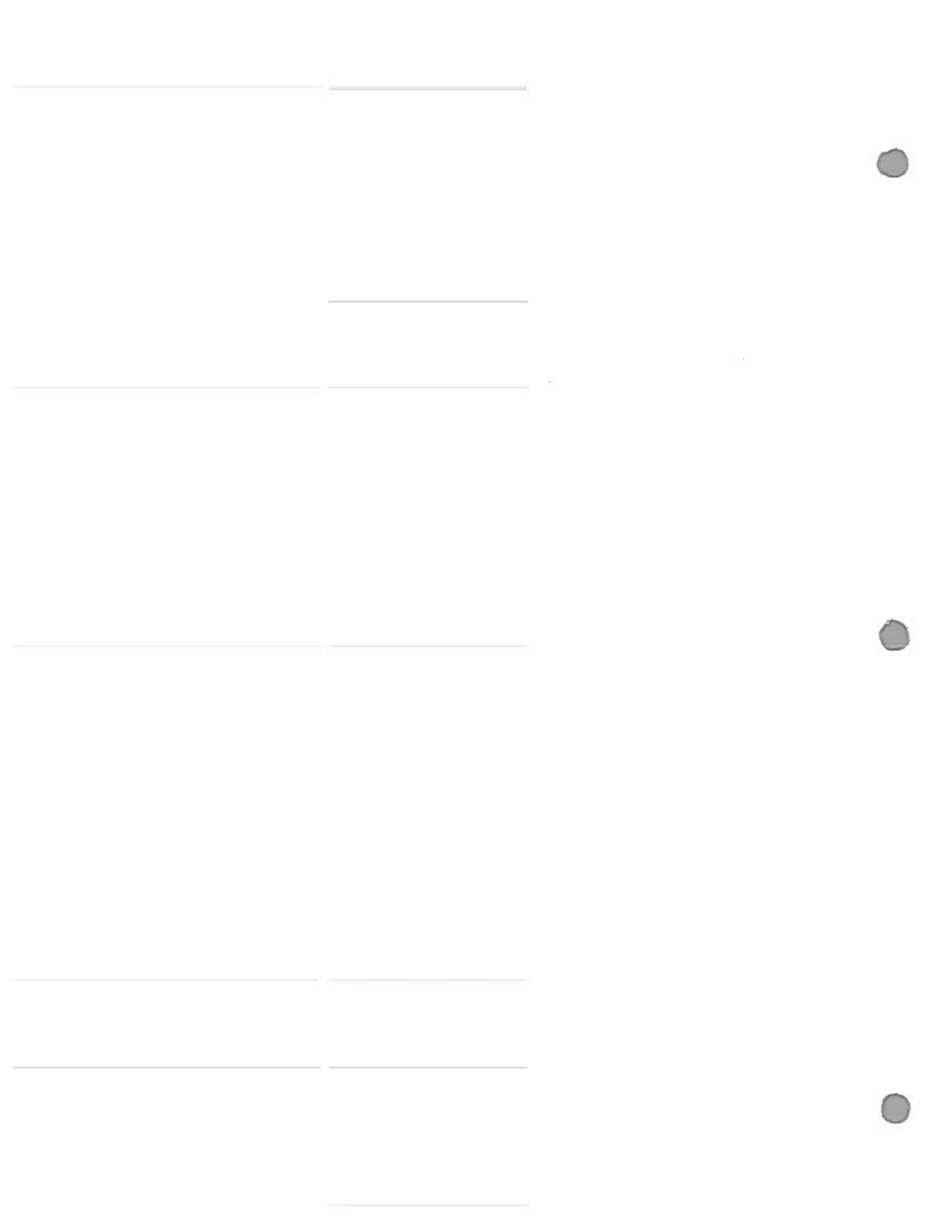
    thid = sceKernelCreateThread("update_thread", CallbackThread, 0x11, 0xFA0, 0, ✓
0);
    if(thid >= 0) {
        sceKernelStartThread(thid, 0, 0);
    }

    return thid;
}

int main() {
    char buffer[200];    //character buffer to store the image "ourimage" which is ✓
the map
    Image* ourImage;    //image type evoked by the graphics.h file

    char buffer2[200];  //character buffer to store the image "splash" which is ✓
the splash screen
    Image* splash;      //image type evoked by the graphics.h file

```



```

    int i = 0;                //counter for the splash screen timer

    int U_Start = 0;          //main.c Starting position for the user which is set to 0
    = "YOU" in mapPoints
    int U_Finish = 21;        //main.c Ending position for the user which is set to 21
    = "Rusle West" in mapPoints

    pspDebugScreenInit();     //initialize PSP screen to blank setting default
    paramater
    SetupCallbacks();         //sets up the thread to run the program on
    initGraphics();           //initilizes the graphic.h file

    SceCtrlData pad;           //sets the variable pad to type
    SceStrlData to capture buttons into
    sceCtrlSetSamplingCycle(0); //sets the sampling cycle for the
    joy stick
    sceCtrlSetSamplingMode(PSP_CTRL_MODE_ANALOG); //is necessary to capture the joy
    stick

    sprintf(buffer, "ourImage.png"); //loads the image file into the
    char buffer
    ourImage = loadImage(buffer);    //sets the image file stored in the
    buffer to the image variable

    sprintf(buffer2, "splash.png"); //loads the image file into the
    char buffer
    splash = loadImage(buffer2);     //sets the image file stored in the
    buffer to the image variable

    changePositionManual(U_Start, U_Finish); //Feed the interface.h file with the
    positions 0 and 21
    HeadInit();                       //HeadInit from the index.h file

    if (!ourImage || !splash) {       // checks to see if the pictures have
    loaded.
        //Image load failed
        printf("Image load failed!\n"); // error message if not
    }
    else {                             // if pictures have loaded
        for(i=0;i<1000;i++){           //timer to show
        splash screen
            blitAlphaImageToScreen(0 ,0 ,480 , 272, splash, 0, 0); //puts the
        picture on the PSP's screen at position x=0,y=0
            flipScreen();              //flips to
        the other memory, is necessary to update screen
        }
        //////////////////////////////////10-31-06@2:22PM-START//////////////////////////////////
        //////////////////////////////////
        while(1) {
            passXY(pad.Lx, pad.Ly);    // captures the joy sticks position
            and passes it to pass.h

            HeadInit();                //invoked
            from Index.h
            blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0); //draws map
            to the screen
            drawPathGPSMAP(result_plotted_points,result_counter);      //draws lines
            on map
            updateMapPicture();      //writes text
            at bottom of screen
            flipScreen();            //refreshes
            the screen to other pice of memory

            sceCtrlReadBufferPositive(&pad, 1); // sets pad
            as the buffer for button presses

```

This changed allow leaving the mouse.


```

        if(pad.Buttons & PSP_CTRL_UP) {                                // if user  ✓
press up
            U_Finish=U_Finish+1;                                        // Update the ✓
            destination index by adding one
            if(U_Finish > 21){                                          // make sure ✓
it is still in the acceptable range
                U_Finish=1;}                                           // start over ✓
            if not in acceptable range
                changePositionManual(U_Start, U_Finish); // update end ✓
            index in the interface.h file
                HeadInit();                                           // ✓
reinitialize all values for a new draw
                blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0) ✓
;            //draw the base map
                drawPathGPSMAP(result_plotted_points,result_counter); ✓
            //draw the lines
                updateMapPicture();                                    ✓
            //update and write text
                flipScreen();                                          ✓
            //refresh the screen
                for(i=0; i<10; i++) {                                  ✓
            // buisy wait to prevent over pressing
                sceDisplayWaitVblankStart();                          ✓
            // blanks screen behind the map
                }                                                       ✓

        }
        else if(pad.Buttons & PSP_CTRL_DOWN) {                        // if user  ✓
presses down
            U_Finish=U_Finish-1;                                        // Update the ✓
            destination index by subtracting one
            if(U_Finish<1){                                            // make sure ✓
it is still in the acceptable range
                U_Finish=21;}                                           // start over ✓
            if not in acceptable range
                changePositionManual(U_Start, U_Finish); // update end ✓
            index in the interface.h file
                HeadInit();                                           // ✓
reinitialize all values for a new draw
                blitAlphaImageToScreen(0 ,0 ,480 , 272, ourImage, 0, 0); ✓
            //draw the base map
                drawPathGPSMAP(result_plotted_points,result_counter); ✓
            //draw the lines
                updateMapPicture();                                    ✓
            //update and write text
                flipScreen();                                          ✓
            //refresh the screen
                for(i=0; i<10; i++) {                                  ✓
            // buisy wait to prevent over pressing
                sceDisplayWaitVblankStart();                          ✓
            // blanks screen behind the map
                }
            }
        } // while
    } //else
    //////////////////////////////////////10-31-06@2:22PM-END//////////////////////////////////// ✓
    //////////////////////////////////////
    sceKernelSleepThread();      //makes the thread go to sleep (kill) so the program can ✓
shut down
    return 0;
}

```





```

// Dru Sepulveda
// Dynamic GPS Location
// Sep, 28 2006

#ifndef GPS_CURRENT_LOCATION
#define GPS_CURRENT_LOCATION

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "mapPoints.h"
#include "pass.h" // used top pass x and y values

struct YOUR_LOCATION{ // structure to catch and structure the lat and long to readable ✓
    format for the psp
    float x;
    float y;
};

struct YOUR_LOCATION YOU; // a variable of the the type struct

int Map_Zero[GPS_MAP_SIZE]; // used to hold all the distances possible with

/////////////////////////prototypes/////////////////////////
void initMapZero();           // initiate Map_Zero array
void GPS_LOCATION_YOU();      // gather your current location
void DISTANCE_TO_ALL_NODES(); // find the distance to all nodes and return smallest by ✓
    Map_Zero array
/////////////////////////

void initMapZero(){ // initializes all values in the Map_Zero array to infinity
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        Map_Zero[i]=150000;
    }
}

void GPS_LOCATION_YOU(){
    //This information will be taken from the GPS device
    YOU.x = passX;
    YOU.y = passY;
}

void DISTANCE_TO_ALL_NODES(){
    // find the distance between each point on the map and the user, then save it to the ✓
    array.
    int i;
    for(i=1;i<GPS_MAP_SIZE;i++){ ///altered this! :D needs to go from 1 to 0 to find ✓
    least.
    Map_Zero[i] = floor(sqrt(pow((gpsMap[i].x-YOU.x),2)+pow((gpsMap[i].y-YOU.y),2)));
    // set distances for Dijkstra
    //printf("Distance %d --> %d\n", i, Map_Zero[i]); // for debug
    }

    int indexGPSSmallest = 0; // variable for the smallest index in ✓
    Map_Zero
    int GPSSmallest = 150000; // initiates the smallest variable to ✓
    "infinity"
    for(i=0;i<GPS_MAP_SIZE;i++){ // for all indexes (including itself but ✓
    is fixed above)
        if(Map_Zero[i]<GPSSmallest){ // find the smallest indexed distance
            GPSSmallest = Map_Zero[i];
            indexGPSSmallest = i;
        }
    }
}

```



```
    for(i=0;i<GPS_MAP_SIZE;i++){
        infinity but the smallest node
        if(i != indexGPSSmallest){
            pick the closest node and not get
            Map_Zero[i] = 150000;
        }
    }
}

#endif
```

// make all entries to Map_Zero = ✓

// this makes it easy for the program to ✓

// confused by the other ones.


```

////////////////////////////////////
// Dru Sepulveda
// mapPoints v 1.0
// 9-26-06
//
// This header file contains a set of functions to associate the
// generated shortest path with the actual points on the map.
//
////////////////////////////////////

#ifndef __MAPPOINTS__
#define __MAPPOINTS__

#include "graphics.h" //prewritten header from a lines program
#include "pass.h" // used to pass x and y value to map

#define GPS_MAP_SIZE 22 // actual number of nodes on the map (real nodes + ghost nodes)
#define OFFSET_X 0 // how far off along the x axis should we offset
#define OFFSET_Y 0 // how far off along the y axis should we offset
#define SCALE 1 // change the whole scale, a scale of 1 = 1:1 scale

////////////////////////////////////variables////////////////////////////////////
struct gpsMapVertex{
    int x; // actual x value of the pixel on the screen
    int y; // actual y value of the pixel on the screen
    int ghost; // This gives me the ability to have nodes at turns with out having them
               show up in the selectiable array
    char turn; // This will let me give the direction of the turn at notes as is
               realitive to the direction vector of the user.
    char *location; // This will give the name of the location.
};

struct gpsMapVertex gpsMap[GPS_MAP_SIZE];
//Color white = RGB(0, 0, 0); //Added 10-31-2006 @ 2:21PM
////////////////////////////////////

////////////////////////////////////PROTOTYPES////////////////////////////////////
void initGPSMap(); //initalizes the map to default
void defaultGPSMAP(); //loads the default map or map from the users input file
void printAllGPSMAP(); //prints the array out in plane text for debug purposes
void drawAllGPSMAP(); //draws the lines based on the points given
void drawPathGPSMAP(int *result_plotted_points, int result_counter); // prints out path
                           specified by dijkstra
////////////////////////////////////

void initGPSMap(){ //simple program
    //printf("Initalizing GPSMAP\n"); //debug line
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        gpsMap[i].x = -1; // default value
        gpsMap[i].y = -1; // default value
        gpsMap[i].ghost = 0; // not a ghost
        gpsMap[i].turn = 'o'; // default
        gpsMap[i].location = NULL; // location unknown and defaulted
    }
}

void defaultGPSMAP(){ // sets map from a file or from default map <assumes size 6>
    // get information from a file?

    //---> open and read file dialogue comming soon

```



```
// default test informaiton
printf("Loading Default Map\n"); //debug line
gpsMap[0].x = (floor(passX) + OFFSET_X)*SCALE;
gpsMap[0].y = (floor(passY) + OFFSET_Y)*SCALE;
gpsMap[0].ghost = 0;
gpsMap[0].turn = 'o';
gpsMap[0].location = "You";

gpsMap[1].x = (125 + OFFSET_X)*SCALE;
gpsMap[1].y = (48 + OFFSET_Y)*SCALE;
gpsMap[1].ghost = 0;
gpsMap[1].turn = 'o';
gpsMap[1].location = "Quads";

gpsMap[2].x = (125 + OFFSET_X)*SCALE;
gpsMap[2].y = (72 + OFFSET_Y)*SCALE;
gpsMap[2].ghost = 1;
gpsMap[2].turn = 'o';
gpsMap[2].location = "Infront of Quads";

gpsMap[3].x = (165 + OFFSET_X)*SCALE;
gpsMap[3].y = (72 + OFFSET_Y)*SCALE;
gpsMap[3].ghost = 1;
gpsMap[3].turn = 'o';
gpsMap[3].location = "Lightsy Intersection";

gpsMap[4].x = (209 + OFFSET_X)*SCALE;
gpsMap[4].y = (72 + OFFSET_Y)*SCALE;
gpsMap[4].ghost = 1;
gpsMap[4].turn = 'o';
gpsMap[4].location = "Infront of Whittington Hall";

gpsMap[5].x = (209 + OFFSET_X)*SCALE;
gpsMap[5].y = (63 + OFFSET_Y)*SCALE;
gpsMap[5].ghost = 0;
gpsMap[5].turn = 'o';
gpsMap[5].location = "Whittington Hall";

gpsMap[6].x = (277 + OFFSET_X)*SCALE;
gpsMap[6].y = (72 + OFFSET_Y)*SCALE;
gpsMap[6].ghost = 1;
gpsMap[6].turn = 'o';
gpsMap[6].location = "Infront of Brewer Center";

gpsMap[7].x = (277 + OFFSET_X)*SCALE;
gpsMap[7].y = (63 + OFFSET_Y)*SCALE;
gpsMap[7].ghost = 0;
gpsMap[7].turn = 'o';
gpsMap[7].location = "Brewer Center";

gpsMap[8].x = (165 + OFFSET_X)*SCALE;
gpsMap[8].y = (107 + OFFSET_Y)*SCALE;
gpsMap[8].ghost = 1;
gpsMap[8].turn = 'o';
gpsMap[8].location = "Pond Circle";

gpsMap[9].x = (177 + OFFSET_X)*SCALE;
gpsMap[9].y = (107 + OFFSET_Y)*SCALE;
gpsMap[9].ghost = 1;
gpsMap[9].turn = 'o';
gpsMap[9].location = "Pond Circle";

gpsMap[10].x = (177 + OFFSET_X)*SCALE;
gpsMap[10].y = (107 + OFFSET_Y)*SCALE;
gpsMap[10].ghost = 0;
```







```
gpsMap[10].turn = 'o';
gpsMap[10].location = "Strom Thurmond Center";

gpsMap[11].x = (158 + OFFSET_X)*SCALE;
gpsMap[11].y = (107 + OFFSET_Y)*SCALE;
gpsMap[11].ghost = 1;
gpsMap[11].turn = 'o';
gpsMap[11].location = "Infront of Lightsy Chapel";

gpsMap[12].x = (158 + OFFSET_X)*SCALE;
gpsMap[12].y = (97 + OFFSET_Y)*SCALE;
gpsMap[12].ghost = 0;
gpsMap[12].turn = 'o';
gpsMap[12].location = "Lightsy Chappel";

gpsMap[13].x = (136 + OFFSET_X)*SCALE;
gpsMap[13].y = (100 + OFFSET_Y)*SCALE;
gpsMap[13].ghost = 0;
gpsMap[13].turn = 'o';
gpsMap[13].location = "Rivers Library";

gpsMap[14].x = (136 + OFFSET_X)*SCALE;
gpsMap[14].y = (107 + OFFSET_Y)*SCALE;
gpsMap[14].ghost = 1;
gpsMap[14].turn = 'o';
gpsMap[14].location = "Infront of Rivers Library";

gpsMap[15].x = (136 + OFFSET_X)*SCALE;
gpsMap[15].y = (122 + OFFSET_Y)*SCALE;
gpsMap[15].ghost = 1;
gpsMap[15].turn = 'o';
gpsMap[15].location = "Infront of Science Building";

gpsMap[16].x = (121 + OFFSET_X)*SCALE;
gpsMap[16].y = (122 + OFFSET_Y)*SCALE;
gpsMap[16].ghost = 0;
gpsMap[16].turn = 'o';
gpsMap[16].location = "Science Building";

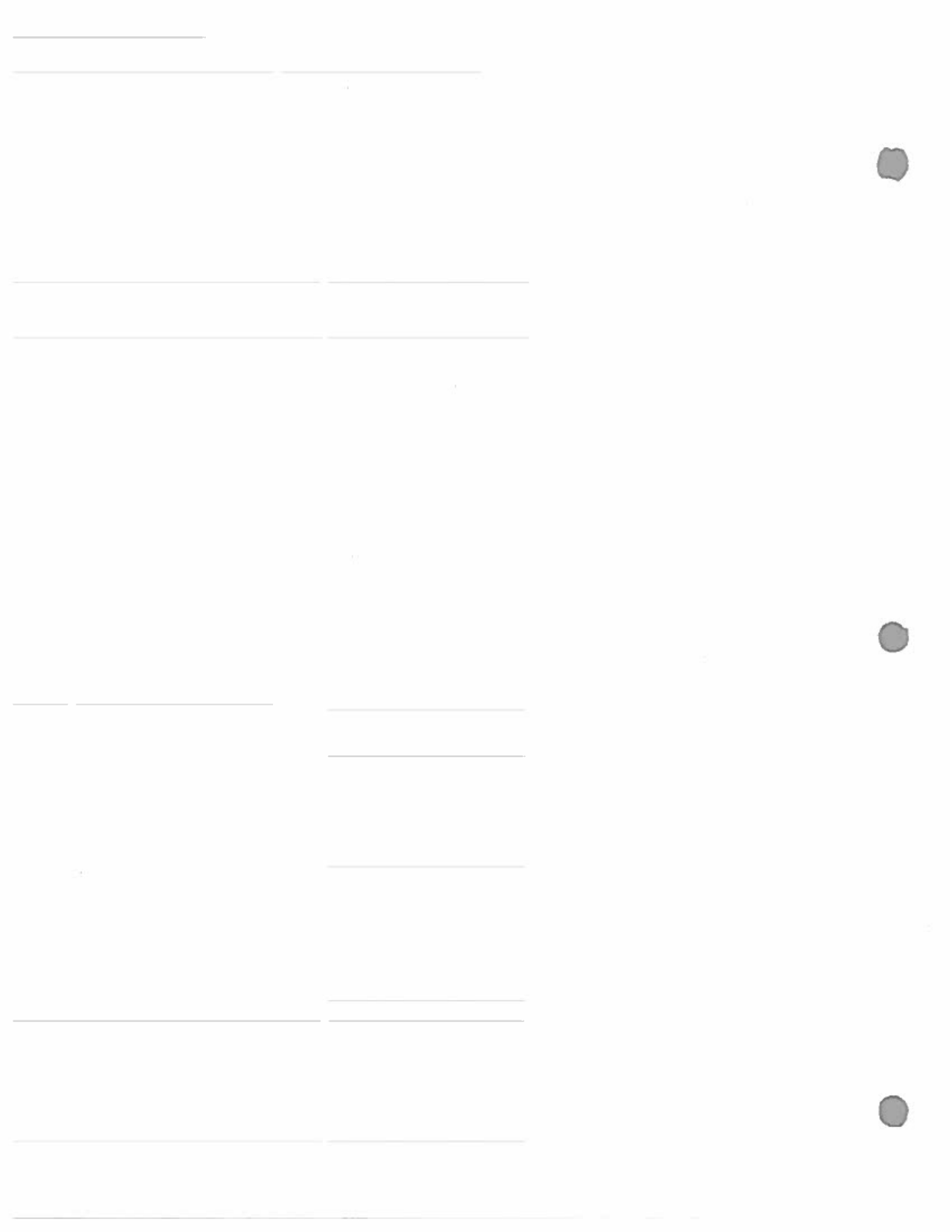
gpsMap[17].x = (136 + OFFSET_X)*SCALE;
gpsMap[17].y = (135 + OFFSET_Y)*SCALE;
gpsMap[17].ghost = 1;
gpsMap[17].turn = 'o';
gpsMap[17].location = "Infront of Wingo/Norris";

gpsMap[18].x = (136 + OFFSET_X)*SCALE;
gpsMap[18].y = (145 + OFFSET_Y)*SCALE;
gpsMap[18].ghost = 0;
gpsMap[18].turn = 'o';
gpsMap[18].location = "Wingo/Norris";

gpsMap[19].x = (177 + OFFSET_X)*SCALE;
gpsMap[19].y = (135 + OFFSET_Y)*SCALE;
gpsMap[19].ghost = 1;
gpsMap[19].turn = 'o';
gpsMap[19].location = "Infront of Jones/Ashby";

gpsMap[20].x = (177 + OFFSET_X)*SCALE;
gpsMap[20].y = (145 + OFFSET_Y)*SCALE;
gpsMap[20].ghost = 0;
gpsMap[20].turn = 'o';
gpsMap[20].location = "Jones/Ashby";

gpsMap[21].x = (230 + OFFSET_X)*SCALE;
gpsMap[21].y = (107 + OFFSET_Y)*SCALE;
gpsMap[21].ghost = 0;
```



```
    gpsMap[21].turn = 'o';
    gpsMap[21].location = "Russel West";
}

void printAllGPSMAP(){ // prints out the entire map for debug
    // normally it would impliment the graphics.h from the lines and circles program
    // but while testing it will cimply print out the information.
    printf("Printing out GPSMAP\n"); //debug line
    int i;
    for(i=0;i<GPS_MAP_SIZE;i++){
        printf("Index %d has x = %d and y = %d, ghost = %d, turn = %c, name = %s \n", i ,
            gpsMap[i].x, gpsMap[i].y, gpsMap[i].ghost, gpsMap[i].turn, gpsMap[i].location);
    }
}

void drawAllGPSMAP(){ // draws the entire map
    //printf("Starting drawALLgpsMAP Draw\n");
    int i;
    for(i=0;i<GPS_MAP_SIZE-1;i++){
        drawLineScreen(gpsMap[i].x,gpsMap[i].y,gpsMap[i+1].x,gpsMap[i+1].y,R(100)); //color
        is defined 10-31-06 @ 2:21PM
        //printf("(%d,%d) to (%d,%d) with color %d %d %d\n", gpsMap[i].x,gpsMap[i].y,gpsMap[i+1].x,gpsMap[i+1].y,255,0,0); //debug line
    }
}

void drawPathGPSMAP(int *result_plotted_points, int result_counter){ // draws the path
    selected by Dijkstra
    //printf("Starting drawPathGPSMAP\n");
    int i;
    for(i=0;i<result_counter-1;i++){
        drawLineScreen(gpsMap[result_plotted_points[i]].x,gpsMap[result_plotted_points[i]].y,
            gpsMap[result_plotted_points[i+1]].x,gpsMap[result_plotted_points[i+1]].y,R(100)); //
        color can be 0 - 255
        //printf("(%d,%d) to (%d,%d) with color %d %d %d\n", gpsMap[result_plotted_points[i]].
        .x,gpsMap[result_plotted_points[i]].y,gpsMap[result_plotted_points[i+1]].x,gpsMap
        [result_plotted_points[i+1]].y,255,0,0); //debug line
    }
}

#endif
```



18



19



```

// Dru Sepulveda
// Interface for pspGPS_Program
// Oct, 19 2006
// 1:00PM <right after the fire alarm>
//

#ifndef __INTERFACE__
#define __INTERFACE__

#include <stdio.h>
#include <stdlib.h>
// #include "graphics.h"
#include "mapPoints.h"
#include "Dijkstra.h"

////////////////////variables////////////////////////////////////
//
int destinationIndex;           // the index of the user defined destination
float distanceLeft;             // the distance remaining untill the end of the trip
float distanceTotal;            // the total distance (constant for a chosen destination)
float lat;                      // the latitude value at your current location
float lng;                      // the long value at your current location
char *currentLocationName;      // the name of your current location
char *destinationName;          // the name of the user defined destination

int User_Defined_Start_Current; //holds the value passes from main
int User_Defined_Destination;    //holds the value passes from main
////////////////////////////////////
//

////////////////////prototypes////////////////////////////////////
//
void changePositionManual(int start, int end); // 10-31-06 @ 3:00PM
void initInterface();           // initiates all of the default settings for the interface
void firstRun();                // sets the values for the first iteration of the algorithm
void updateMapStatus();         // updates all of the data that will be used to draw on
    the map
void updateMapPicture();        // updates the actual picture on the map with the users input
//void debugPrint();            // print function used for debug only
////////////////////////////////////
//

void changePositionManual(int start, int end){
    User_Defined_Start_Current = start;           //starting index passed by main
    User_Defined_Destination = end;               //ending index passes by main
}

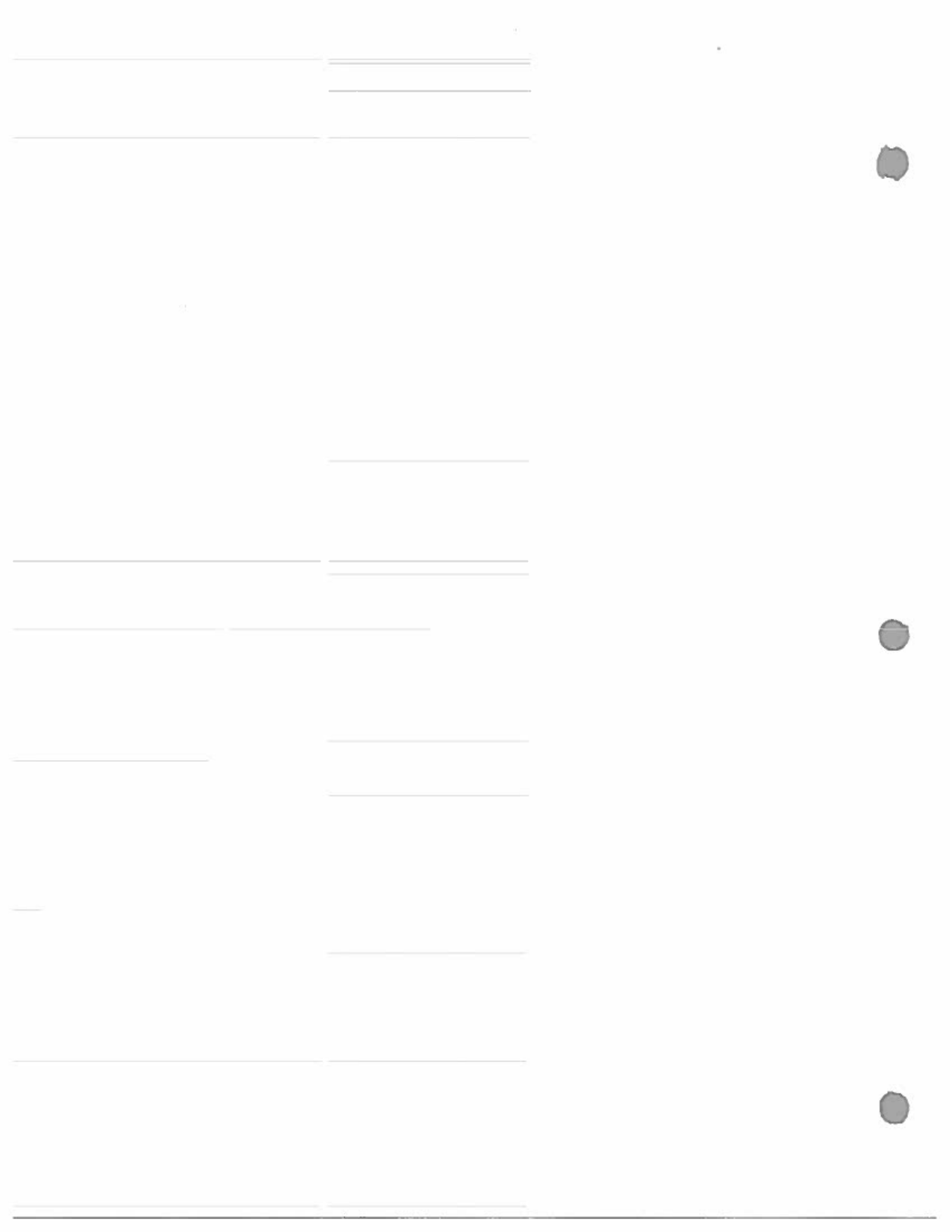
void initInterface(){ // initializing all values to default

    destinationIndex = 0;
    distanceLeft = 0;
    distanceTotal = 0;
    lat = 0;
    lng = 0;
    currentLocationName = NULL;
    destinationName = NULL;
}

void firstRun(){ // sets the needed values each time a new location is chosen

    distanceTotal = vertexArray[SIZE-1].dist; //The Last value from the first time
    Dijkstra runs
    destinationIndex = User_Defined_Destination; // The index of the location chosen
    by the user from the chooseable list in gpsMaps.h
    destinationName = gpsMap[User_Defined_Destination].location; //this value is pulled
    from the inbeded information in the gpsMaps.h file

```



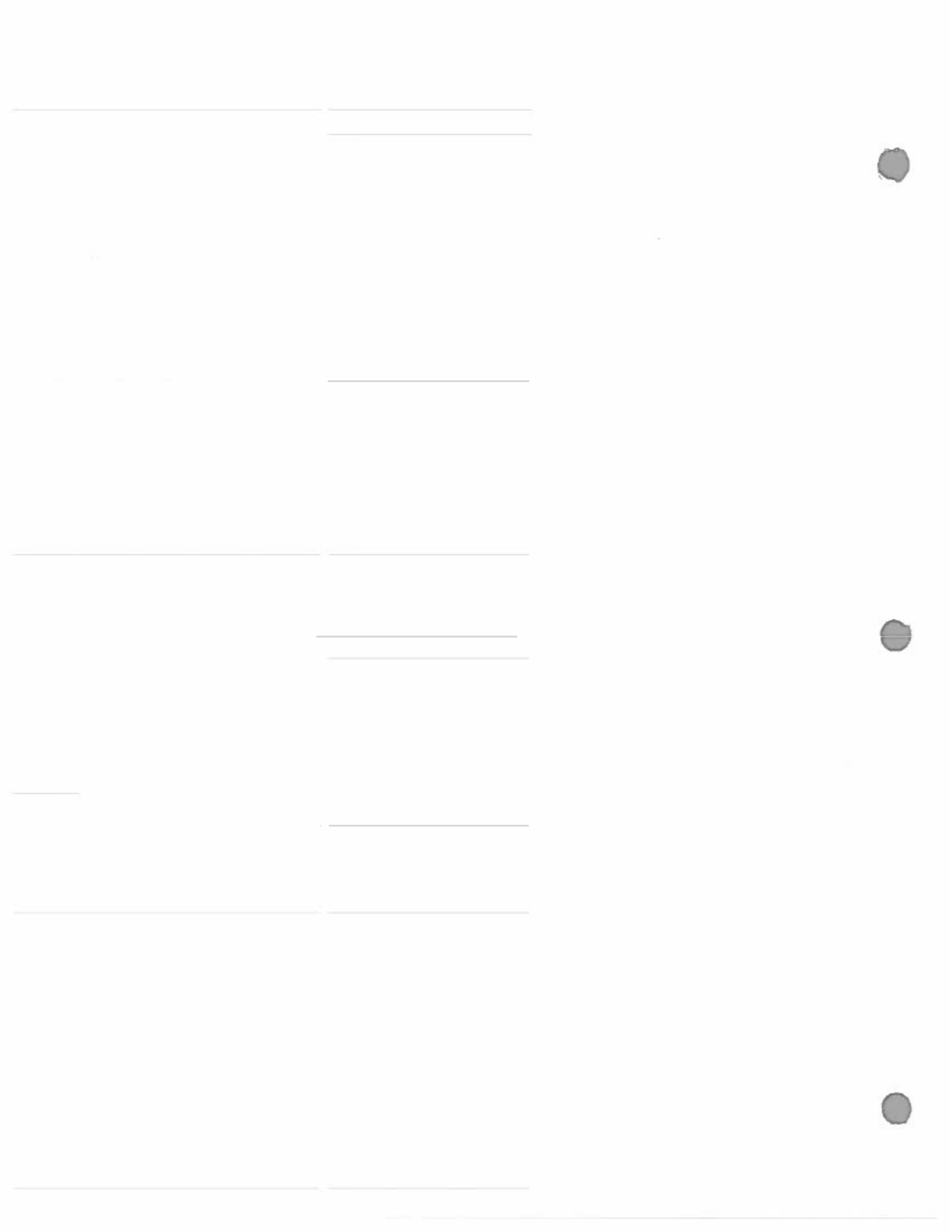
```
}

void updateMapStatus(){ // updates all data that will be used to draw on the map
    currentLocationName = gpsMap[User_Defined_Start_Current].location; // this name ✓
    is gotten by the Second jump in dijkstra after you
}

void updateMapPicture(){ // This function actually writes to the PSP screen the TEXT that ✓
    has been gathered.
    // void printTextScreen(int x, int y, const char* text, u32 color);
    printTextScreen(15,245,"C_Location: ",B(100));
    printTextScreen(105,245,currentLocationName,B(100));
    printTextScreen(15,260,"D_Location: ",B(100));
    printTextScreen(105,260,destinationName,B(100));
    //printTextScreen(400,254,"DIST: ",B(100));
    //printTextScreen(420,254,convertDistTotal,B(100));
}

//void debugPrint(){
//printf("Lat: %f\nLong: %f\ntotalDistance: %f\ndestinationIndex: %d\ndistanceLeft: %f\
    ncurrentLocN: %s\ndestLocN: %s", lat, lng, distanceTotal, destinationIndex, ✓
    distanceLeft, currentLocationName, destinationName); ✓
//}

#endif
```




```

// Dru Sepulvedca
// 10-29-06
// Index.h
// This .h contains a list of all include files, functions and variables and creates a
// easy init for all of them.

#include "Dijkstra.h" // The program that does the graph theory and finds the shortest
// path
//includes
// #include <stdio.h>
// #include <stdlib.h>

/* Functions
void initVertexArray(); // initiates vertex array to -1
void initMap(); // initiates the map to 0
int smallestDist(); // finds smallest distance (private to Dijkstra)
void printMap(); // DEBUG ONLY Prints out the map in text format
int dijkstra(int first); // function that finds the shortest overall path
// takes int first node as param
void printPath( struct vertex v); // saves path to result_plotted_points[] takes a
// struct array as param
void printResult_Plotting_Points(); // DEBUG ONLY Prints out the array in text format
*/

/* Variables
int map[SIZE][SIZE]; // structure of the map (a two dentional array (Row x and Colum y) =
// From X to Y) vector format

struct vertex{ // structure of each vertex
    int current; // current index of vertex
    int dist; // distance to adjacent + distance from start
    int prev; // previous vertex with shortest path
    int known; // is this vertex known?
};

struct vertex vertexArray[SIZE]; // an array of type vertex and of size SIZE

int result_plotted_points[SIZE]; // used to hold the printed order of found vertexes in
// found shortest path
int result_counter = 0; // used to to count in the the array for result ^^^^^^^
// ^^^^^^^^^^^^^

*/

#include "mapPoints.h" // The program that creates a map that is true to the picture and
// prints lines on it.
//includes
// #include "graphics.h" //prewritten header from a lines program

/* Functions
void initGPSMap(); //initializes the map to default
void defaultGPSMAP(); //loads the default map or map from the users input file
void printAllGPSMAP(); //DEBUG ONLY prints the array out in plane text for debug
// purposes
void drawAllGPSMAP(); //draws the lines based on the points given
void drawPathGPSMAP(int *result_plotted_points, int result_counter); // prints out path
// specified by dijkstra
*/

/* Variables
#define GPS_MAP_SIZE 22 // actual number of nodes on the map (real nodes + ghost nodes)
#define OFFSET_X 0 // how far off along the x axis should we offset
#define OFFSET_Y 0 // how far off along the y axis should we offset
#define SCALE 1 // chage the whole scale, a scale of 1 = 1:1 scale

struct gpsMapVertex{
    int x; // actual x value of the pixel on the screen

```



```

    int y; // actual y value of the pixel on the screen
    int ghost; // This gives me the ability to have nodes at turns with out having them
    show up in the selectiable array
    char turn; // This will let me give the direction of the turn at notes as is
    realitive to the direction vector of the user.
    char *location; // This will give the name of the location.
};

struct gpsMapVertex gpsMap[GPS_MAP_SIZE];
*/

#include "GPS_CURRENT_LOCATION.h" // sets the distance of you from every other node
//Includes
// #include <stdio.h>
// #include <stdlib.h>
// #include <math.h>
// #include "mapPoints.h"

/* Functions
void initMapZero(); // initiates the array that holds the distances
void GPS_LOCATION_YOU(); // get your current location from the GPS device
void DISTANCE_TO_ALL_NODES(); // fills the array with your distance from each node
    using gpsMap.h
*/

/* variables
struct YOUR_LOCATION{ // structure to catch and structure the lat and long to readable
    format for the psp
    float x;
    float y;
};

struct YOUR_LOCATION YOU; // a variable of the the type struct

int Map_Zero[GPS_MAP_SIZE]; // used to hold all the distances possible with

*/

#include "interface.h" // gathers information to update the screen with.
// includes
// #include <stdio.h>
// #include <stdlib.h>
// #include "graphics.h"
// #include "mapPoints.h"
// #include "Dijkstra.h"

/* Functions
void initInterface(); // initiates all of the default settings for the interface
void firstRun(); // sets the values for the first iliteration of the algorithm
void updateMapStatus(); // updates all of the data that will be used to draw on the
    map
void updateMapPicture(); // updates the actual picture on the map with the users input
void debugPrint(); // print function used for debug only
*/

/* Variables
int destinationIndex; // the index of the user defined destination
float distanceLeft; // the distance remaining untill the end of the trip
float distanceTotal; // the total distance (constant for a chosen destination)
float lat; // the latitude value at your current location
float lng; // the long value at yout current location
char *currentLocationName; // the name of your current location
char *destinationName; // the name of the user defined destination
*/

#include "pass.h"

```



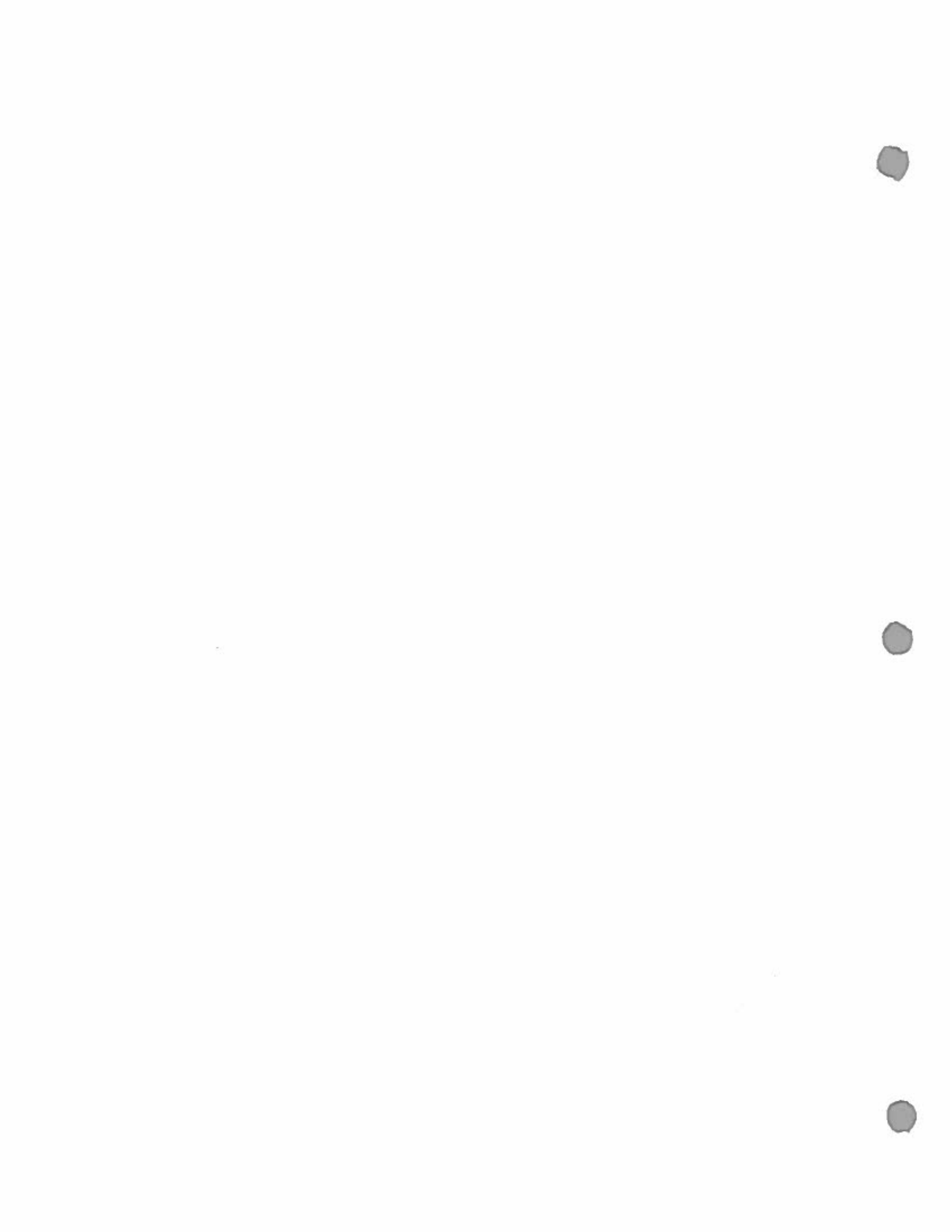
```
/* passes x and y to .hs that need them. */
```

```
//////////Variables//////////
//////////
```

```
//////////protypes//////////
void InitAll();          // init all for driver test
void HeadInit();         // init some for header
//////////
```

```
//void initAll(){
//initVertexArray();          //Dijkstra
//initMapZero();              //GPS_MAP_LOCATION
//initGPSMap();               //mapPoints
//defaultGPSMAP();            //mapPoints
//initInterface();            //interface
//GPS_LOCATION_YOU();         //GPS_MAP_LOCATION
//DISTANCE_TO_ALL_NODES();    //GPS_MAP_LOCATION
//initMap();                  //Dijkstra
//dijkstra(User_Defined_Start_Current); //Dijkstra
//printPath(vertexArray[User_Defined_Destination]); //Dijkstra
//printf("\n");
//firstRun();
//updateMapStatus();          //interface
//debugPrint();               //interface
//printf("\n");               //interface
//drawPathGPSMAP(result_plotted_points,result_counter); //mapPoints
//}
```

```
void HeadInit(){
initVertexArray();          //Dijkstra
initMapZero();              //GPS_MAP_LOCATION
initGPSMap();               //mapPoints
defaultGPSMAP();            //mapPoints
initInterface();            //interface
GPS_LOCATION_YOU();         //GPS_MAP_LOCATION
DISTANCE_TO_ALL_NODES();    //GPS_MAP_LOCATION
initMap();                  //Dijkstra
dijkstra(User_Defined_Start_Current); //Dijkstra
printPath(vertexArray[User_Defined_Destination]); //Dijkstra
firstRun();                 //interface
updateMapStatus();          //interface
}
```



```

#ifndef GRAPHICS_H
#define GRAPHICS_H

#include <psptypes.h>

#define PSP_LINE_SIZE 512
#define SCREEN_WIDTH 480
#define SCREEN_HEIGHT 272

typedef u32 Color;
#define A(color) ((u8)(color >> 24 & 0xFF))
#define B(color) ((u8)(color >> 16 & 0xFF))
#define G(color) ((u8)(color >> 8 & 0xFF))
#define R(color) ((u8)(color & 0xFF))

typedef struct
{
    int textureWidth; // the real width of data, 2^n with n>=0
    int textureHeight; // the real height of data, 2^n with n>=0
    int imageWidth; // the image width
    int imageHeight;
    Color* data;
} Image;

/**
 * Load a PNG image.
 *
 * @pre filename != NULL
 * @param filename - filename of the PNG image to load
 * @return pointer to a new allocated Image struct, or NULL on failure
 */
extern Image* loadImage(const char* filename);

/**
 * Blit a rectangle part of an image to another image.
 *
 * @pre source != NULL && destination != NULL &&
 *      sx >= 0 && sy >= 0 &&
 *      width > 0 && height > 0 &&
 *      sx + width <= source->width && sy + height <= source->height &&
 *      dx + width <= destination->width && dy + height <= destination->height
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 * @param destination - pointer to Image struct of the destination image
 */
extern void blitImageToImage(int sx, int sy, int width, int height, Image* source, int dx,
    int dy, Image* destination);

/**
 * Blit a rectangle part of an image to screen.
 *
 * @pre source != NULL && destination != NULL &&
 *      sx >= 0 && sy >= 0 &&
 *      width > 0 && height > 0 &&
 *      sx + width <= source->width && sy + height <= source->height &&
 *      dx + width <= SCREEN_WIDTH && dy + height <= SCREEN_HEIGHT
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image

```



```

* @param dx - left target position in destination image
* @param dy - top target position in destination image
*/
extern void blitImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int dy);

/**
 * Blit a rectangle part of an image to another image without alpha pixels in source image.
 *
 * @pre source != NULL && destination != NULL &&
 *       sx >= 0 && sy >= 0 &&
 *       width > 0 && height > 0 &&
 *       sx + width <= source->width && sy + height <= source->height &&
 *       dx + width <= destination->width && dy + height <= destination->height
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 * @param destination - pointer to Image struct of the destination image
 */
extern void blitAlphaImageToImage(int sx, int sy, int width, int height, Image* source, int dx, int dy, Image* destination);

/**
 * Blit a rectangle part of an image to screen without alpha pixels in source image.
 *
 * @pre source != NULL && destination != NULL &&
 *       sx >= 0 && sy >= 0 &&
 *       width > 0 && height > 0 &&
 *       sx + width <= source->width && sy + height <= source->height &&
 *       dx + width <= SCREEN_WIDTH && dy + height <= SCREEN_HEIGHT
 * @param sx - left position of rectangle in source image
 * @param sy - top position of rectangle in source image
 * @param width - width of rectangle in source image
 * @param height - height of rectangle in source image
 * @param source - pointer to Image struct of the source image
 * @param dx - left target position in destination image
 * @param dy - top target position in destination image
 */
extern void blitAlphaImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int dy);

/**
 * Create an empty image.
 *
 * @pre width > 0 && height > 0 && width <= 512 && height <= 512
 * @param width - width of the new image
 * @param height - height of the new image
 * @return pointer to a new allocated Image struct, all pixels initialized to color 0, or NULL on failure
 */
extern Image* createImage(int width, int height);

/**
 * Frees an allocated image.
 *
 * @pre image != null
 * @param image a pointer to an image struct
 */
extern void freeImage(Image* image);

```



```

* Initialize all pixels of an image with a color.

```

```

*

```

```

* @pre image != NULL

```

```

* @param color - new color for the pixels

```

```

* @param image - image to clear

```

```

*/

```

```

extern void clearImage(Color color, Image* image);

```

```

/**

```

```

* Initialize all pixels of the screen with a color.

```

```

*

```

```

* @param color - new color for the pixels

```

```

*/

```

```

extern void clearScreen(Color color);

```

```

/**

```

```

* Fill a rectangle of an image with a color.

```

```

*

```

```

* @pre image != NULL

```

```

* @param color - new color for the pixels

```

```

* @param x0 - left position of rectangle in image

```

```

* @param y0 - top position of rectangle in image

```

```

* @param width - width of rectangle in image

```

```

* @param height - height of rectangle in image

```

```

* @param image - image

```

```

*/

```

```

extern void fillImageRect(Color color, int x0, int y0, int width, int height, Image*
    image);

```

```

/**

```

```

* Fill a rectangle of an image with a color.

```

```

*

```

```

* @pre image != NULL

```

```

* @param color - new color for the pixels

```

```

* @param x0 - left position of rectangle in image

```

```

* @param y0 - top position of rectangle in image

```

```

* @param width - width of rectangle in image

```

```

* @param height - height of rectangle in image

```

```

*/

```

```

extern void fillScreenRect(Color color, int x0, int y0, int width, int height);

```

```

/**

```

```

* Set a pixel on screen to the specified color.

```

```

*

```

```

* @pre x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT

```

```

* @param color - new color for the pixels

```

```

* @param x - left position of the pixel

```

```

* @param y - top position of the pixel

```

```

*/

```

```

extern void putPixelScreen(Color color, int x, int y);

```

```

/**

```

```

* Set a pixel in an image to the specified color.

```

```

*

```

```

* @pre x >= 0 && x < image->imageWidth && y >= 0 && y < image->imageHeight && image !=
    NULL

```

```

* @param color - new color for the pixels

```

```

* @param x - left position of the pixel

```

```

* @param y - top position of the pixel

```

```

*/

```

```

extern void putPixelImage(Color color, int x, int y, Image* image);

```

```

/**

```

```

* Get the color of a pixel on screen.

```

```

*

```

```

* @pre x >= 0 && x < SCREEN_WIDTH && y >= 0 && y < SCREEN_HEIGHT

```



```

* @param x - left position of the pixel
* @param y - top position of the pixel
* @return the color of the pixel
*/
extern Color getPixelScreen(int x, int y);

/**
* Get the color of a pixel of an image.
*
* @pre x >= 0 && x < image->imageWidth && y >= 0 && y < image->imageHeight && image != NULL
* @param x - left position of the pixel
* @param y - top position of the pixel
* @return the color of the pixel
*/
extern Color getPixelImage(int x, int y, Image* image);

/**
* Print a text (pixels out of the screen or image are clipped).
*
* @param x - left position of text
* @param y - top position of text
* @param text - the text to print
* @param color - new color for the pixels
*/
extern void printTextScreen(int x, int y, const char* text, u32 color);

/**
* Print a text (pixels out of the screen or image are clipped).
*
* @param x - left position of text
* @param y - top position of text
* @param text - the text to print
* @param color - new color for the pixels
* @param image - image
*/
extern void printTextImage(int x, int y, const char* text, u32 color, Image* image);

/**
* Save an image or the screen in PNG format.
*
* @pre filename != NULL
* @param filename - filename of the PNG image
* @param data - start of Color type pixel data (can be getVramDisplayBuffer())
* @param width - logical width of the image or SCREEN_WIDTH
* @param height - height of the image or SCREEN_HEIGHT
* @param lineSize - physical width of the image or PSP_LINE_SIZE
* @param saveAlpha - if 0, image is saved without alpha channel
*/
extern void saveImage(const char* filename, Color* data, int width, int height, int lineSize, int saveAlpha);

/**
* Exchange display buffer and drawing buffer.
*/
extern void flipScreen();

/**
* Initialize the graphics.
*/
extern void initGraphics();

/**
* Disable graphics, used for debug text output.
*/
extern void disableGraphics();

```



```
/**
 * Draw a line to screen.
 *
 * @pre x0 >= 0 && x0 < SCREEN_WIDTH && y0 >= 0 && y0 < SCREEN_HEIGHT &&
 *       x1 >= 0 && x1 < SCREEN_WIDTH && y1 >= 0 && y1 < SCREEN_HEIGHT
 * @param x0 - x line start position
 * @param y0 - y line start position
 * @param x1 - x line end position
 * @param y1 - y line end position
 */
void drawLineScreen(int x0, int y0, int x1, int y1, Color color);

/**
 * Draw a line to screen.
 *
 * @pre x0 >= 0 && x0 < image->imageWidth && y0 >= 0 && y0 < image->imageHeight &&
 *       x1 >= 0 && x1 < image->imageWidth && y1 >= 0 && y1 < image->imageHeight
 * @param x0 - x line start position
 * @param y0 - y line start position
 * @param x1 - x line end position
 * @param y1 - y line end position
 */
extern void drawLineImage(int x0, int y0, int x1, int y1, Color color, Image* image);

/**
 * Get the current draw buffer for fast unchecked access.
 *
 * @return the start address of the current draw buffer
 */
extern Color* getVramDrawBuffer();

/**
 * Get the current display buffer for fast unchecked access.
 *
 * @return the start address of the current display buffer
 */
extern Color* getVramDisplayBuffer();

extern void guStart();

#endif
```



```
// Final Driver for Dijkstra Program
// Dru Sepulveda
// 10-29-06
// Runs from Index
```

```
#include "Index.h"
```

```
int main(){
    initAll();
    return 0;
}
```



```
#ifndef FRAMEBUFFER_H
#define FRAMEBUFFER_H

#include <psptypes.h>

extern u32* g_vram_base;

#endif
```



```
#include <psptypes.h>
#include "graphics.h"
```

```
Color* g_vram_base = (Color*) (0x40000000 | 0x04000000);
```



```
#include <stdlib.h>
#include <malloc.h>
#include <pspdisplay.h>
#include <psputils.h>
#include <png.h>
#include <pspgu.h>

#include "graphics.h"
#include "framebuffer.h"

#define IS_ALPHA(color) (((color)&0xff000000)==0xff000000?0:1)
#define FRAMEBUFFER_SIZE (PSP_LINE_SIZE*SCREEN_HEIGHT*4)
#define MAX(X, Y) ((X) > (Y) ? (X) : (Y))

typedef struct
{
    unsigned short u, v;
    short x, y, z;
} Vertex;

extern u8 msx[];

unsigned int __attribute__((aligned(16))) list[262144];
static int dispBufferNumber;
static int initialized = 0;

static int getNextPower2(int width)
{
    int b = width;
    int n;
    for (n = 0; b != 0; n++) b >>= 1;
    b = 1 << n;
    if (b == 2 * width) b >>= 1;
    return b;
}

Color* getVramDrawBuffer()
{
    Color* vram = (Color*) g_vram_base;
    if (dispBufferNumber == 0) vram += FRAMEBUFFER_SIZE / sizeof(Color);
    return vram;
}

Color* getVramDisplayBuffer()
{
    Color* vram = (Color*) g_vram_base;
    if (dispBufferNumber == 1) vram += FRAMEBUFFER_SIZE / sizeof(Color);
    return vram;
}

void user_warning_fn(png_structp png_ptr, png_const_charp warning_msg)
{
}

Image* loadImage(const char* filename)
{
    png_structp png_ptr;
    png_infop info_ptr;
    unsigned int sig_read = 0;
    png_uint_32 width, height;
    int bit_depth, color_type, interlace_type, x, y;
    u32* line;
    FILE *fp;
    Image* image = (Image*) malloc(sizeof(Image));
    if (!image) return NULL;
```



```

    if ((fp = fopen(filename, "rb")) == NULL) return NULL;
    png_ptr = png_create_read_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
    if (png_ptr == NULL) {
        free(image);
        fclose(fp);
        return NULL;
    }
    png_set_error_fn(png_ptr, (png_voidp) NULL, (png_error_ptr) NULL, user_warning_fn);
    info_ptr = png_create_info_struct(png_ptr);
    if (info_ptr == NULL) {
        free(image);
        fclose(fp);
        png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
        return NULL;
    }
    png_init_io(png_ptr, fp);
    png_set_sig_bytes(png_ptr, sig_read);
    png_read_info(png_ptr, info_ptr);
    png_get_IHDR(png_ptr, info_ptr, &width, &height, &bit_depth, &color_type, &
    interlace_type, int_p_NULL, int_p_NULL);
    if (width > 512 || height > 512) {
        free(image);
        fclose(fp);
        png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
        return NULL;
    }
    image->imageWidth = width;
    image->imageHeight = height;
    image->textureWidth = getNextPower2(width);
    image->textureHeight = getNextPower2(height);
    png_set_strip_16(png_ptr);
    png_set_packing(png_ptr);
    if (color_type == PNG_COLOR_TYPE_PALETTE) png_set_palette_to_rgb(png_ptr);
    if (color_type == PNG_COLOR_TYPE_GRAY && bit_depth < 8) png_set_gray_1_2_4_to_8
    (png_ptr);
    if (png_get_valid(png_ptr, info_ptr, PNG_INFO_tRNS)) png_set_tRNS_to_alpha(png_ptr);
    png_set_filler(png_ptr, 0xff, PNG_FILLER_AFTER);
    image->data = (Color*) memalign(16, image->textureWidth * image->textureHeight *
    sizeof(Color));
    if (!image->data) {
        free(image);
        fclose(fp);
        png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
        return NULL;
    }
    line = (u32*) malloc(width * 4);
    if (!line) {
        free(image->data);
        free(image);
        fclose(fp);
        png_destroy_read_struct(&png_ptr, png_infopp_NULL, png_infopp_NULL);
        return NULL;
    }
    for (y = 0; y < height; y++) {
        png_read_row(png_ptr, (u8*) line, png_bytep_NULL);
        for (x = 0; x < width; x++) {
            u32 color = line[x];
            image->data[x + y * image->textureWidth] = color;
        }
    }
    free(line);
    png_read_end(png_ptr, info_ptr);
    png_destroy_read_struct(&png_ptr, &info_ptr, png_infopp_NULL);
    fclose(fp);
    return image;
}

```



```

void blitImageToImage(int sx, int sy, int width, int height, Image* source, int dx, int
dy, Image* destination)
{
    Color* destinationData = &destination->data[destination->textureWidth * dy + dx];
    int destinationSkipX = destination->textureWidth - width;
    Color* sourceData = &source->data[source->textureWidth * sy + sx];
    int sourceSkipX = source->textureWidth - width;
    int x, y;
    for (y = 0; y < height; y++, destinationData += destinationSkipX, sourceData +=
sourceSkipX) {
        for (x = 0; x < width; x++, destinationData++, sourceData++) {
            *destinationData = *sourceData;
        }
    }
}

void blitImageToScreen(int sx, int sy, int width, int height, Image* source, int dx, int
dy)
{
    if (!initialized) return;
    Color* vram = getVramDrawBuffer();
    sceKernelDcacheWritebackInvalidateAll();
    guStart();
    sceGuCopyImage(GU_PSM_8888, sx, sy, width, height, source->textureWidth, source->data
, dx, dy, PSP_LINE_SIZE, vram);
    sceGuFinish();
    sceGuSync(0,0);
}

void blitAlphaImageToImage(int sx, int sy, int width, int height, Image* source, int dx,
int dy, Image* destination)
{
    // TODO Blend!
    Color* destinationData = &destination->data[destination->textureWidth * dy + dx];
    int destinationSkipX = destination->textureWidth - width;
    Color* sourceData = &source->data[source->textureWidth * sy + sx];
    int sourceSkipX = source->textureWidth - width;
    int x, y;
    for (y = 0; y < height; y++, destinationData += destinationSkipX, sourceData +=
sourceSkipX) {
        for (x = 0; x < width; x++, destinationData++, sourceData++) {
            Color color = *sourceData;
            if (!IS_ALPHA(color)) *destinationData = color;
        }
    }
}

void blitAlphaImageToScreen(int sx, int sy, int width, int height, Image* source, int dx,
int dy)
{
    if (!initialized) return;

    sceKernelDcacheWritebackInvalidateAll();
    guStart();
    sceGuTexImage(0, source->textureWidth, source->textureHeight, source->textureWidth,
(void*) source->data);
    float u = 1.0f / ((float)source->textureWidth);
    float v = 1.0f / ((float)source->textureHeight);
    sceGuTexScale(u, v);

    int j = 0;
    while (j < width) {
        Vertex* vertices = (Vertex*) sceGuGetMemory(2 * sizeof(Vertex));
        int sliceWidth = 64;
        if (j + sliceWidth > width) sliceWidth = width - j;
    }
}

```



```
    vertices[0].u = sx + j;
    vertices[0].v = sy;
    vertices[0].x = dx + j;
    vertices[0].y = dy;
    vertices[0].z = 0;
    vertices[1].u = sx + j + sliceWidth;
    vertices[1].v = sy + height;
    vertices[1].x = dx + j + sliceWidth;
    vertices[1].y = dy + height;
    vertices[1].z = 0;
    sceGuDrawArray(GU_SPRITES, GU_TEXTURE_16BIT | GU_VERTEX_16BIT | GU_TRANSFORM_2D, 2, 0, vertices);
    j += sliceWidth;
}

sceGuFinish();
sceGuSync(0, 0);
}

Image* createImage(int width, int height)
{
    Image* image = (Image*) malloc(sizeof(Image));
    if (!image) return NULL;
    image->imageWidth = width;
    image->imageHeight = height;
    image->textureWidth = getNextPower2(width);
    image->textureHeight = getNextPower2(height);
    image->data = (Color*) memalign(16, image->textureWidth * image->textureHeight * sizeof(Color));
    if (!image->data) return NULL;
    memset(image->data, 0, image->textureWidth * image->textureHeight * sizeof(Color));
    return image;
}

void freeImage(Image* image)
{
    free(image->data);
    free(image);
}

void clearImage(Color color, Image* image)
{
    int i;
    int size = image->textureWidth * image->textureHeight;
    Color* data = image->data;
    for (i = 0; i < size; i++, data++) *data = color;
}

void clearScreen(Color color)
{
    if (!initialized) return;
    guStart();
    sceGuClearDepth(0);
    sceGuClear(GU_COLOR_BUFFER_BIT | GU_DEPTH_BUFFER_BIT);
    sceGuFinish();
    sceGuSync(0, 0);
}

void fillImageRect(Color color, int x0, int y0, int width, int height, Image* image)
{
    int skipX = image->textureWidth - width;
    int x, y;
    Color* data = image->data + x0 + y0 * image->textureWidth;
    for (y = 0; y < height; y++, data += skipX) {
        for (x = 0; x < width; x++, data++) *data = color;
    }
}
```



```
}

void fillScreenRect(Color color, int x0, int y0, int width, int height)
{
    if (!initialized) return;
    int skipX = PSP_LINE_SIZE - width;
    int x, y;
    Color* data = getVramDrawBuffer() + x0 + y0 * PSP_LINE_SIZE;
    for (y = 0; y < height; y++, data += skipX) {
        for (x = 0; x < width; x++, data++) *data = color;
    }
}

void putPixelScreen(Color color, int x, int y)
{
    Color* vram = getVramDrawBuffer();
    vram[PSP_LINE_SIZE * y + x] = color;
}

void putPixelImage(Color color, int x, int y, Image* image)
{
    image->data[x + y * image->textureWidth] = color;
}

Color getPixelScreen(int x, int y)
{
    Color* vram = getVramDrawBuffer();
    return vram[PSP_LINE_SIZE * y + x];
}

Color getPixelImage(int x, int y, Image* image)
{
    return image->data[x + y * image->textureWidth];
}

void printTextScreen(int x, int y, const char* text, u32 color)
{
    int c, i, j, l;
    u8 *font;
    Color *vram_ptr;
    Color *vram;

    if (!initialized) return;

    for (c = 0; c < strlen(text); c++) {
        if (x < 0 || x + 8 > SCREEN_WIDTH || y < 0 || y + 8 > SCREEN_HEIGHT) break;
        char ch = text[c];
        vram = getVramDrawBuffer() + x + y * PSP_LINE_SIZE;

        font = &msx[ (int)ch * 8];
        for (i = l = 0; i < 8; i++, l += 8, font++) {
            vram_ptr = vram;
            for (j = 0; j < 8; j++) {
                if ((*font & (128 >> j))) *vram_ptr = color;
                vram_ptr++;
            }
            vram += PSP_LINE_SIZE;
        }
        x += 8;
    }
}

void printTextImage(int x, int y, const char* text, u32 color, Image* image)
{
    int c, i, j, l;
    u8 *font;
```




```

    Color *data_ptr;
    Color *data;

    if (!initialized) return;

    for (c = 0; c < strlen(text); c++) {
        if (x < 0 || x + 8 > image->imageWidth || y < 0 || y + 8 > image->imageHeight)  ✎
        break;
        char ch = text[c];
        data = image->data + x + y * image->textureWidth;

        font = &msx[ (int)ch * 8];
        for (i = l = 0; i < 8; i++, l += 8, font++) {
            data_ptr = data;
            for (j = 0; j < 8; j++) {
                if ((*font & (128 >> j))) *data_ptr = color;
                data_ptr++;
            }
            data += image->textureWidth;
        }
        x += 8;
    }
}

void saveImage(const char* filename, Color* data, int width, int height, int lineSize,  ✎
int saveAlpha)
{
    png_structp png_ptr;
    png_infop info_ptr;
    FILE* fp;
    int i, x, y;
    u8* line;

    if ((fp = fopen(filename, "wb")) == NULL) return;
    png_ptr = png_create_write_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
    if (!png_ptr) return;
    info_ptr = png_create_info_struct(png_ptr);
    if (!info_ptr) {
        png_destroy_write_struct(&png_ptr, (png_infopp)NULL);
        return;
    }
    png_init_io(png_ptr, fp);
    png_set_IHDR(png_ptr, info_ptr, width, height, 8,
        saveAlpha ? PNG_COLOR_TYPE_RGBA : PNG_COLOR_TYPE_RGB,
        PNG_INTERLACE_NONE, PNG_COMPRESSION_TYPE_DEFAULT, PNG_FILTER_TYPE_DEFAULT);
    png_write_info(png_ptr, info_ptr);
    line = (u8*) malloc(width * (saveAlpha ? 4 : 3));
    for (y = 0; y < height; y++) {
        for (i = 0, x = 0; x < width; x++) {
            Color color = data[x + y * lineSize];
            u8 r = color & 0xff;
            u8 g = (color >> 8) & 0xff;
            u8 b = (color >> 16) & 0xff;
            u8 a = saveAlpha ? (color >> 24) & 0xff : 0xff;
            line[i++] = r;
            line[i++] = g;
            line[i++] = b;
            if (saveAlpha) line[i++] = a;
        }
        png_write_row(png_ptr, line);
    }
    free(line);
    png_write_end(png_ptr, info_ptr);
    png_destroy_write_struct(&png_ptr, (png_infopp)NULL);
    fclose(fp);
}

```



```

void flipScreen()
{
    if (!initialized) return;
    sceGuSwapBuffers();
    dispBufferNumber ^= 1;
}

static void drawLine(int x0, int y0, int x1, int y1, int color, Color* destination, int width)
{
    int dy = y1 - y0;
    int dx = x1 - x0;
    int stepx, stepy;

    if (dy < 0) { dy = -dy; stepy = -width; } else { stepy = width; }
    if (dx < 0) { dx = -dx; stepx = -1; } else { stepx = 1; }
    dy <<= 1;
    dx <<= 1;

    y0 += width;
    y1 += width;
    destination[x0+y0] = color;
    if (dx > dy) {
        int fraction = dy - (dx >> 1);
        while (x0 != x1) {
            if (fraction >= 0) {
                y0 += stepy;
                fraction -= dx;
            }
            x0 += stepx;
            fraction += dy;
            destination[x0+y0] = color;
        }
    } else {
        int fraction = dx - (dy >> 1);
        while (y0 != y1) {
            if (fraction >= 0) {
                x0 += stepx;
                fraction -= dy;
            }
            y0 += stepy;
            fraction += dx;
            destination[x0+y0] = color;
        }
    }
}

void drawLineScreen(int x0, int y0, int x1, int y1, Color color)
{
    drawLine(x0, y0, x1, y1, color, getVramDrawBuffer(), PSP_LINE_SIZE);
}

void drawLineImage(int x0, int y0, int x1, int y1, Color color, Image* image)
{
    drawLine(x0, y0, x1, y1, color, image->data, image->textureWidth);
}

#define BUF_WIDTH (512)
#define SCR_WIDTH (480)
#define SCR_HEIGHT (272)
#define PIXEL_SIZE (4) /* change this if you change to another screenmode */
#define FRAME_SIZE (BUF_WIDTH * SCR_HEIGHT * PIXEL_SIZE)
#define ZBUF_SIZE (BUF_WIDTH SCR_HEIGHT * 2) /* zbuffer seems to be 16-bit? */

void initGraphics()

```



```
{
    dispBufferNumber = 0;

    sceGuInit();

    guStart();
    sceGuDrawBuffer(GU_PSM_8888, (void*)FRAMEBUFFER_SIZE, PSP_LINE_SIZE);
    sceGuDispBuffer(SCREEN_WIDTH, SCREEN_HEIGHT, (void*)0, PSP_LINE_SIZE);
    sceGuClear(GU_COLOR_BUFFER_BIT | GU_DEPTH_BUFFER_BIT);
    sceGuDepthBuffer((void*) (FRAMEBUFFER_SIZE*2), PSP_LINE_SIZE);
    sceGuOffset(2048 - (SCREEN_WIDTH / 2), 2048 - (SCREEN_HEIGHT / 2));
    sceGuViewport(2048, 2048, SCREEN_WIDTH, SCREEN_HEIGHT);
    sceGuDepthRange(0xc350, 0x2710);
    sceGuScissor(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);
    sceGuEnable(GU_SCISSOR_TEST);
    sceGuAlphaFunc(GU_GREATER, 0, 0xff);
    sceGuEnable(GU_ALPHA_TEST);
    sceGuDepthFunc(GU_EQUAL);
    sceGuEnable(GU_DEPTH_TEST);
    sceGuFrontFace(GU_CW);
    sceGuShadeModel(GU_SMOOTH);
    sceGuEnable(GU_CULL_FACE);
    sceGuEnable(GU_TEXTURE_2D);
    sceGuEnable(GU_CLIP_PLANES);
    sceGuTexMode(GU_PSM_8888, 0, 0, 0);
    sceGuTexFunc(GU_TFX_REPLACE, GU_TCC_RGBA);
    sceGuTexFilter(GU_NEAREST, GU_NEAREST);
    sceGuAmbientColor(0xffffffff);
    sceGuEnable(GU_BLEND);
    sceGuBlendFunc(GU_ADD, GU_SRC_ALPHA, GU_ONE_MINUS_SRC_ALPHA, 0, 0);
    sceGuFinish();
    sceGuSync(0, 0);

    sceDisplayWaitVblankStart();
    sceGuDisplay(GU_TRUE);
    initialized = 1;
}

void disableGraphics()
{
    initialized = 0;
}

void guStart()
{
    sceGuStart(GU_DIRECT, list);
}
```


Original Plans in Manuscript

www.pspupdates.com

KXploit

~~Homebrew~~

1. install cyswlr make sure to include
libroes. for or no compile.

windows
2. ~~make~~ make , gcc , g++
include

3. devkitpro

~~at~~

COINS 330
Computer Architecture
COURSE OUTLINE

Turn in the homework on the date due! Late assignments are heavily penalized (see section F).

Week	Date	Topic	Lab
1	7/5~7/6	Overview of Networking	
1	7/7~7/8	Application Layer / Browser	L1:
2	7/11~7/12	XHTML	L2:
2	7/13~7/15	XHTML/CSS	L3:
3	7/18~7/22	JavaScript/ Test 1 [7/22]	L4: L5:
4	7/25~26	Transport / Network Layer	L6:
4	7/27	XML	L7:
4	7/28	Webservers GF [3/25]	L8:
4	7/29	Database / Test 2 [4/1]	
5	8/1~8/4	ASP	L10: L11:
	8/5	Final Exam at 9 AM	

Schedule and topics are subject to change.

Promoting Academic Excellence in a Christian Environment

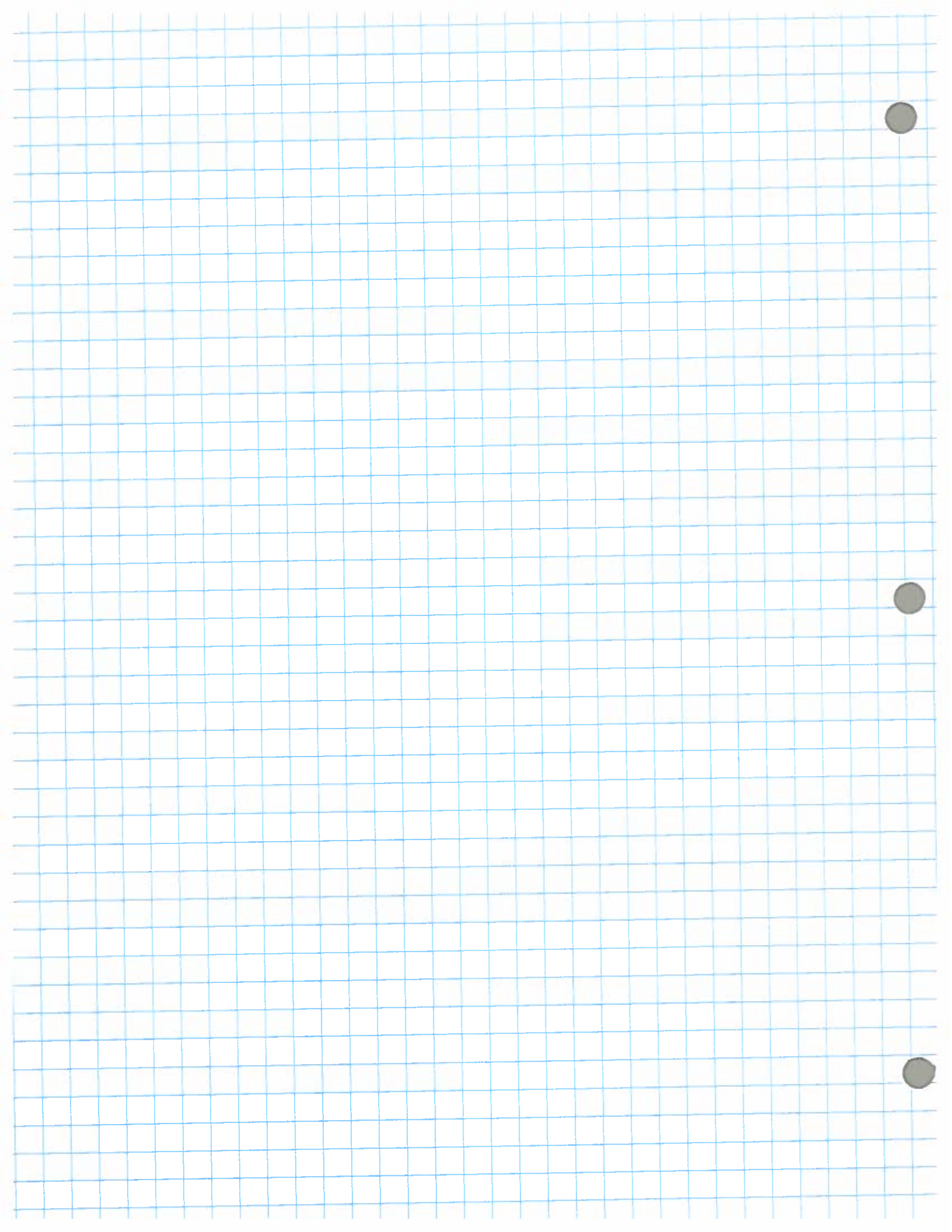
9200 UNIVERSITY BOULEVARD POST OFFICE BOX 118087 CHARLESTON, SOUTH CAROLINA 29423-8087
 WWW.CharlestonSouthern.edu • Phone (843) 863-8091/7529 • FAX (843) 863-7290 • jbriner@csuniv.edu

1. Build the data structure for graph

2. Build - - - - - for greedy algorithm

3. Build the graph

4. Algorithm you will use



- Graph datastructure
 - - Build an array (2-D)
 - - fill in necessary data to ^{the} array

2. Shortest path

- study algorithm

- Come up with your own algorithm → see flowchart

- adapt to your data structure
- code it

3. PSP Programming

- port VC++ version to PSP

- graphical user interface

- IR

4. Testing

- Benchmarks

5. Presentation

